

PUREBREATH-AI: AN EXPLAINABLE AND PERSONALIZED FRAMEWORK FOR AQI FORECASTING, EXPOSURE TRACKING AND HEALTH-RISK ASSESSMENT

SURYA J - 23619118

KARUN C - 23619119

SANJAY S - 23619133

VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)

PROJECT PHASE 1 REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

2026

BONAFIDE CERTIFICATE

Certified that this project report **"PUREBREATH-AI: AN EXPLAINABLE AND PERSONALIZED FRAMEWORK FOR AQI FORECASTING, EXPOSURE TRACKING AND HEALTH-RISK ASSESSMENT",** is the Bonafide work of **SURYA J (23619118), KARUN C (23619119), SANJAY S (23619133)** carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya M.E., Ph.D

Associate Professor & Head of
Department of Artificial Intelligence
& Machine Learning
School of Engineering
VISTAS, Chennai

SIGNATURE OF THE SUPERVISOR

Dr. K. Ulaga Priya M.E., Ph.D

Associate Professor & Head of
Department of Artificial Intelligence
& Machine Learning
School of Engineering
VISTAS, Chennai

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Dr. Ishari K. Ganesh**, and it is with great pleasure that we take this opportunity to thank him.

We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya**, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE, Dr. K. Ulaga Priya, Associate Professor & Head of the Department of AI & ML**, who has patiently guided us and provided invaluable advice and help throughout the work. We are thankful to our **Project Coordinator, Ms. S.J. Kavitha** We would like to express our greatest gratitude to our staff members, Librarian, and non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the least, we wish to thank our parents and friends for their individual support and interest, who inspired and encouraged us to go our own way; without them we would be unable to complete this project.

SURYA J - 23619118

KARUN C - 23619119

SANJAY S -23619133

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii
	ABBREVIATIONS	iii
1	**INTRODUCTION**	**1**
	1.1 Overview of the project	1
	1.2 Scope and objective	3
2	**LITERATURE REVIEW**	**7**
	2.1 Literature overview	7
	2.2 Literature survey	7
	2.3 Comparative analysis with the surveyed work	10
	2.4 Research gap	12
3	**SYSTEM DESIGN**	**13**
	3.1 Existing system	13
	3.2 Proposed system	13
	3.3 System architecture	14
	3.4 Hardware requirements	17
	3.5 Software requirements	17
	3.6 Non-functional requirements and design considerations	18
4	**IMPLEMENTATION**	**19**
	4.1 Methodology	19
	4.2 Implementation	21
	4.2.1 Data Preprocessing Module	21
	4.2.2 GAN Augmentation Module	21
	4.2.3 Forecasting Module	22
	4.2.4 Explainable AI Module	23
	4.2.5 Personalized Health-Risk Module	23
	4.2.6 Cumulative Exposure Module	24
	4.2.7 API and User Interface Module	25
	4.2.8 Execution Flow	25
	4.2.9 Design Rationale	26
5	**CONCLUSION AND FUTURE WORK**	**27**
	5.1 Conclusion	27
	5.2 Future work	28
	5.3 Source code	29
	5.4 Results	75
	5.5 User interface outputs	86
	REFERENCES	91

ABSTRACT

Most air-quality applications still communicate only the ambient pollution level or the air quality index (AQI) and do not explain what it means for an individual. Two people in the same city can have very different real exposures, depending on the time they spend outdoors, their occupation and their health. This project presents PureBreath-AI, a working environmental-health system that takes historical CPCB air-quality data and weather data for five Indian cities (Delhi, Mumbai, Chennai, Kolkata and Bangalore) and runs it through preprocessing, conditional-GAN class balancing, AQI forecasting, SHAP-based explanation with counterfactual recommendations, personalized health-risk scoring, cumulative-exposure tracking and tiered alerting, all served through a FastAPI back end and a React dashboard.

The forecasting stage compares XGBoost, LightGBM and Random Forest regressors on 55 engineered features. On held-out data LightGBM is the best, with an MAE of 1.71, an RMSE of 2.37, an R^2 of 0.9996 and a MAPE of 0.58%, which is 52.7% lower MAE and 64.9% lower RMSE than the Random Forest baseline. SHAP attributes 98.6% of the model's attribution to PM2.5 and PM10, and counterfactual advice (a 20% reduction of the top three drivers) lowered the predicted AQI in 162 of 200 evaluated cases. A rule-based engine grounded in WHO and CPCB guidance combines age, occupation, outdoor hours, asthma and pregnancy into a 0–100 risk score, giving mean scores from 35.2 (Bangalore and Chennai) to 93.7 (Delhi), and a seven-day dose model raises Safe, Moderate, High or Critical alerts. The report compares the system with fifteen cited studies and distinguishes quantitatively validated components from rule-based and planned ones.

Keywords

Air quality prediction, explainable artificial intelligence, SHAP, counterfactual explanation, conditional GAN, LightGBM, personalized exposure, health-risk assessment, PM2.5, CPCB AQI.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 1	Composition of the GAN-augmented training set (10,960 real and 3,000 synthetic records)	2
Figure 2	Composition of the 55-feature input space	5
Figure 3	CPCB sub-index curves for PM2.5 and PM10 used to build the AQI target	6
Figure 4	Capability heat-map of research directions versus PureBreath-AI	10
Figure 5	Capability coverage score (maximum 12)	11
Figure 6	Reported R^2 of PureBreath-AI and of published studies (indicative only)	12
Figure 7	System architecture of PureBreath-AI	15
Figure 8	Age and occupation multipliers of the risk engine	24
Figure 9	Daily and weekly dose thresholds for the four alert tiers	25
Figure 10	Held-out MAE, RMSE, R^2 and MAPE of the three forecasting models	76
Figure 11	Error reduction achieved by LightGBM relative to the Random Forest and XGBoost models	76
Figure 12	Unexplained variance ($1 - R^2$) of the three models on a logarithmic scale	77
Figure 13	Top fifteen features by mean absolute SHAP value (LightGBM, 200-row sample)	78
Figure 14	Share of total mean absolute SHAP attribution	78
Figure 15	Predicted AQI before and after the counterfactual, and the distribution of AQI reduction	79
Figure 16	Counterfactual outcomes on 200 cases and AQI category transitions	79
Figure 17	Features most frequently targeted by the counterfactual reasoner	80
Figure 18	Base AQI of the five cities and their seven-day AQI profiles	80
Figure 19	Distribution of personal risk scores in each city (100 users per city)	81
Figure 20	Risk-tier distribution of users in each city	81
Figure 21	Risk score of five personas across the CPCB AQI categories (computed from the risk formula)	82
Figure 22	Weekly cumulative dose and alert-tier distribution by city	83
Figure 23	Weekly dose against daily outdoor hours for a healthy adult in each city	83
Figure 24	Dashboard: city cards, Delhi overview, AQI-versus-risk chart and multi-metric radar	87
Figure 25	Predict AQI page: pollutant conditions, 7-day forecast, SHAP features and counterfactual scenario	88
Figure 26	Exposure page: profile inputs, weekly dose, alert level, daily dose trend and breakdown	89
Figure 27	Health Risk page: AI summary, health effects and personalised recommendations	90

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
API	Application Programming Interface
AQHI	Air Quality Health Index
AQI	Air Quality Index
CORS	Cross-Origin Resource Sharing
CPCB	Central Pollution Control Board
CPU	Central Processing Unit
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
GAN / cGAN	Generative Adversarial Network / Conditional GAN
GBDT	Gradient Boosting Decision Tree
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
IoT	Internet of Things
JSON	JavaScript Object Notation
LIME	Local Interpretable Model-agnostic Explanations
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
PM2.5 / PM10	Particulate Matter below 2.5 / 10 micrometres
PSO	Particle Swarm Optimization
R^2	Coefficient of Determination
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SHAP	SHapley Additive exPlanations
UI	User Interface
VMD	Variational Mode Decomposition
WHO	World Health Organization
XAI	Explainable Artificial Intelligence
XGBoost	Extreme Gradient Boosting

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Air pollution is one of the most persistent environmental health risks in urban India. Fine particulate matter (PM2.5) and coarse particulate matter (PM10) penetrate the respiratory system, and gaseous pollutants such as nitrogen dioxide (NO2), sulphur dioxide (SO2), carbon monoxide (CO) and ozone (O3) add to the burden. Most public air-quality applications report only the ambient concentration or a single Air Quality Index (AQI) for a city. That number describes the air, not the person breathing it. Two people living in the same neighbourhood can receive very different doses of pollution depending on their age, occupation, pre-existing conditions and the number of hours they spend outdoors.

PureBreath-AI is a working environmental-health system that closes this gap. It takes historical CPCB air-quality data and gridded weather data for five Indian cities (Delhi, Mumbai, Chennai, Kolkata and Bangalore) and passes them through a chain of connected modules: data preprocessing, conditional-GAN class balancing, multi-model AQI forecasting, SHAP-based explanation with counterfactual recommendations, personalized health-risk scoring, cumulative-exposure tracking and tiered alerting. The modules are served through a FastAPI backend and presented through a React dashboard, so a user can enter a profile and immediately see what AQI to expect, why the model expects it, how a behavioural change would alter it, what that means for their own health, and how their exposure builds up over a week.

1.1.1 Motivation

Forecasting research has matured considerably. Gradient boosting, XGBoost, LightGBM and stacked ensembles now predict AQI with very small error, and explainability research has begun to open the black box using SHAP and LIME. Separately, studies of personal exposure and cumulative PM2.5 burden have started to link pollution to health outcomes. Each of these directions is strong on its own, but they are rarely combined into one deployed pipeline. A citizen who simply wants to know whether it is safe to commute, exercise or send a child to school cannot use a forecasting paper, an explainability plot and a cumulative-exposure study separately. PureBreath-AI was built to give that citizen one integrated answer.

1.1.2 Problem Statement

Existing air-quality tools communicate ambient pollution but do not translate it into personal risk, do not explain why a forecast was made, and do not track how exposure accumulates over time. The problem addressed in this project is to design, implement and evaluate an end-to-end, explainable and personalized AQI decision-support system for Indian cities that (i) forecasts AQI accurately from pollutant and meteorological data, (ii) explains each forecast in plain language, (iii) converts the forecast into an individual risk score, and (iv) tracks cumulative exposure with actionable alerts.

1.1.3 Overview of the Approach

The system is organised as a pipeline of seven stages. Raw CPCB daily pollutant records and a gridded meteorological dataset are merged on a City-Date key, cleaned and enriched with calendar, lag and rolling-window features. A conditional GAN generates 3,000 synthetic records (500 per AQI class) to rebalance under-represented classes. Three regressors (XGBoost, LightGBM and Random Forest) are trained and compared, and LightGBM is selected as the primary model. A SHAP TreeExplainer attributes each prediction to its input features, and a counterfactual reasoner reduces the top three positive drivers by 20% to show how the predicted AQI would respond. A transparent, rule-based engine combines the AQI category with age, occupation, outdoor hours, asthma and pregnancy to produce a 0–100 risk score, and a dose model accumulates AQI × outdoor hours × vulnerability over a seven-day window to trigger Safe, Moderate, High or Critical alerts.

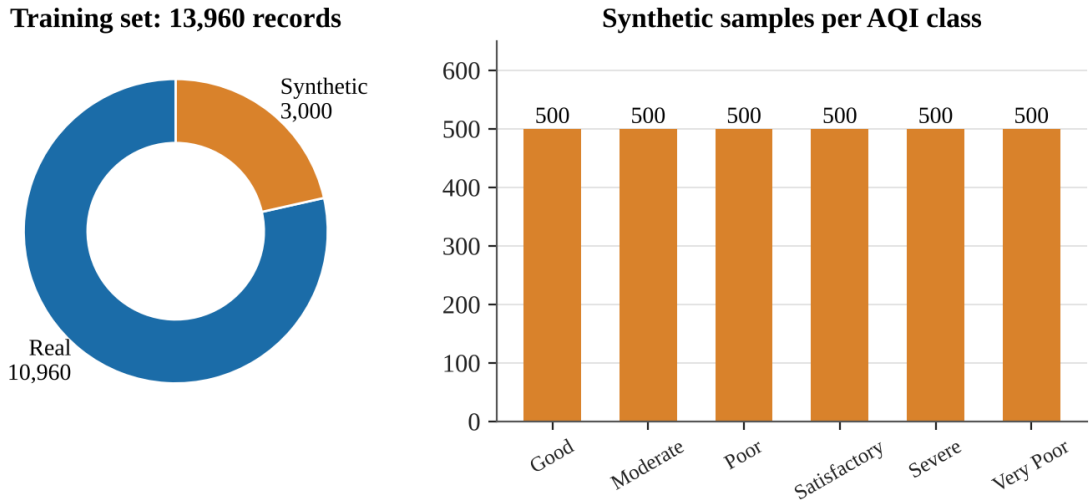


Figure 1 – Composition of the GAN-augmented training set (10,960 real and 3,000 synthetic records)

1.1.4 Key Contributions

- An integrated pipeline that links preprocessing, GAN-based class balancing, forecasting, SHAP explanation, counterfactual recommendation, personalized risk scoring, cumulative-dose alerting, an API and a dashboard, built on real CPCB and weather data for five Indian cities.
- A forecasting engine whose best model (LightGBM) reaches an MAE of 1.71, an RMSE of 2.37 and an R^2 of 0.9996 on held-out data, with 52.7% lower MAE and 64.9% lower RMSE than the Random Forest baseline.
- An explainability layer that pairs SHAP attribution with counterfactual advice; on a 200-case evaluation set the advice lowered predicted AQI in 162 cases (81%), and in the 101 high-AQI cases (AQI $\geq$ 200) the mean reduction was 58.6 points.
- A transparent personalized exposure and health-risk engine whose scores range from 35.2 (Bangalore and Chennai) to 93.7 (Delhi) across 500 user-city assessments.
- A clear statement of which components are quantitatively validated, which are rule-based and not yet clinically validated, and which remain planned.

1.1.5 Organisation of the Report

Chapter 1 introduces the problem, scope and objectives. Chapter 2 reviews the literature and compares fifteen related studies with PureBreath-AI. Chapter 3 describes the existing and proposed systems, the architecture, and the hardware and software requirements. Chapter 4 presents the methodology and the implementation of each module. Chapter 5 concludes the report, lists future work, reproduces the key source code and presents the results with supporting graphs and a comparison against published work.

1.2 SCOPE AND OBJECTIVE

1.2.1 Scope

The scope of the project covers batch processing of historical daily air-quality and weather records for five Indian metropolitan cities, and the full chain of analysis built on top of them. Within this scope the system forecasts a CPCB-standard AQI, explains each forecast, estimates individual health risk, accumulates exposure over seven days and raises alerts. The user interface is a web dashboard, and the application layer is a REST API that other front ends could also call.

The following items are explicitly outside the current scope and are listed as future work in Chapter 5: live sensor or IoT data feeds, wearable-based personal exposure measurement, true multi-day forecasting for the exposure module, clinical validation of the risk weights, and deployment beyond the five study cities. The risk scores are decision-support guidance and must not be read as medical diagnosis.

1.2.2 Objectives

1. To clean, merge and enrich CPCB pollutant data and gridded weather data for Delhi, Mumbai, Chennai, Kolkata and Bangalore into a single model-ready dataset.
2. To address AQI class imbalance by generating class-conditioned synthetic samples with a conditional GAN.
3. To train and compare XGBoost, LightGBM and Random Forest regressors for CPCB AQI prediction using MAE, RMSE, R^2 and MAPE.
4. To explain every prediction with SHAP values and to turn those values into counterfactual, plain-language recommendations.
5. To compute a personalized 0–100 health-risk score from age, occupation, outdoor hours, asthma and pregnancy, with advice appropriate to each risk tier.
6. To model cumulative exposure as a seven-day dose and to raise Safe, Moderate, High or Critical alerts.
7. To expose all functions through a FastAPI backend and a React dashboard that a non-technical user can operate.
8. To report honestly which parts of the system have been quantitatively validated and which have not.

1.2.3 Target Users and Applications

Table 1 – Target users and how PureBreath-AI supports them

User group	Need	Supporting module
Citizens and commuters	Know whether the day is safe for outdoor travel	AQI forecast, risk score
People with asthma or pregnancy	Receive precautions tuned to their condition	Health-risk engine, advisories
Outdoor workers	Understand cumulative occupational exposure	Cumulative-dose model, alerts
Parents and schools	Decide when children should stay indoors	Age multiplier, advisories

User group	Need	Supporting module
Municipal planners	See which pollutants drive AQI	SHAP global importance
Researchers and students	Reuse a documented, open pipeline	Source code, API

1.2.4 Expected Benefits

- **Personalization:** the same AQI produces different risk scores for a child, an elderly asthmatic and a healthy office worker.
- **Transparency:** every forecast is accompanied by the features that drove it and by what-if advice.
- **Cumulative view:** exposure is tracked over a week rather than judged from a single reading.
- **Low cost:** the models run on a standard CPU; no sensors or special hardware are required.
- **Honest evidence:** each module is labelled as validated, rule-based or planned so that claims are not overstated.

1.2.5 Methodology Summary

The development followed seven steps: requirement analysis, data collection, preprocessing and feature engineering, model training and evaluation, explainability and risk modules, API and dashboard integration, and testing. Figure 2 summarises the volume of engineered features that feed the forecasting models, and Figure 3 shows how the CPCB sub-index rules convert pollutant concentrations into the AQI that the models are trained to predict.

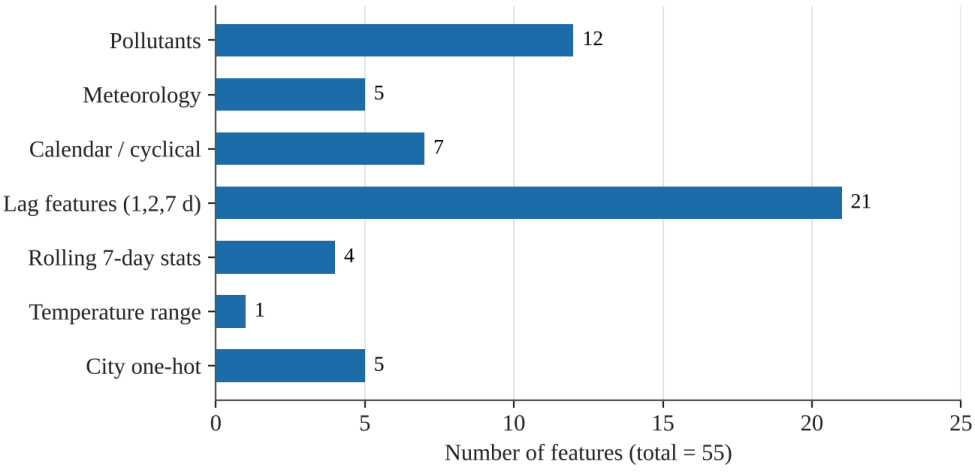


Figure 2 – Composition of the 55-feature input space

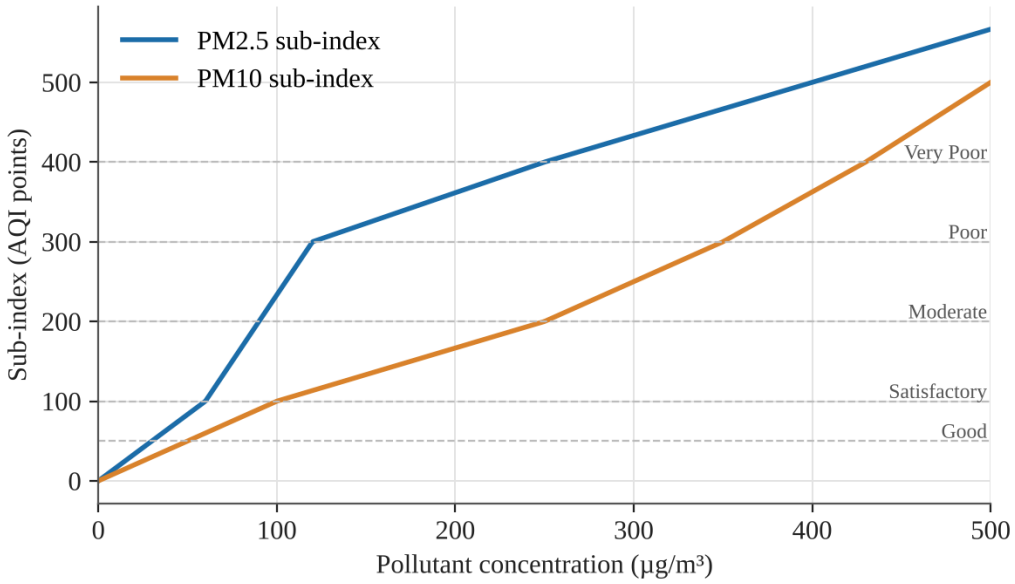


Figure 3 – CPCB sub-index curves for PM2.5 and PM10 used to build the AQI target

1.2.6 Limitations of the Present Work

The forecasting scores were obtained with a random 80/20 split, so temporally adjacent days can appear in both the training and the test set; the scores therefore measure same-distribution accuracy rather than generalisation to unseen seasons. The seven-day AQI path used by the cumulative-exposure module is generated from a city base value with a ±15% daily variation, so it illustrates the alerting logic rather than predicting future exposure. The personalized risk weights are drawn from WHO and CPCB guidance and have not been calibrated against clinical outcomes. These limitations are discussed in detail in Chapter 5.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

The literature relevant to PureBreath-AI falls into five connected streams: (i) AQI and multi-pollutant forecasting, (ii) explainable AI for air-quality models, (iii) personal exposure estimation, (iv) health-risk modelling, and (v) cumulative-exposure analysis. This chapter reviews fifteen recent studies across these streams, identifies what each contributes, and then positions PureBreath-AI against them using both a capability comparison and a comparison of reported forecasting accuracy.

Forecasting studies treat the problem as a regression task over pollutant, meteorological and calendar features. Tree-based ensembles are dominant because they handle tabular data with mixed scales and non-linear interactions with little tuning, while deep sequence models are used where long temporal dependencies matter. Explainability work has focused on SHAP and LIME to rank features and to justify individual predictions. Exposure and health studies move from ambient concentration to the person, using time-activity patterns, wearable monitors and cohort data. Taken together, the literature treats forecasting and explanation as connected stages, and exposure and health as a second, separate family of problems.

2.2 LITERATURE SURVEY

2.2.1 Forecasting and Explainability

Singh et al. [1] combined gradient boosting, XGBoost, LightGBM and stacking with SHAP-based interpretability for AQI prediction. Working on a Taiwan EPA dataset of 4.6 million hourly records (a 10,000-sample subset was used for the model comparison), they report a tuned gradient-boosting regressor with R^2 of 0.9973 and RMSE of 0.75, a weighted ensemble with R^2 of 0.9969, and a test-set R^2 of 0.9978 with a temporal-test R^2 of 0.9942. Their work shows that SHAP can be paired with boosted ensembles, but it stops at the level of the city or station and does not reach an individual.

Wang et al. [3] addressed missing AQI values with TimeGAN data recovery combined with PSO-optimized variational-mode decomposition and a deep-learning predictor. Their contribution is the use of generative modelling to repair incomplete series, which is conceptually close to the GAN-based class balancing used in PureBreath-AI, although the

purposes differ: Wang et al. recover missing values, whereas PureBreath-AI synthesises minority-class samples.

Zhu et al. [8] analysed the applicability of tree-based ensembles to multi-pollutant prediction using 5,844 records from four Chinese municipalities (Beijing, Shanghai, Tianjin and Chongqing) over 2021–2024. A Random Forest was best for most pollutants, with R^2 of 0.99 for PM2.5, 0.98 for PM10, 0.88 for NO2, 0.99 for SO2 and 0.98 for CO, ahead of GBDT and decision-tree baselines. This supports PureBreath-AI's choice of tree-based models and also shows how much accuracy varies by pollutant.

Narkhede et al. [9] extended prediction to a deep-ensemble setting with advanced preprocessing, again targeting several pollutants at once. Tipsavak et al. [7] used XGBoost with SHAP to predict and explain ultrafine particles (PM0.1) from meteorological variables, demonstrating that attribution can be applied to pollutants that are rarely monitored. Tasioulis et al. [6] compared several model families and several XAI methods (SHAP and LIME among them) on air-quality tasks, concluding that the quality of an explanation depends on both the model and the explainer. Agbehadji and Obagbuwa [5] surveyed XAI methods for air-pollution risk and respiratory outcomes, linking explainable models to health endpoints.

2.2.2 Personal Exposure

Wang et al. [10] reviewed personal-exposure monitoring methods and the growing role of big data and machine learning, noting the gap between fixed-monitor concentrations and what individuals actually breathe. Li et al. [11] predicted personal PM2.5 exposure in two Chinese megacities from time-activity patterns and household determinants using several machine-learning algorithms. Matthaios et al. [13] predicted in-vehicle exposure by combining mass-balance modelling with machine learning on on-road and air-quality data. Arvind et al. [14] used a network of stationary and wearable monitors, including subjects in Delhi, to predict spatio-temporal personal PM2.5 exposure. Mythili et al. [4] built MYAIRCARE, a platform that unites outdoor pollution tracking, personalized risk prediction, IoT sensing and real-time alerts.

These studies obtain exposure from measurements (wearables, in-vehicle sensors, time-activity diaries). PureBreath-AI takes the complementary route of estimating exposure from a transparent profile (age, occupation, outdoor hours, conditions), which needs no

hardware but also cannot capture micro-environments. The two approaches are compatible, and live sensor integration is listed as future work.

2.2.3 Health Risk and Cumulative Burden

Rajesh et al. [2] connected real-time air-quality assessment to environmental health-risk mapping with a machine-learning framework. Zhang et al. [12] used machine learning to construct and predict an Air Quality Health Index (AQHI) in Guangzhou. Wijata et al. [15] folded a cumulative PM2.5 exposure metric into a cardiovascular-mortality prediction model for high-risk groups in a retrospective cohort study. These works establish that cumulative burden, and not only the instantaneous AQI, matters for health, which motivates the seven-day dose model in PureBreath-AI.

2.2.4 Summary of the Surveyed Studies

Table 2 – Summary of the fifteen surveyed studies

Ref.	Authors (year)	Focus	Relation to PureBreath-AI
[1]	Singh et al. (2026)	Boosting and stacking with SHAP for AQI	Same model family; no personalization
[2]	Rajesh et al. (2025)	Real-time AQ assessment and health-risk mapping	Links AQ to health risk; area-level
[3]	Wang et al. (2025)	TimeGAN recovery with VMD deep learning	Generative modelling of AQ data
[4]	Mythili et al. (2025)	MYAIRCARE: IoT, risk prediction, alerts	Closest end-to-end system; sensor based
[5]	Agbehadji and Obagbuwa (2025)	XAI review for pollution risk	Supports XAI-to-health linkage
[6]	Tasioulis et al. (2025)	SHAP versus LIME across AQ models	Justifies explainer choice
[7]	Tipsavak et al. (2025)	XGBoost and SHAP for PM0.1	SHAP on tree models
[8]	Zhu et al. (2026)	Tree ensembles for multi-pollutant prediction	Supports tree-based choice
[9]	Narkhede et al. (2025)	Deep ensemble with preprocessing	Multi-pollutant forecasting
[10]	Wang et al. (2025)	Review of personal-exposure monitoring	Motivates personal exposure
[11]	Li et al. (2024)	Personal PM2.5 exposure, two megacities	Determinants-based exposure
[12]	Zhang et al. (2024)	ML-based AQHI, Guangzhou	Health index construction
[13]	Matthaios et al. (2024)	In-vehicle exposure prediction	Micro-environment exposure
[14]	Arvind et al. (2024)	Wearable and stationary monitors, incl. Delhi	Sensor-based exposure
[15]	Wijata et al. (2026)	Cumulative PM2.5 and cardiovascular mortality	Supports cumulative dose

2.3 COMPARATIVE ANALYSIS WITH THE SURVEYED WORK

2.3.1 Capability Comparison

Table 3 and Figure 4 compare the dominant capability of each research direction with PureBreath-AI across six capabilities: forecasting, explainability, personal exposure, health risk, cumulative exposure and alerting. Each cell uses the ratings of the PureBreath-AI paper (Yes, Partial, Limited, Related, No); the heat-map in Figure 4 converts them to a numeric scale (Yes = 2, Yes/Partial = 1.5, Partial, Limited or Related = 1, No = 0).

Table 3 – Capability comparison of research directions

Direction	Fcst	XAI	Exp	Risk	Cum	Alert
AQI forecasting [1]–[3], [8], [9]	Yes	Part.	No	Ltd.	No	Ltd.
XAI air quality [1], [5]–[8]	Y/P	Yes	No	Ltd.	No	Ltd.
Personal exposure [4], [10], [11], [13], [14]	Ltd.	Ltd.	Yes	Part.	Part.	Ltd.
Health risk [2], [5], [12], [15]	Part.	Part.	Ltd.	Yes	Part.	Ltd.
Cumulative exposure [11], [15]	Ltd.	Ltd.	Rel.	Rel.	Yes	Ltd.
PureBreath-AI (implemented)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

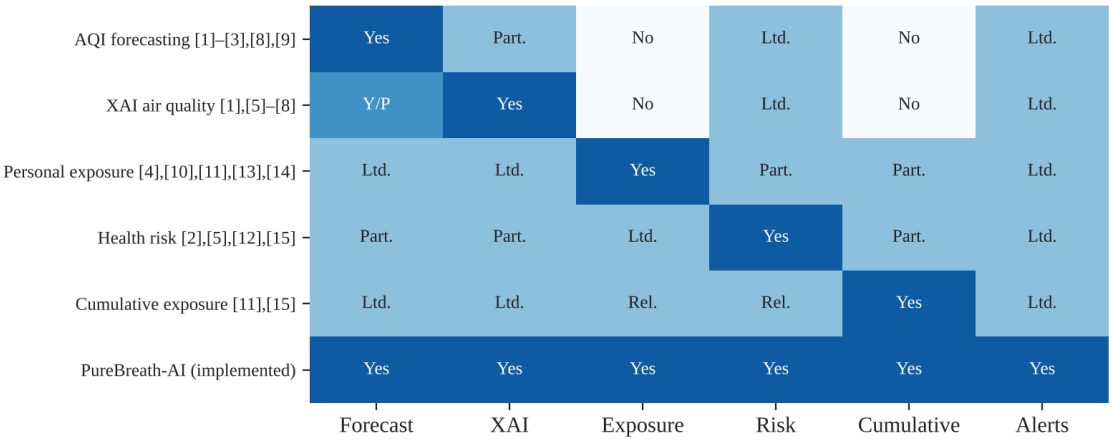


Figure 4 – Capability heat-map of research directions versus PureBreath-AI

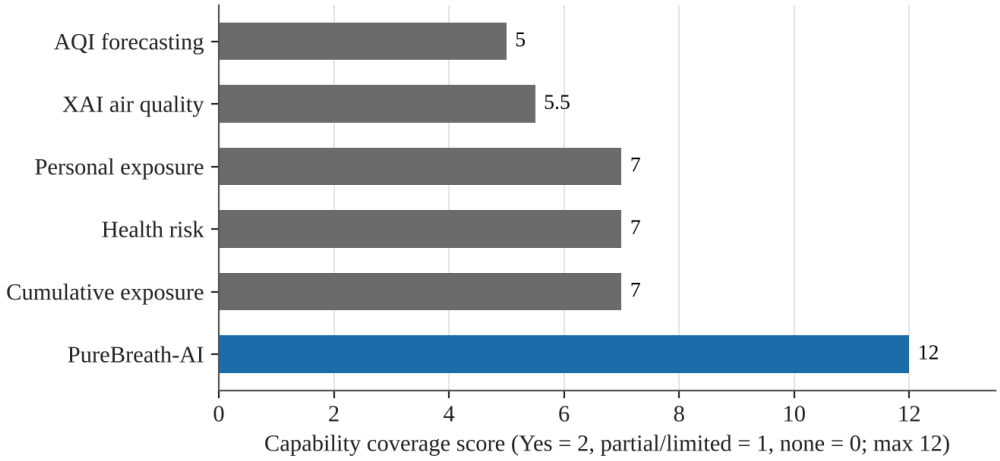


Figure 5 – Capability coverage score (maximum 12)

Each surveyed direction is strong in one or two columns and weak elsewhere, scoring between 5 and 7 out of 12, while PureBreath-AI covers all six capabilities (12 out of 12). The comparison does not claim that any single PureBreath-AI stage beats the corresponding stage of the specialised studies; the contribution is integration, which is the research gap identified in the next section.

2.3.2 Forecasting Accuracy Comparison

Published forecasting studies use different datasets, targets, sampling frequencies and splits, so their numbers are not directly interchangeable. Figure 6 places the R^2 of PureBreath-AI beside values reported by Singh et al. [1] and Zhu et al. [8] as an indicative comparison only. PureBreath-AI's LightGBM (0.9996) and XGBoost (0.9994) are in the same range as the best published ensembles (0.9973 and 0.9969 in [1]) and above the Random Forest results reported for PM2.5, PM10 and NO2 in [8] (0.99, 0.98 and 0.88). The caution in Section 5.4.7 applies: the target here is an index computed from two of the input pollutants, which makes the regression easier than predicting a raw concentration.

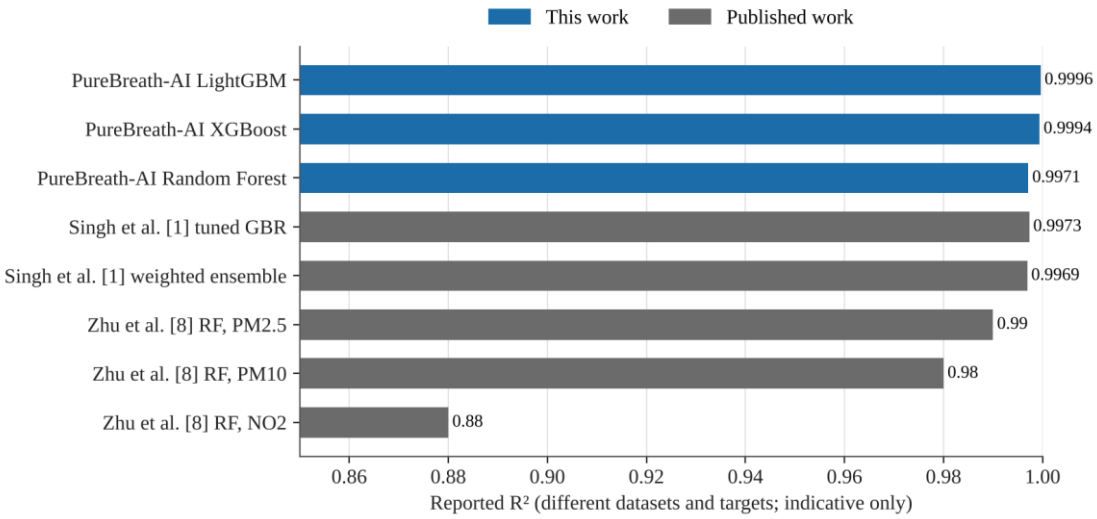


Figure 6 – Reported R^2 of PureBreath-AI and of published studies (indicative only)

2.4 RESEARCH GAP

Reviewing the literature reveals five gaps that motivated this project:

1. Forecasting studies describe conditions in the air, not what a particular person breathes [1], [3], [8], [9].
2. Explanation is often added after the fact and not designed as a stage of the pipeline [1], [6]–[8].
3. Exposure and health-risk modelling live in separate literatures, even though one system could link them while keeping decision support distinct from clinical diagnosis [2], [10]–[12].
4. Short-term forecasts and long-term cumulative burden are rarely modelled together [11], [15].
5. No reviewed system covers the whole chain from data to forecast, explanation, personalized and cumulative exposure, health risk and alerting [4].

PureBreath-AI addresses these gaps by building and deploying every stage. Closing the integration gap is a systems contribution, and it is not a claim that any single stage outperforms the models it uses.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

Existing air-quality services fall into three groups. The first group consists of public dashboards and mobile applications that display the latest AQI or pollutant concentration for a monitoring station or a city. The second group consists of research forecasting models that predict AQI or individual pollutants from historical and meteorological data. The third group consists of exposure and health studies that use wearables, in-vehicle sensors or cohort data to relate pollution to individual or population outcomes. Platforms such as MYAIRCARE [4] have begun to combine tracking, risk prediction, IoT and alerts, but most deployed tools still stop at the ambient number.

3.1.1 Limitations of Existing Systems

Table 4 – Limitations of existing approaches

Limitation	Consequence for the user
Report only ambient AQI or concentration	The number is the same for a child and a traffic constable
Forecast without explanation	Users cannot see why the AQI will be high and so trust it less
Explanation added after the model	Advice is generic and not tied to the drivers of the prediction
Exposure and health risk modelled separately	No single decision is produced from forecast to personal risk
Single-day view	Accumulated exposure over a week is not visible
Hardware-dependent exposure sensing	Wearables and IoT networks raise cost and limit reach
Models trained on other regions	Weights do not transfer reliably to Indian cities

In addition, many studies evaluate only forecasting accuracy and leave the integration between stages undescribed, so a reader cannot tell which parts of a claimed system were actually built and tested.

3.2 PROPOSED SYSTEM

PureBreath-AI is proposed as a single, deployed pipeline that starts with historical data and ends with a personal recommendation. Its design principles are integration (every stage feeds the next), transparency (every score can be traced to a rule or a feature), minimal data (only the profile fields needed are collected) and honest reporting (each module is labelled as validated, rule-based or planned).

3.2.1 Features of the Proposed System

- **Data acquisition and preprocessing:** merges CPCB daily records with gridded weather data for five cities, imputes missing values by interpolation, removes duplicates and clips physical outliers.
- **GAN class balancing:** a conditional GAN generates 500 synthetic samples for each of six AQI classes (3,000 in total).
- **Forecasting engine:** XGBoost, LightGBM and Random Forest regressors trained on 55 engineered features.
- **Explainability:** SHAP TreeExplainer on the LightGBM model, plus counterfactual recommendations produced by reducing the top three positive drivers by 20%.
- **Personalized risk:** a 0–100 score from AQI category, age, occupation, outdoor hours, asthma and pregnancy, mapped to five tiers with advice.
- **Cumulative exposure:** daily and seven-day dose with four alert tiers.
- **Application layer:** FastAPI endpoints and a React dashboard with an Apple-Health-style report view.

3.2.2 Advantages over Existing Systems

Table 5 – Advantages of PureBreath-AI over existing approaches

Aspect	Existing tools	PureBreath-AI
Output	Ambient AQI	AQI, drivers, personal risk, weekly dose, alerts
Explanation	None or post hoc	SHAP and counterfactual advice for each prediction
Personalization	None	Age, occupation, outdoor hours, asthma, pregnancy
Time horizon	Single reading	Daily and seven-day cumulative dose
Hardware	Sensors or wearables	CPU-only; no sensors needed
Evidence statement	Often implicit	Validated, rule-based and planned modules separated

3.3 SYSTEM ARCHITECTURE

Figure 7 shows the architecture. Historical AQI and weather data, and a user profile, enter a preprocessing and feature-engineering stage. The prepared data feed the AQI forecasting model, whose output fans out into three parallel modules: explainability,

personalized risk assessment and cumulative-exposure tracking. All three are surfaced through the React web application.

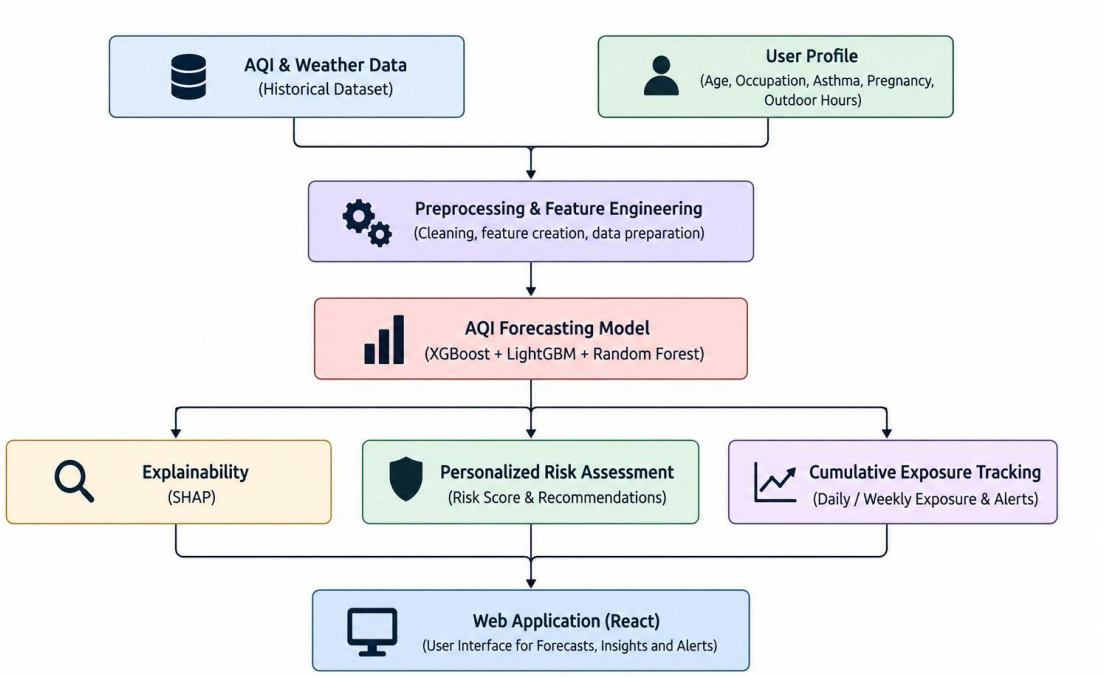


Figure 7 – System architecture of PureBreath-AI

3.3.1 Module Description

Input layer. Two inputs are used. The first is the historical dataset: a CPCB daily air-quality table (PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene) and a gridded meteorological table (temperature, precipitation, pressure height). The second is the user profile: age, gender, occupation, asthma, pregnancy and daily outdoor hours.

Preprocessing and feature engineering. The weather table is streamed in chunks of one million rows, filtered to the grid cells of the five cities, aggregated to daily resolution and merged with the pollutant table on City and Date. Missing values are interpolated within each city, duplicates are removed and pollutants are clipped to non-negative values. Calendar, lag, rolling and temperature-range features are then built.

GAN augmentation. A conditional generator and discriminator, both conditioned on the AQI class through an embedding, are trained on the standardised pollutant and weather features. The trained generator produces 500 samples per class, which are appended to the real data.

Forecasting engine. Three regressors predict the CPCB AQI. LightGBM, the best performer, is the default model of the API.

Explainability. A SHAP TreeExplainer computes feature contributions; a counterfactual reasoner perturbs the top three positive contributors and re-predicts the AQI.

Personalized risk and cumulative exposure. The risk engine converts the AQI category into a score using the user's profile. The exposure engine computes dose per day and per week and maps it to an alert tier.

Application layer. A FastAPI service exposes the modules; the React front end renders the dashboard, prediction form, risk form, exposure form and health report.

3.3.2 Data Flow

1. The user submits pollutant readings, or chooses a city for automatic prediction, together with a profile.
2. The API builds the 55-element feature vector and calls the LightGBM model for the predicted AQI.
3. SHAP values and counterfactual advice are computed for the same vector.
4. The AQI category and the profile feed the risk engine, which returns a score, tier and advisory.
5. The AQI path and the profile feed the exposure engine, which returns daily and weekly dose and the alert tier.
6. The front end combines the responses into a single report.

3.3.3 API Interface

Table 6 – REST endpoints of the FastAPI backend

Endpoint	Method	Purpose
/health	GET	Liveness probe
/predict	POST	AQI prediction with SHAP explanation and counterfactual recommendations
/health-risk	POST	Personalized risk score, tier and advisory
/health-report	POST	Full narrative health report
/exposure	POST	Seven-day cumulative exposure and alerts
/auto-predict/{city}	GET	Automated forecast for a city
/city-summary	GET	Pre-aggregated risk and exposure summaries for all five cities

3.4 HARDWARE REQUIREMENTS

Model inference and the API run on any standard CPU-only machine. Training the conditional GAN benefits from an NVIDIA GPU with CUDA but falls back to CPU if none is available (the recorded augmentation run used a CUDA device). No environmental sensors or IoT devices are required, because all data come from the CPCB and weather datasets.

Table 7 – Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core 2 GHz, 64-bit	Quad-core or better
Memory	8 GB	16 GB (large weather file streaming, GAN training)
Storage	5 GB free	20 GB free (weather dataset, models, reports)
GPU	Not required for serving	NVIDIA GPU with CUDA for GAN training
Display and network	Standard monitor, local network	Broadband access for package installation

3.5 SOFTWARE REQUIREMENTS

The software stack follows the project's requirements.txt and package.json files. The back end needs Python 3.11 or later (3.14 was confirmed working), and the front end needs Node.js 18 or later.

Table 8 – Back-end software (requirements.txt)

Package	Version	Use
FastAPI / Uvicorn	0.115.0 / 0.30.6	REST framework and ASGI server
Pydantic	2.9.2	Request and response validation
NumPy / pandas	2.4.6 / 3.0.5	Numerical work and data frames
scikit-learn	1.9.0	Splitting, metrics, scaling, Random Forest
XGBoost	3.3.0	Gradient-boosted regressor
LightGBM	4.7.0	Primary AQI regressor
PyTorch	2.11.0	Conditional GAN
SHAP	0.52.0	TreeExplainer attributions
joblib	1.4.2	Model serialisation

Table 9 – Front-end software (package.json)

Package	Version	Use
React / React DOM	19.2.8	Component-based user interface
Vite	8.2.0	Build tool and development server

Package	Version	Use
Recharts	3.10.1	Charts in the dashboard
Axios	1.19.0	HTTP client for the API
Framer Motion	13.0.0	Page and component transitions
Lucide React	1.28.0	Icons

The software was developed on Windows and runs equally on Linux or macOS. Both servers can be started with one command (npm run dev:full), which launches Uvicorn for the API and Vite for the front end.

3.6 NON-FUNCTIONAL REQUIREMENTS AND DESIGN CONSIDERATIONS

Beyond the functional modules, the design takes the following quality attributes into account.

Table 10 – Non-functional requirements

Attribute	Design decision
Reproducibility	Fixed random seeds (42) for splitting, training and GAN; artefacts saved to models/ and reports/
Transparency	Rule-based risk and dose engines with published weights; SHAP for the learned model
Privacy	Only the profile fields needed for scoring are collected (age, gender, occupation, asthma, pregnancy, outdoor hours); no location tracking
Portability	CPU-only inference; Python and Node.js on Windows, Linux or macOS
Modularity	Each stage is a separate Python module that reads and writes files, so a stage can be replaced independently
Extensibility	The report generator is isolated so that a language model can replace it; new cities need only data and a grid-cell mapping
Maintainability	Typed request models (Pydantic), docstrings on every function, pinned dependency versions

The system supports decision making and is not a medical device. Risk tiers and advisories are guidance derived from published WHO and CPCB categories, and users with symptoms are always directed to seek medical advice.

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

PureBreath-AI was developed incrementally, one module per working week, so that each stage could be tested before the next one depended on it. The sequence was: data preprocessing, model training, GAN augmentation, health-risk assessment, cumulative exposure, explainable AI, and finally the API and dashboard. Every module reads the artefacts written by the previous one (for example the cleaned CSV, the saved model files and the feature list), which keeps the pipeline reproducible.

4.1.1 Development Workflow

Table 11 – Development stages and their artefacts

Stage	Source file	Main output
Data preprocessing	data_preprocessing.py	merged_dataset.csv, cleaned_dataset.csv
Model training	model_training.py	three .pkl models, model_features.json, evaluation metrics
GAN augmentation	gan_augmentation.py	augmented dataset, gan_augmentation_meta.json
Health risk	health_risk_assessment.py	per-city risk CSVs, health_risk_summary.json
Cumulative exposure	cumulative_exposure.py	per-city exposure CSVs, cumulative_exposure_summary.json
Explainable AI	xai_explainer.py	SHAP importance, SHAP values, counterfactual CSV, summary
API and dashboard	api.py, frontend/	REST service and React application

4.1.2 Dataset Description

The air-quality input is a CPCB daily city table covering Delhi, Mumbai, Chennai, Kolkata and Bangalore with twelve pollutant fields. The weather input is a gridded reanalysis table of roughly 4 GB with six-hourly observations of 2 m temperature, accumulated precipitation and geopotential height. Each city is mapped to a grid cell, for example Delhi (22, 11), Mumbai (13, 7), Chennai (7, 14), Kolkata (16, 22) and Bangalore (7, 12). After merging and cleaning, the dataset contains 10,960 real records; the GAN adds 3,000 synthetic ones to reach 13,960.

4.1.3 Preprocessing Steps

1. Parse the Datetime field into a normalised Date key.

2. Stream the weather file in chunks of one million rows and keep only the grid cells of the five cities.
3. Convert temperature from Kelvin to Celsius and aggregate the six-hourly values to daily mean, minimum and maximum temperature, total precipitation and mean pressure height.
4. Impute missing numeric values within each city by linear interpolation followed by back-fill and forward-fill.
5. Remove duplicate City-Date records and cast columns to explicit data types.
6. Inner-join the pollutant and weather tables on City and Date.
7. Clip pollutant concentrations and precipitation to non-negative values and round floats to two decimals.

4.1.4 Target Variable and Feature Engineering

The prediction target is the CPCB AQI computed from the PM2.5 and PM10 sub-indices, taking the larger of the two. The sub-index is a piecewise-linear function of concentration (Figure 3). For PM2.5, for example, 0–30 µg/m³ maps to 0–50, 30–60 to 50–100, 60–90 to 100–200, 90–120 to 200–300, 120–250 to 300–400 and above 250 to 400 and beyond.

$$AQI = max(I_PM2.5 , I_PM10) \qquad (1)$$

The feature space has 55 columns (Figure 2): the twelve pollutants; five meteorological variables; seven calendar and cyclical terms (month, day of week, day of year, quarter, weekend flag, and sine and cosine of the month); lag-1, lag-2 and lag-7 values of PM2.5, PM10, NO2, SO2, CO, O3 and mean temperature (21 features); the previous-day-shifted 7-day rolling mean and standard deviation of PM2.5 and PM10 (4 features); the daily temperature range; and a one-hot encoding of the city. Rows with undefined lags are dropped.

4.1.5 Evaluation Metrics

Forecasting models are compared on a held-out 20% test set using four metrics:

$$MAE = (1/n)\,\Sigma\,|y_i - \hat{y}_i| \qquad RMSE = \sqrt{[(1/n)\,\Sigma\,(y_i - \hat{y}_i)^2]} \qquad (2)$$

$$R^2 = 1 - \Sigma\,(y_i - \hat{y}_i)^2 / \Sigma\,(y_i - \bar{y})^2 \qquad MAPE = (100/n)\,\Sigma\,|y_i - \hat{y}_i| / max(y_i, 1) \qquad (3)$$

MAE and RMSE are in AQI points; RMSE penalises large errors more heavily. R^2 is the share of variance explained, and MAPE is the percentage error relative to the true AQI (with the denominator floored at 1).

4.1.6 Testing Strategy

- **Unit testing:** each module was run on its own and its output files inspected (row counts, null counts, types).
- **Model testing:** MAE, RMSE, R^2 and MAPE on the held-out split for all three regressors.
- **Explainability testing:** a 200-row sample (100 high-AQI and 100 other rows) was explained and perturbed, recording the number of cases with lower AQI, with a changed category and the mean reduction.
- **Rule verification:** risk and dose rules were applied to a 100-user profile file for every city and the tier distributions were checked for consistency.
- **Integration testing:** the API endpoints were called from the React front end to confirm end-to-end operation.

4.2 IMPLEMENTATION

4.2.1 Data Preprocessing Module

The module data_preprocessing.py contains one function per step: load_aqi_data, load_and_aggregate_weather_data, handle_missing_values, remove_duplicates, convert_datatypes, merge_datasets, clean_merged_dataset and save_dataset, orchestrated by run_preprocessing_pipeline. Chunked reading keeps memory use bounded while scanning the multi-gigabyte weather file, and the CITY_GRID_MAPPING dictionary holds the grid cell of each city. Interpolation is performed city by city so that values never leak across cities.

4.2.2 GAN Augmentation Module

The cleaned data have far more Moderate and Satisfactory days than Good or Severe days. To rebalance the classes, gan_augmentation.py trains a conditional GAN. The generator concatenates a 64-dimensional noise vector with a class embedding and passes it through two hidden layers of 256 units with layer normalisation and LeakyReLU activations. The discriminator receives a sample and its class embedding and passes them through dense layers with dropout of 0.3 and a sigmoid output. Both networks use Adam

(learning rate 2×10^{-4}, $\beta_1 = 0.5$), binary cross-entropy loss and a batch size of 256. Features are standardised before training and inverse-transformed after generation.

Table 12 – Conditional GAN configuration

Item	Value
Latent dimension	64
Hidden width	256
Batch size	256
Learning rate	2×10^{-4} (Adam, $\beta = 0.5, 0.999$)
Epochs in recorded run	200 (gan_augmentation_meta.json); the current source constant is 500
Input features	17 (12 pollutants and 5 meteorological variables)
Classes	Good, Satisfactory, Moderate, Poor, Very Poor, Severe
Synthetic samples	500 per class, 3,000 in total
Final training set	10,960 real + 3,000 synthetic = 13,960

Because a downstream ablation (training with and without synthetic data) has not yet been run, the report treats the GAN as an implemented data-balancing step whose benefit is yet to be quantified.

4.2.3 Forecasting Module

model_training.py builds the features, performs a random 80/20 split with a fixed seed, trains the three regressors and saves the models, the feature list and the metrics. LightGBM and XGBoost share similar settings so that the comparison is fair, and the Random Forest acts as a bagging baseline.

Table 13 – Regressor hyper-parameters

Parameter	XGBoost	LightGBM	Random Forest
n_estimators	300	300	100
max_depth	6	6	6
learning_rate	0.05	0.05	–
subsample	0.85	0.85	–
colsample_bytree	0.85	0.85	–
num_leaves	–	31	–
random_state	42	42	42

The scores obtained on the held-out test set are given in Table 13 and visualised in Chapter 5. LightGBM is the best model on every metric and is therefore the default model used by the API and the explainer.

Table 14 – Held-out test performance of the forecasting models

Model	MAE	RMSE	R^2	MAPE (%)
LightGBM	**1.7055**	**2.3667**	**0.9996**	**0.58**
XGBoost	2.1983	3.1600	0.9994	0.80
Random Forest	3.6069	6.7474	0.9971	1.98

4.2.4 Explainable AI Module

xai_explainer.py loads the LightGBM model and applies shap.TreeExplainer to a 200-row sample: 100 rows with AQI ≥ 200 and 100 other rows. The mean absolute SHAP value of each feature gives the global importance, and the per-row values give local explanations satisfying the additive identity

$$f(x) = \varphi_0 + \Sigma\, \varphi_i \qquad (4)$$

For each row, generate_counterfactual selects the three features with the largest positive SHAP value (those that push AQI up), multiplies each by 0.8, and re-predicts the AQI. A template library turns each modified feature into a sentence, for example “Reduce PM2.5 by 20% (e.g. avoid biomass burning, use cleaner fuels)” or “Reduce PM10 by 20% (e.g. water roads, control construction dust)”. The module records the original and counterfactual AQI, the category before and after, the reduction and the recommendations. Because the SHAP values and the counterfactuals describe the model and not the atmosphere, they are presented as decision support and not as causal claims.

4.2.5 Personalized Health-Risk Module

health_risk_assessment.py computes a transparent score from the AQI category and the user profile:

$$Score = Base(bucket) \times Age \times Occupation \times Outdoor \times Asthma \times Pregnancy \qquad (5)$$

The base score rises with the CPCB category. The age multiplier is 1.8 for children up to 5 years, 1.5 up to 12, 1.2 up to 17, 1.0 for adults to 60, 1.3 to 70 and 1.6 above 70. The occupation multiplier ranges from 1.5 for traffic police to 1.0 for sedentary jobs. The outdoor modifier adds 2% per hour above a two-hour baseline, and asthma and pregnancy multiply the score by 1.35 and 1.25. The final value is capped at 100 and mapped to five tiers.

Table 15 – Base risk by CPCB AQI category and risk tier thresholds

AQI category	Base score	Risk tier	Score range
Good	10	Low	< 20
Satisfactory	25	Moderate	20 – 39
Moderate	45	High	40 – 59
Poor	62	Very High	60 – 79
Very Poor	78	Hazardous	≥ 80
Severe	95		

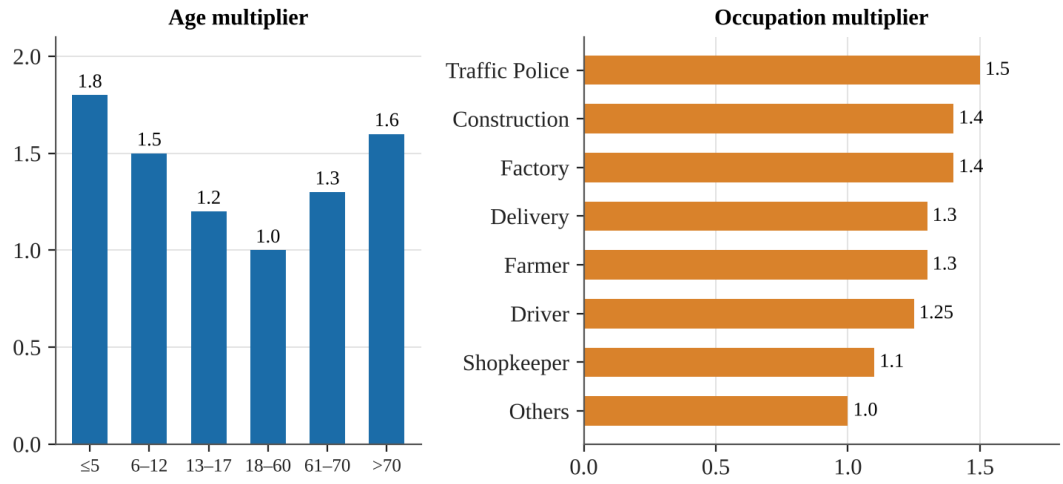


Figure 8 – Age and occupation multipliers of the risk engine

Each tier carries an advisory (for example, use an N95 mask for the High tier, and stay indoors for the Hazardous tier), and extra sentences are appended for asthma, pregnancy, young children and the elderly. A report generator also produces a narrative summary, a 0–100 health score and the safe outdoor hours for the category, and its structure is designed so that it can be replaced by a language-model call without changing the rest of the pipeline.

4.2.6 Cumulative Exposure Module

cumulative_exposure.py estimates the dose accumulated by a user:

$$Dose_day = AQI_day \times OutdoorHours \times Vulnerability \qquad Dose_week = \Sigma\,(d = 1 \ldots 7)\ Dose_d \qquad (6)$$

The vulnerability weight combines the age multiplier with 1.35 for asthma and 1.25 for pregnancy. Daily and weekly doses are compared with thresholds to give Safe, Moderate, High or Critical alerts (Figure 9). The seven-day AQI path is generated from a base AQI per city (Delhi 320, Kolkata 210, Mumbai 145, Chennai 95, Bangalore 78) with a uniform

±15% daily variation. It therefore demonstrates the alert logic and is not a multi-day forecast.

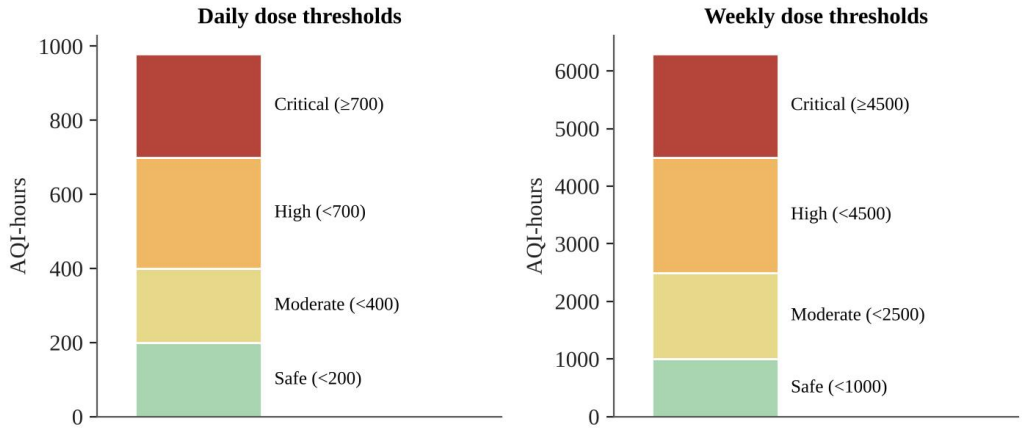


Figure 9 – Daily and weekly dose thresholds for the four alert tiers

4.2.7 API and User Interface Module

api.py defines the FastAPI application with permissive CORS for local development, Pydantic request models (PollutantInput, UserProfile, HealthRiskRequest, ExposureRequest, HealthReportRequest) and the endpoints listed in Table 6. For a prediction request the function build_feature_vector assembles the 55-element row in the order stored in model_features.json; lag and rolling features default to the current readings and the calendar defaults to a mid-year weekday when no history is supplied, which is a simplification of the live service.

The React front end (Vite, Recharts, Axios, Framer Motion) contains a Dashboard that calls /city-summary, a PredictForm that calls /auto-predict/{city}, a HealthRiskForm and HealthReport that call /health-report, an ExposureForm that calls /exposure, and shared layout components (Sidebar, Header and a theme module). Reusable interface primitives such as cards, badges, stat cards, gauges and progress rings are defined in a single ui.jsx file.

4.2.8 Execution Flow

1. The preprocessing pipeline produces the cleaned dataset and the GAN module produces the augmented dataset.
2. The forecasting module trains the models and writes the .pkl files and metrics.
3. The risk and exposure modules write per-city CSV reports and JSON summaries.
4. The explainability module writes the SHAP and counterfactual reports.

5. At run time the API loads the saved artefacts and serves requests from the dashboard.

4.2.9 Design Rationale

Several design choices deserve explanation. Tree-based ensembles were chosen because the AQI is a piecewise-linear function of the pollutant concentrations with interactions across pollutants and weather, a structure that boosted trees capture well, and because TreeExplainer provides exact SHAP values for them in polynomial time. LightGBM and XGBoost were both trained because their gradient-boosting implementations differ in tree growth (leaf-wise against level-wise), and the Random Forest provides a bagging baseline that is independent of boosting. A conditional GAN was preferred to simple oversampling because it can generate new, class-consistent combinations of pollutant and weather values instead of duplicating existing rows.

The risk and exposure engines were deliberately kept rule-based. A learned model of individual risk would need outcome labels that are not available in the datasets used here, and an opaque score would undermine the transparency goal. Instead, each multiplier is a visible constant that a clinician or public-health officer can inspect and adjust, which makes the engine easy to audit and to recalibrate when better evidence becomes available.

Table 16 – Artefacts exchanged between modules

Artefact	Written by	Read by
cleaned_dataset.csv	Preprocessing	Forecasting, GAN, XAI
lightgbm / xgboost / randomforest _aqi_model.pkl	Forecasting	XAI, API
model_features.json	Forecasting	XAI, API (feature order)
model_evaluation_metrics.json	Forecasting	Reporting
gan_augmentation_meta.json	GAN	Reporting
health_risk_summary.json	Health risk	API (/city-summary)
cumulative_exposure_summary.json	Exposure	API (/city-summary)
reports/xai/*.csv, xai_summary.json	XAI	Reporting

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

This project designed, implemented and evaluated PureBreath-AI, an explainable and personalized environmental-health framework that connects air-quality forecasting with individual health risk and cumulative exposure. Raw CPCB and weather data for Delhi, Mumbai, Chennai, Kolkata and Bangalore are cleaned, balanced with a conditional GAN, modelled with three tree-based regressors, explained with SHAP and counterfactuals, converted into personal risk scores, accumulated into a seven-day dose and delivered through a FastAPI back end and a React dashboard.

The main findings are as follows. First, LightGBM was the best forecasting model, with an MAE of 1.71, an RMSE of 2.37, an R^2 of 0.9996 and a MAPE of 0.58%, which is 52.7% lower MAE and 64.9% lower RMSE than the Random Forest baseline and 22.4% and 25.1% lower than XGBoost. Second, the explainability layer produced actionable counterfactual advice: a 20% reduction in the top three positive drivers lowered the predicted AQI in 162 of 200 cases, changed the AQI category in 24 cases and reduced the AQI by 29.9 points on average (58.6 points on average for the 101 high-AQI cases). Third, the personalized engine separated cities and users clearly: mean risk scores ranged from 35.2 for Bangalore and Chennai to 93.7 for Delhi, and within a single city the score depended strongly on age, condition and occupation. Fourth, the dose model showed that the mean weekly dose in Delhi (11,707 AQI-hours) was 3.7 times that in Bangalore (3,154), with 84% of Delhi users in the Critical tier.

A comparison with fifteen cited studies shows that each addresses one or two parts of the problem, whereas PureBreath-AI covers all six capabilities (forecasting, explainability, personal exposure, health risk, cumulative exposure and alerting). Its forecasting accuracy is in the same range as the best published ensembles on different datasets; the report is careful to note that such cross-study comparisons are indicative only.

The work also identifies honestly what remains unproven. The forecasting scores come from a random split and from a target that is computed from two of the input pollutants; the exposure module uses a simulated seven-day path; and the risk weights are rule-based and not clinically calibrated. These are the priorities for the next iteration, and they do not detract from the central contribution, which is a working, end-to-end, transparent pipeline.

5.2 FUTURE WORK

1. **Time-aware validation.** Re-evaluate the forecasting models with a chronological or walk-forward split so that the claims of generalisation to unseen dates and seasons can be tested.
2. **Leakage-free target.** Evaluate the models on a target that is not a deterministic function of the input features, for example next-day AQI or next-day PM2.5 predicted from lagged values only, and compare against persistence and seasonal baselines.
3. **GAN ablation.** Train with and without the synthetic samples, and report per-class errors, to quantify the benefit of the augmentation.
4. **True multi-day forecasting.** Replace the simulated seven-day AQI path with a recursive or direct multi-horizon forecast so that the dose and alerts become genuine predictions.
5. **Live data and wearables.** Add live CPCB feeds, low-cost sensors and wearable devices for measured personal exposure.
6. **Clinical calibration.** Validate the risk weights against an independent epidemiological or clinical standard using precision, recall, F1 and calibration measures.
7. **Language-model reports.** Swap the template-based report generator for a language model while keeping the deterministic risk score as a guard rail.
8. **Performance and usability testing.** Measure API latency and alert-generation time, and run user studies on trust, clarity and usability.
9. **Privacy-preserving personalization.** Keep profile data on the device, or use federated approaches, so that sensitive health information is not centralised.
10. **Single AQI source.** Use one AQI value across the dashboard, exposure, risk and report pages; at present the dashboard and exposure pages use each city's base AQI, whereas the prediction and report pages use the model's prediction from the latest dataset row (Section 5.5).
11. **Scale-out.** Extend from five cities to more Indian cities and then to other regions, retraining on local data.

5.3 SOURCE CODE

This section reproduces the complete source of the seven back-end modules of the PureBreath-AI repository, exactly as written. The project is organised as src/ (the seven Python modules below), frontend/ (the React application), models/ (saved models and JSON metrics) and reports/ (SHAP, counterfactual, risk and exposure outputs).

5.3.1 Data preprocessing (src/data_preprocessing.py)

```
"""
PureBreath-AI: Week 2 - Data Preprocessing Module

This module handles reading AQI and Weather datasets, analyzing structure,
handling missing values, deduplicating, type conversions, spatial grid mapping,
merging datasets on Date and City, cleaning, and exporting processed artifacts.

Author: PureBreath-AI Team
License: MIT
"""


import os
import logging
import pandas as pd
import numpy as np
from typing import Dict, Tuple

# Configure logging for production tracing
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler()]
)

# Geographic grid coordinate mapping for target Indian cities in ERA5/NCEP weather grid
CITY_GRID_MAPPING: Dict[str, Tuple[int, int]] = {
    "Delhi": (22, 11),
    "Mumbai": (13, 7),
    "Chennai": (7, 14),
    "Kolkata": (16, 22),
    "Bangalore": (7, 12)
}



def load_aqi_data(filepath: str) -> pd.DataFrame:
    """
    Reads the AQI city daily dataset, validates presence of required columns,
    and standardizes date formatting.

    Args:
        filepath (str): Relative path to city_day.csv

    Returns:
        pd.DataFrame: Cleaned AQI DataFrame with normalized Date column.
    """
    if not os.path.exists(filepath):
        raise FileNotFoundError(f"AQI dataset not found at path: {filepath}")

    logging.info(f"Loading AQI dataset from '{filepath}'...")
```

```
    df_aqi = pd.read_csv(filepath)

    # Standardize column names
    df_aqi.columns = [col.strip() for col in df_aqi.columns]

    # Convert Datetime to standard YYYY-MM-DD Date string
    df_aqi["Date"] = pd.to_datetime(df_aqi["Datetime"]).dt.strftime("%Y-%m-%d")

    logging.info(f"Loaded {len(df_aqi)} records for cities: {df_aqi['City'].unique().tolist()}")
    return df_aqi


def load_and_aggregate_weather_data(filepath: str, city_grid_map: Dict[str, Tuple[int, int]]) ->
pd.DataFrame:
    """
    Streams the raw 4GB weather dataset, filters spatial grid cells corresponding to target cities,
    converts temperatures from Kelvin to Celsius, and aggregates metrics to daily resolution.

    Args:
        filepath (str): Relative path to weather.csv
        city_grid_map (dict): Mapping of City name to (latitude_idx, longitude_idx)

    Returns:
        pd.DataFrame: Daily aggregated weather data per City and Date.
    """
    if not os.path.exists(filepath):
        raise FileNotFoundError(f"Weather dataset not found at path: {filepath}")

    logging.info(f"Processing weather dataset from '{filepath}'...")

    filtered_chunks = []
    chunksize = 1_000_000

    # Stream in chunks for memory safety
    for chunk_idx, chunk in enumerate(pd.read_csv(filepath, chunksize=chunksize)):
        mask = pd.Series(False, index=chunk.index)
        for (lat_idx, lon_idx) in city_grid_map.values():
            mask |= (chunk["latitude"] == lat_idx) & (chunk["longitude"] == lon_idx)

        sub = chunk[mask].copy()
        if not sub.empty:
            filtered_chunks.append(sub)

    df_weather = pd.concat(filtered_chunks, ignore_index=True)

    # Map latitude and longitude grid indices back to City names
    grid_to_city = {coords: name for name, coords in city_grid_map.items()}
    df_weather["City"] = df_weather.apply(
        lambda row: grid_to_city.get((int(row["latitude"]), int(row["longitude"])), "Unknown"),
        axis=1
    )

    # Parse timestamps and derive Date
    df_weather["time"] = pd.to_datetime(df_weather["time"])
    df_weather["Date"] = df_weather["time"].dt.strftime("%Y-%m-%d")

    # Temperature conversion Kelvin -> Celsius
```

```
    df_weather["TMP_2m_C"] = df_weather["TMP_2m"] - 273.15

    # Aggregate 6-hourly observations into daily summary features
    logging.info("Aggregating weather readings to daily resolution per city...")
    df_daily_weather = df_weather.groupby(["City", "Date"]).agg(
        temp_mean_c=("TMP_2m_C", "mean"),
        temp_min_c=("TMP_2m_C", "min"),
        temp_max_c=("TMP_2m_C", "max"),
        precipitation_mm=("APCP_sfc", "sum"),
        pressure_hgt=("HGT_prl", "mean")
    ).reset_index()

    logging.info(f"Aggregated daily weather contains {len(df_daily_weather)} records.")
    return df_daily_weather


def handle_missing_values(df: pd.DataFrame) -> pd.DataFrame:
    """
    Imputes missing values using time-series forward/backward fill per city.

    Args:
        df (pd.DataFrame): Raw or merged DataFrame.

    Returns:
        pd.DataFrame: Imputed DataFrame with zero null values.
    """
    logging.info("Handling missing values using time-series linear interpolation & forward/backfill...")
    df = df.copy()

    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()

    # Sort by City and Date before time-series fill
    df = df.sort_values(by=["City", "Date"]).reset_index(drop=True)

    for city in df["City"].unique():
        city_mask = df["City"] == city
        # Interpolate numeric features along time dimension
        df.loc[city_mask, numeric_cols] = df.loc[city_mask, numeric_cols].interpolate(method="linear")
        df.loc[city_mask, numeric_cols] = df.loc[city_mask, numeric_cols].bfill().ffill()

    # Fallback for remaining missing categorical values
    if "AQI_Bucket" in df.columns:
        df["AQI_Bucket"] = df["AQI_Bucket"].fillna("Moderate")

    missing_remaining = df.isnull().sum().sum()
    logging.info(f"Missing values handling complete. Remaining nulls: {missing_remaining}")
    return df


def remove_duplicates(df: pd.DataFrame) -> pd.DataFrame:
    """
    Detects and removes duplicate records based on unique City and Date key.

    Args:
        df (pd.DataFrame): Input DataFrame.

    Returns:
```

```
        pd.DataFrame: Deduplicated DataFrame.
    """
    initial_count = len(df)
    df_dedup = df.drop_duplicates(subset=["City", "Date"]).reset_index(drop=True)
    duplicates_removed = initial_count - len(df_dedup)
    logging.info(f"Deduplication complete. Removed {duplicates_removed} duplicate records.")
    return df_dedup


def convert_datatypes(df: pd.DataFrame) -> pd.DataFrame:
    """
    Ensures optimal, explicit data types for downstream machine learning pipelines.

    Args:
        df (pd.DataFrame): Input DataFrame.

    Returns:
        pd.DataFrame: DataFrame with standardized numeric, date, and categorical types.
    """
    logging.info("Converting data types to strict production standards...")
    df = df.copy()

    # Convert Date column to datetime format
    df["Date"] = pd.to_datetime(df["Date"])

    # Explicit numeric cast for floating point metrics
    float_columns = [
        "PM2.5", "PM10", "NO", "NO2", "NOx", "NH3", "CO", "SO2", "O3",
        "Benzene", "Toluene", "Xylene", "AQI", "temp_mean_c", "temp_min_c",
        "temp_max_c", "precipitation_mm", "pressure_hgt"
    ]
    for col in float_columns:
        if col in df.columns:
            df[col] = df[col].astype(np.float64)

    # Categorical conversions
    df["City"] = df["City"].astype("category")
    if "AQI_Bucket" in df.columns:
        df["AQI_Bucket"] = df["AQI_Bucket"].astype("category")

    return df


def merge_datasets(aqi_df: pd.DataFrame, weather_df: pd.DataFrame) -> pd.DataFrame:
    """
    Merges AQI and daily weather datasets on matching Date and City.

    Args:
        aqi_df (pd.DataFrame): Processed AQI DataFrame.
        weather_df (pd.DataFrame): Daily aggregated weather DataFrame.

    Returns:
        pd.DataFrame: Inner-joined merged dataset.
    """
    logging.info("Merging AQI and Weather datasets on ['City', 'Date']...")

    # Ensure Date format matches string representation for seamless join
    aqi_df["Date_str"] = pd.to_datetime(aqi_df["Date"]).dt.strftime("%Y-%m-%d")
```

```
    weather_df["Date_str"] = pd.to_datetime(weather_df["Date"]).dt.strftime("%Y-%m-%d")

    merged_df = pd.merge(
        aqi_df,
        weather_df,
        left_on=["City", "Date_str"],
        right_on=["City", "Date_str"],
        how="inner",
        suffixes=("", "_weather")
    )

    # Clean up auxiliary columns
    if "Date_str" in merged_df.columns:
        merged_df = merged_df.drop(columns=["Date_str"])
    if "Datetime" in merged_df.columns:
        merged_df = merged_df.drop(columns=["Datetime"])

    logging.info(f"Successfully merged datasets. Merged shape: {merged_df.shape}")
    return merged_df


def clean_merged_dataset(merged_df: pd.DataFrame) -> pd.DataFrame:
    """
    Applies domain sanity checks, removes unreasonable physical outliers (e.g. negative pollutant
concentrations),
    and formats columns cleanly.

    Args:
        merged_df (pd.DataFrame): Raw merged DataFrame.

    Returns:
        pd.DataFrame: Thoroughly cleaned production dataset.
    """
    logging.info("Performing final sanity checks & cleaning on merged dataset...")
    df_clean = merged_df.copy()

    # Clip physical pollutant values to non-negative range
    pollutant_cols = ["PM2.5", "PM10", "NO", "NO2", "NOx", "NH3", "CO", "SO2", "O3", "Benzene",
"Toluene", "Xylene"]
    for col in pollutant_cols:
        if col in df_clean.columns:
            df_clean[col] = df_clean[col].clip(lower=0.0)

    # Clip precipitation to non-negative
    if "precipitation_mm" in df_clean.columns:
        df_clean["precipitation_mm"] = df_clean["precipitation_mm"].clip(lower=0.0)

    # Round floating metrics to 2 decimal places for cleaner reporting
    numeric_cols = df_clean.select_dtypes(include=[np.float64]).columns
    df_clean[numeric_cols] = df_clean[numeric_cols].round(2)

    # Sort deterministically
    df_clean = df_clean.sort_values(by=["City", "Date"]).reset_index(drop=True)
    return df_clean


def save_dataset(df: pd.DataFrame, output_path: str) -> None:
    """
```

```
    Saves DataFrame to CSV format after confirming target directory exists.

    Args:
        df (pd.DataFrame): DataFrame to save.
        output_path (str): Output relative file path.
    """
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    df.to_csv(output_path, index=False)
    logging.info(f"Saved dataset successfully to: '{output_path}' (Shape: {df.shape})")


def run_preprocessing_pipeline(
    aqi_path: str = "datasets/air_quality/city_day.csv",
    weather_path: str = "datasets/weather/weather.csv",
    merged_output_path: str = "datasets/processed/merged_dataset.csv",
    cleaned_output_path: str = "datasets/processed/cleaned_dataset.csv"
) -> Tuple[pd.DataFrame, pd.DataFrame]:
    """
    Executes the complete end-to-end Week 2 Data Preprocessing Pipeline.

    Returns:
        Tuple[pd.DataFrame, pd.DataFrame]: (merged_df, cleaned_df)
    """
    logging.info("================================================")
    logging.info("Starting PureBreath-AI Week 2 Preprocessing Pipeline")
    logging.info("================================================")

    # Step 1 & 3: Load & Analyze AQI Data
    aqi_df = load_aqi_data(aqi_path)

    # Step 2 & 3: Load, Spatial Filter & Aggregate Weather Data
    weather_df = load_and_aggregate_weather_data(weather_path, CITY_GRID_MAPPING)

    # Step 4: Handle Missing Values
    aqi_df = handle_missing_values(aqi_df)
    weather_df = handle_missing_values(weather_df)

    # Step 5: Remove Duplicates
    aqi_df = remove_duplicates(aqi_df)
    weather_df = remove_duplicates(weather_df)

    # Step 6: Convert Data Types
    aqi_df = convert_datatypes(aqi_df)
    weather_df = convert_datatypes(weather_df)

    # Step 7 & 8: Merge Datasets & Save merged_dataset.csv
    merged_df = merge_datasets(aqi_df, weather_df)
    save_dataset(merged_df, merged_output_path)

    # Step 9 & 10: Clean Merged Dataset & Save cleaned_dataset.csv
    cleaned_df = clean_merged_dataset(merged_df)
    save_dataset(cleaned_df, cleaned_output_path)

    logging.info("================================================")
    logging.info("Week 2 Data Preprocessing Completed Successfully!")
    logging.info("================================================")
    return merged_df, cleaned_df
```

```
if __name__ == "__main__":
    run_preprocessing_pipeline()
```

5.3.2 Feature engineering and model training (src/model_training.py)

```
"""
PureBreath-AI: Week 3 - AQI Prediction Model Training Module

This module builds, trains, evaluates, and persists XGBoost and LightGBM models
for Air Quality Index (AQI) forecasting and health risk estimation using environmental
and meteorological features, adhering to CPCB (Central Pollution Control Board) standards.

Author: PureBreath-AI Team
License: MIT
"""

import os
import json
import logging
import joblib
import pandas as pd
import numpy as np
from typing import Dict, Tuple, Any

from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Configure logging for production tracing
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler()]
)



def calculate_cpcb_pm25_subindex(pm25: float) -> float:
    """Calculates CPCB sub-index for PM2.5 based on Indian NAQI standards."""
    if pm25 <= 30:
        return pm25 * 50.0 / 30.0
    elif pm25 <= 60:
        return 50.0 + (pm25 - 30.0) * 50.0 / 30.0
    elif pm25 <= 90:
        return 100.0 + (pm25 - 60.0) * 100.0 / 30.0
    elif pm25 <= 120:
        return 200.0 + (pm25 - 90.0) * 100.0 / 30.0
    elif pm25 <= 250:
        return 300.0 + (pm25 - 120.0) * 100.0 / 130.0
    else:
        return 400.0 + (pm25 - 250.0) * 100.0 / 150.0



def calculate_cpcb_pm10_subindex(pm10: float) -> float:
    """Calculates CPCB sub-index for PM10 based on Indian NAQI standards."""
    if pm10 <= 50:
        return pm10
```

```
    elif pm10 <= 100:
        return pm10
    elif pm10 <= 250:
        return 100.0 + (pm10 - 100.0) * 100.0 / 150.0
    elif pm10 <= 350:
        return 200.0 + (pm10 - 250.0) * 100.0 / 100.0
    elif pm10 <= 430:
        return 300.0 + (pm10 - 350.0) * 100.0 / 80.0
    else:
        return 400.0 + (pm10 - 430.0) * 100.0 / 70.0


def load_and_engineer_model_features(filepath: str) -> pd.DataFrame:
    """
    Loads cleaned dataset and constructs time-series lag, rolling, and CPCB AQI features.

    Args:
        filepath (str): Relative path to cleaned_dataset.csv

    Returns:
        pd.DataFrame: Feature-engineered dataset ready for training.
    """
    if not os.path.exists(filepath):
        raise FileNotFoundError(f"Cleaned dataset not found at path: {filepath}")

    logging.info(f"Loading cleaned dataset from '{filepath}' for feature construction...")
    df = pd.read_csv(filepath)
    df["Date"] = pd.to_datetime(df["Date"])
    df = df.sort_values(by=["City", "Date"]).reset_index(drop=True)

    # Calculate CPCB NAQI Standard Target
    df["PM2.5_sub"] = df["PM2.5"].apply(calculate_cpcb_pm25_subindex)
    df["PM10_sub"] = df["PM10"].apply(calculate_cpcb_pm10_subindex)
    df["AQI_CPCB"] = df[["PM2.5_sub", "PM10_sub"]].max(axis=1)

    # 1. Temporal & Calendar Features
    df["Month"] = df["Date"].dt.month
    df["DayOfWeek"] = df["Date"].dt.dayofweek
    df["DayOfYear"] = df["Date"].dt.dayofyear
    df["Quarter"] = df["Date"].dt.quarter
    df["Is_Weekend"] = df["DayOfWeek"].isin([5, 6]).astype(int)

    # Cyclical harmonic encodings
    df["Month_sin"] = np.sin(2 * np.pi * df["Month"] / 12.0)
    df["Month_cos"] = np.cos(2 * np.pi * df["Month"] / 12.0)

    # 2. Time-Series Lag Features per City
    lag_columns = ["PM2.5", "PM10", "NO2", "SO2", "CO", "O3", "temp_mean_c"]
    for col in lag_columns:
        if col in df.columns:
            for lag in [1, 2, 7]:
                df[f"{col}_lag{lag}"] = df.groupby("City")[col].shift(lag)

    # 3. Rolling Window Statistics per City (7-day window)
    for col in ["PM2.5", "PM10"]:
        if col in df.columns:
            df[f"{col}_roll_mean_7d"] = df.groupby("City")[col].transform(lambda x:
x.shift(1).rolling(7).mean())
```

```
        df[f"{col}_roll_std_7d"] = df.groupby("City")[col].transform(lambda x:
x.shift(1).rolling(7).std())

    # 4. Temperature Range interaction feature
    df["temp_range"] = df["temp_max_c"] - df["temp_min_c"]

    # Drop NaNs introduced by lag/rolling operations
    initial_rows = len(df)
    df = df.dropna().reset_index(drop=True)
    logging.info(f"Feature engineering complete. Dataset shape: {df.shape} (Dropped {initial_rows -
len(df)} boundary lag rows)")

    return df



def prepare_train_test_split(
    df: pd.DataFrame,
    test_size: float = 0.20,
    random_state: int = 42
) -> Tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series, list]:
    """
    Performs stratified random train-test split for model training and evaluation.

    Args:
        df (pd.DataFrame): Feature-engineered DataFrame.
        test_size (float): Proportion of dataset to hold out for testing.
        random_state (int): Seed for reproducibility.

    Returns:
        Tuple: (X_train, X_test, y_train, y_test, feature_names)
    """
    logging.info(f"Preparing Train/Test split ({int((1-test_size)*100)}% Train / {int(test_size*100)}%
Test)...")

    target_col = "AQI_CPCB"
    drop_cols = ["City", "Date", "AQI_Bucket", "Date_weather", "AQI", "AQI_CPCB", "PM2.5_sub",
"PM10_sub"]

    # One-hot encode City feature
    df_encoded = pd.get_dummies(df, columns=["City"], drop_first=False)
    feature_cols = [col for col in df_encoded.columns if col not in drop_cols]

    X = df_encoded[feature_cols]
    y = df_encoded[target_col]

    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, shuffle=True
    )

    logging.info(f"Train set: {X_train.shape[0]} samples | Test set: {X_test.shape[0]} samples | Features:
{len(feature_cols)}")
    return X_train, X_test, y_train, y_test, feature_cols



def train_xgboost_model(X_train: pd.DataFrame, y_train: pd.Series) -> XGBRegressor:
    """
```

```
    Initializes and fits an optimized XGBoost Regressor model.
    """
    logging.info("Training XGBoost Regressor model...")
    xgb_model = XGBRegressor(
        n_estimators=300,
        max_depth=6,
        learning_rate=0.05,
        subsample=0.85,
        colsample_bytree=0.85,
        random_state=42,
        n_jobs=-1
    )
    xgb_model.fit(X_train, y_train)
    logging.info("XGBoost training completed successfully.")
    return xgb_model


def train_lightgbm_model(X_train: pd.DataFrame, y_train: pd.Series) -> LGBMRegressor:
    """
    Initializes and fits an optimized LightGBM Regressor model.
    """
    logging.info("Training LightGBM Regressor model...")
    lgb_model = LGBMRegressor(
        n_estimators=300,
        max_depth=6,
        num_leaves=31,
        learning_rate=0.05,
        subsample=0.85,
        colsample_bytree=0.85,
        random_state=42,
        n_jobs=-1,
        verbose=-1
    )
    lgb_model.fit(X_train, y_train)
    logging.info("LightGBM training completed successfully.")
    return lgb_model


def evaluate_models(
    models: Dict[str, Any],
    X_test: pd.DataFrame,
    y_test: pd.Series
) -> Dict[str, Dict[str, float]]:
    """
    Computes evaluation metrics (MAE, RMSE, R2 Score, MAPE) for each trained model.
    """
    logging.info("Evaluating models on holdout Test set...")
    metrics_summary = {}

    for name, model in models.items():
        y_pred = model.predict(X_test)

        mae = mean_absolute_error(y_test, y_pred)
        rmse = np.sqrt(mean_squared_error(y_test, y_pred))
        r2 = r2_score(y_test, y_pred)
        mape = np.mean(np.abs((y_test - y_pred) / np.clip(y_test, a_min=1.0, a_max=None))) * 100

        metrics_summary[name] = {
            "MAE": round(float(mae), 4),
```

```
            "RMSE": round(float(rmse), 4),
            "R2_Score": round(float(r2), 4),
            "MAPE_percent": round(float(mape), 2)
        }

        logging.info(f"[{name}] MAE: {mae:.2f} | RMSE: {rmse:.2f} | R² Score: {r2:.4f} | MAPE:
{mape:.2f}%")

    return metrics_summary


def save_model_artifacts(
    models: Dict[str, Any],
    feature_names: list,
    metrics: Dict[str, Dict[str, float]],
    models_dir: str = "models"
) -> None:
    """
    Saves trained model objects, feature metadata, and evaluation results to models/.
    """
    os.makedirs(models_dir, exist_ok=True)

    # 1. Save model binaries
    for name, model in models.items():
        filepath = os.path.join(models_dir, f"{name.lower()}_aqi_model.pkl")
        joblib.dump(model, filepath)
        logging.info(f"Saved {name} model binary to '{filepath}'")

    # 2. Save feature list metadata
    feature_path = os.path.join(models_dir, "model_features.json")
    with open(feature_path, "w") as f:
        json.dump({"features": feature_names}, f, indent=4)
    logging.info(f"Saved feature list to '{feature_path}'")

    # 3. Save evaluation metrics JSON report
    metrics_path = os.path.join(models_dir, "model_evaluation_metrics.json")
    with open(metrics_path, "w") as f:
        json.dump(metrics, f, indent=4)
    logging.info(f"Saved evaluation metrics report to '{metrics_path}'")


def run_model_training_pipeline(
    dataset_path: str = "datasets/processed/cleaned_dataset.csv"
) -> Dict[str, Dict[str, float]]:
    """
    Executes the end-to-end Week 3 Model Training Pipeline.
    """
    logging.info("================================================")
    logging.info("Starting PureBreath-AI Week 3 ML Model Training")
    logging.info("================================================")

    # 1. Load and construct features
    df = load_and_engineer_model_features(dataset_path)

    # 2. Prepare train/test splits
    X_train, X_test, y_train, y_test, feature_names = prepare_train_test_split(df)

    # 3. Train XGBoost model
```

```
    xgb_model = train_xgboost_model(X_train, y_train)

    # 4. Train LightGBM model
    lgb_model = train_lightgbm_model(X_train, y_train)

    # 5. Train Baseline Random Forest model
    logging.info("Training Random Forest baseline model...")
    rf_model = RandomForestRegressor(n_estimators=100, max_depth=6, random_state=42, n_jobs=-1)
    rf_model.fit(X_train, y_train)

    trained_models = {
        "XGBoost": xgb_model,
        "LightGBM": lgb_model,
        "RandomForest": rf_model
    }

    # 6. Evaluate all models
    metrics = evaluate_models(trained_models, X_test, y_test)

    # 7. Save model artifacts
    save_model_artifacts(trained_models, feature_names, metrics)

    logging.info("================================================")
    logging.info("Week 3 Model Training Pipeline Completed Successfully!")
    logging.info("================================================")

    return metrics


if __name__ == "__main__":
    run_model_training_pipeline()
```

5.3.3 Conditional GAN augmentation (src/gan_augmentation.py)

```
"""
PureBreath-AI: Week 4 - GAN Data Augmentation Module

Trains a Conditional GAN (cGAN) on the cleaned AQI dataset to generate
synthetic air quality samples, augmenting underrepresented AQI categories.
Saves the combined real + synthetic dataset for downstream training.

Author: PureBreath-AI Team
License: MIT
"""

import os
import json
import logging
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
from sklearn.preprocessing import StandardScaler, LabelEncoder

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
```

```
    handlers=[logging.StreamHandler()]
)

# ── Reproducibility ──────────────────────────────────────────────────────────
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

# ── Hyper-parameters ─────────────────────────────────────────────────────────
LATENT_DIM   = 64
HIDDEN_DIM   = 256
BATCH_SIZE   = 256
EPOCHS       = 500
LR           = 2e-4
DEVICE       = torch.device("cuda" if torch.cuda.is_available() else "cpu")

FEATURE_COLS = [
    "PM2.5", "PM10", "NO", "NO2", "NOx", "NH3", "CO", "SO2", "O3",
    "Benzene", "Toluene", "Xylene",
    "temp_mean_c", "temp_min_c", "temp_max_c", "precipitation_mm", "pressure_hgt",
]
LABEL_COL    = "AQI_Bucket"
SAMPLES_PER_CLASS = 500          # synthetic samples to generate per AQI class


# ── Architecture ─────────────────────────────────────────────────────────────

class Generator(nn.Module):
    def __init__(self, latent_dim: int, n_classes: int, out_dim: int, hidden: int):
        super().__init__()
        self.label_emb = nn.Embedding(n_classes, n_classes)
        self.net = nn.Sequential(
            nn.Linear(latent_dim + n_classes, hidden),
            nn.LayerNorm(hidden),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden, hidden),
            nn.LayerNorm(hidden),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden, out_dim),
        )

    def forward(self, z: torch.Tensor, labels: torch.Tensor) -> torch.Tensor:
        c = self.label_emb(labels)
        return self.net(torch.cat([z, c], dim=1))


class Discriminator(nn.Module):
    def __init__(self, in_dim: int, n_classes: int, hidden: int):
        super().__init__()
        self.label_emb = nn.Embedding(n_classes, n_classes)
        self.net = nn.Sequential(
            nn.Linear(in_dim + n_classes, hidden),
            nn.LeakyReLU(0.2),
```

```
            nn.Dropout(0.3),
            nn.Linear(hidden, hidden // 2),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),
            nn.Linear(hidden // 2, 1),
            nn.Sigmoid(),
        )

    def forward(self, x: torch.Tensor, labels: torch.Tensor) -> torch.Tensor:
        c = self.label_emb(labels)
        return self.net(torch.cat([x, c], dim=1))


# ── Data helpers ──────────────────────────────────────────────────────────────

def load_features(filepath: str):
    """Load cleaned dataset and return scaled features, encoded labels, helpers."""
    df = pd.read_csv(filepath)
    df = df.dropna(subset=FEATURE_COLS + [LABEL_COL])

    le = LabelEncoder()
    labels = le.fit_transform(df[LABEL_COL].astype(str))

    scaler = StandardScaler()
    features = scaler.fit_transform(df[FEATURE_COLS].values.astype(np.float32))

    logging.info(f"Loaded {len(df)} samples | Classes: {list(le.classes_)}")
    class_counts = pd.Series(labels).value_counts().sort_index()
    for idx, cnt in class_counts.items():
        logging.info(f"  [{le.classes_[idx]}] → {cnt} samples")

    return features, labels, le, scaler, df


# ── Training ──────────────────────────────────────────────────────────────────

def train_cgan(features: np.ndarray, labels: np.ndarray, n_classes: int):
    """Train Conditional GAN and return trained Generator."""
    feat_tensor  = torch.tensor(features, dtype=torch.float32)
    label_tensor = torch.tensor(labels,   dtype=torch.long)

    dataset    = TensorDataset(feat_tensor, label_tensor)
    dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, drop_last=True)

    out_dim = features.shape[1]
    G = Generator(LATENT_DIM, n_classes, out_dim, HIDDEN_DIM).to(DEVICE)
    D = Discriminator(out_dim, n_classes, HIDDEN_DIM).to(DEVICE)

    opt_G = torch.optim.Adam(G.parameters(), lr=LR, betas=(0.5, 0.999))
    opt_D = torch.optim.Adam(D.parameters(), lr=LR, betas=(0.5, 0.999))
    criterion = nn.BCELoss()

    for epoch in range(1, EPOCHS + 1):
```

```
        g_losses, d_losses = [], []

        for real_x, real_y in dataloader:
            real_x, real_y = real_x.to(DEVICE), real_y.to(DEVICE)
            bs = real_x.size(0)

            real_lbl = torch.ones(bs, 1, device=DEVICE)
            fake_lbl = torch.zeros(bs, 1, device=DEVICE)

            # ── Train Discriminator ──────────────────────────────────────
            z      = torch.randn(bs, LATENT_DIM, device=DEVICE)
            fake_x = G(z, real_y).detach()

            d_real = criterion(D(real_x, real_y), real_lbl)
            d_fake = criterion(D(fake_x, real_y), fake_lbl)
            d_loss = (d_real + d_fake) / 2

            opt_D.zero_grad()
            d_loss.backward()
            opt_D.step()

            # ── Train Generator ──────────────────────────────────────────
            z      = torch.randn(bs, LATENT_DIM, device=DEVICE)
            fake_x = G(z, real_y)
            g_loss = criterion(D(fake_x, real_y), real_lbl)

            opt_G.zero_grad()
            g_loss.backward()
            opt_G.step()

            g_losses.append(g_loss.item())
            d_losses.append(d_loss.item())

        if epoch % 100 == 0:
            logging.info(
                f"Epoch [{epoch:>4}/{EPOCHS}] "
                f"G_loss: {np.mean(g_losses):.4f} | D_loss: {np.mean(d_losses):.4f}"
            )

    return G


# ── Synthesis ──────────────────────────────────────────────────────────────────


def generate_synthetic(
    G: Generator,
    le: LabelEncoder,
    scaler: StandardScaler,
    samples_per_class: int,
) -> pd.DataFrame:
    """Generate synthetic samples for every AQI class and inverse-transform."""
    G.eval()
    n_classes   = len(le.classes_)
    all_samples = []
```

```
    with torch.no_grad():
        for class_idx in range(n_classes):
            z      = torch.randn(samples_per_class, LATENT_DIM, device=DEVICE)
            labels = torch.full((samples_per_class,), class_idx, dtype=torch.long, device=DEVICE)
            fake   = G(z, labels).cpu().numpy()
            fake   = scaler.inverse_transform(fake)

            df_syn = pd.DataFrame(fake, columns=FEATURE_COLS)
            df_syn[LABEL_COL] = le.classes_[class_idx]
            all_samples.append(df_syn)

    synthetic_df = pd.concat(all_samples, ignore_index=True)
    logging.info(f"Generated {len(synthetic_df)} synthetic samples across {n_classes} classes.")
    return synthetic_df


# ── Main pipeline ──────────────────────────────────────────────────────────────

def run_gan_augmentation(
    input_path:  str = "datasets/processed/cleaned_dataset.csv",
    output_path: str = "datasets/processed/augmented_dataset.csv",
    meta_path:   str = "models/gan_augmentation_meta.json",
):
    logging.info("=== Week 4: GAN Data Augmentation ===")
    logging.info(f"Device: {DEVICE}")

    # 1. Load data
    features, labels, le, scaler, df_real = load_features(input_path)
    n_classes = len(le.classes_)

    # 2. Train cGAN
    logging.info(f"Training cGAN for {EPOCHS} epochs …")
    G = train_cgan(features, labels, n_classes)

    # 3. Generate synthetic samples
    df_synthetic = generate_synthetic(G, le, scaler, SAMPLES_PER_CLASS)
    df_synthetic["source"] = "synthetic"
    df_real["source"]      = "real"

    # 4. Merge and save
    keep_cols = FEATURE_COLS + [LABEL_COL, "source"]
    df_real_slim = df_real[keep_cols].copy()
    df_augmented = pd.concat([df_real_slim, df_synthetic], ignore_index=True)
    df_augmented.to_csv(output_path, index=False)
    logging.info(f"Augmented dataset saved → '{output_path}' ({len(df_augmented)} rows)")

    # 5. Save metadata
    os.makedirs(os.path.dirname(meta_path), exist_ok=True)
    meta = {
        "real_samples":      int(len(df_real)),
        "synthetic_samples": int(len(df_synthetic)),
        "total_samples":     int(len(df_augmented)),
        "aqi_classes":       list(le.classes_),
        "samples_per_class": SAMPLES_PER_CLASS,
        "feature_cols":      FEATURE_COLS,
```

```
        "epochs":           EPOCHS,
        "latent_dim":       LATENT_DIM,
        "device":           str(DEVICE),
    }
    with open(meta_path, "w") as f:
        json.dump(meta, f, indent=4)
    logging.info(f"GAN metadata saved → '{meta_path}'")

    return df_augmented


if __name__ == "__main__":
    run_gan_augmentation(
        input_path  = "datasets/processed/cleaned_dataset.csv",
        output_path = "datasets/processed/augmented_dataset.csv",
        meta_path   = "models/gan_augmentation_meta.json",
    )
```

5.3.4 SHAP and counterfactual module (src/xai_explainer.py)

```
"""
PureBreath-AI: Week 7 - SHAP + Counterfactual Explainable AI Module

Implements two complementary XAI components described in the abstract:

1. SHAP Feature Attribution
   - Computes SHAP values for LightGBM predictions on a sample of test rows
   - Identifies top driving features for each prediction
   - Saves global feature importance + per-sample local explanations

2. Counterfactual What-If Recommendations
   - For each high-AQI prediction, generates actionable counterfactuals:
     "If PM2.5 dropped by X%, AQI would fall from Y to Z (Severe → Moderate)"
   - Targets the top-3 most impactful SHAP features per sample
   - Translates feature deltas into plain-language recommendations

Author: PureBreath-AI Team
License: MIT
"""

import os
import json
import logging
import joblib
import shap
import numpy as np
import pandas as pd
from typing import Dict, List, Tuple

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler()]
)

# ── AQI bucket helper ─────────────────────────────────────────────────────────
```

```
def aqi_to_bucket(aqi: float) -> str:
    if aqi <= 50:   return "Good"
    if aqi <= 100:  return "Satisfactory"
    if aqi <= 200:  return "Moderate"
    if aqi <= 300:  return "Poor"
    if aqi <= 400:  return "Very Poor"
    return "Severe"


# ── Plain-language counterfactual templates ──────────────────────────────────

REDUCTION_TEMPLATES = {
    "PM2.5":           "Reduce PM2.5 by {pct:.0f}% (e.g. avoid biomass burning, use cleaner fuels).",
    "PM10":            "Reduce PM10 by {pct:.0f}% (e.g. water roads, control construction dust).",
    "NO2":             "Reduce NO2 by {pct:.0f}% (e.g. reduce vehicle traffic, promote EVs).",
    "SO2":             "Reduce SO2 by {pct:.0f}% (e.g. switch to low-sulphur fuels in industry).",
    "CO":              "Reduce CO by {pct:.0f}% (e.g. improve combustion efficiency, reduce idling).",
    "O3":              "Reduce O3 precursors by {pct:.0f}% (e.g. limit VOC and NOx emissions).",
    "NH3":             "Reduce NH3 by {pct:.0f}% (e.g. better agricultural waste management).",
    "NOx":             "Reduce NOx by {pct:.0f}% (e.g. catalytic converters, cleaner engines).",
    "Benzene":         "Reduce Benzene by {pct:.0f}% (e.g. stricter fuel quality standards).",
    "temp_mean_c":     "A {pct:.0f}% drop in temperature would reduce AQI — monitor heat-island
effects.",
    "precipitation_mm": "Increased precipitation by {pct:.0f}% would wash out pollutants.",
    "PM2.5_lag1":      "Sustained PM2.5 reduction yesterday by {pct:.0f}% would lower today's AQI.",
    "PM10_lag1":       "Sustained PM10 reduction yesterday by {pct:.0f}% would lower today's AQI.",
    "PM2.5_roll_mean_7d": "A {pct:.0f}% reduction in 7-day average PM2.5 would significantly lower
AQI.",
    "PM10_roll_mean_7d":  "A {pct:.0f}% reduction in 7-day average PM10 would significantly lower
AQI.",
}

DEFAULT_TEMPLATE = "Reduce {feat} by {pct:.0f}% to lower AQI contribution."

TARGET_REDUCTION_PCT = 20.0   # % reduction applied to each top feature in counterfactual


# ── SHAP explanation ─────────────────────────────────────────────────────────

def compute_shap_explanations(
    model,
    X_sample: pd.DataFrame,
    model_name: str,
) -> Tuple[np.ndarray, pd.Series]:
    """
    Compute SHAP values for X_sample using TreeExplainer.
    Returns (shap_values array, global_importance Series).
    """
    logging.info(f"Computing SHAP values for {model_name} on {len(X_sample)} samples...")
    explainer   = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(X_sample)

    global_importance = pd.Series(
        np.abs(shap_values).mean(axis=0),
        index=X_sample.columns,
    ).sort_values(ascending=False)
```

```
    logging.info(f"Top-5 global features [{model_name}]: "
                 f"{list(global_importance.head(5).index)}")
    return shap_values, global_importance


# ── Counterfactual generation ──────────────────────────────────────────────

def generate_counterfactual(
    model,
    row: pd.Series,
    shap_row: np.ndarray,
    feature_names: List[str],
    top_k: int = 3,
    reduction_pct: float = TARGET_REDUCTION_PCT,
) -> Dict:
    """
    For a single prediction row, generate a counterfactual by reducing
    the top-k positive SHAP features by reduction_pct and re-predicting.
    """
    original_aqi    = float(model.predict(row.values.reshape(1, -1))[0])
    original_bucket = aqi_to_bucket(original_aqi)

    # Only target features with positive SHAP (they push AQI up)
    shap_series = pd.Series(shap_row, index=feature_names)
    top_features = shap_series[shap_series > 0].nlargest(top_k).index.tolist()

    if not top_features:
        top_features = shap_series.abs().nlargest(top_k).index.tolist()

    # Apply reduction
    cf_row = row.copy()
    for feat in top_features:
        if cf_row[feat] > 0:
            cf_row[feat] = cf_row[feat] * (1 - reduction_pct / 100.0)

    cf_aqi    = float(model.predict(cf_row.values.reshape(1, -1))[0])
    cf_bucket = aqi_to_bucket(cf_aqi)
    aqi_drop  = round(original_aqi - cf_aqi, 2)

    # Build plain-language recommendations
    recommendations = []
    for feat in top_features:
        tmpl = REDUCTION_TEMPLATES.get(feat, DEFAULT_TEMPLATE)
        recommendations.append(tmpl.format(feat=feat, pct=reduction_pct))

    return {
        "original_aqi":      round(original_aqi, 2),
        "original_bucket":   original_bucket,
        "counterfactual_aqi": round(cf_aqi, 2),
        "cf_bucket":         cf_bucket,
        "aqi_reduction":     aqi_drop,
        "top_features":      top_features,
        "recommendations":   recommendations,
    }
```

```
# ── Main XAI pipeline ──────────────────────────────────────────────────────────

def run_xai_pipeline(
    model_path:    str = "models/lightgbm_aqi_model.pkl",
    features_path: str = "models/model_features.json",
    data_path:     str = "datasets/processed/cleaned_dataset.csv",
    output_dir:    str = "reports/xai",
    n_samples:     int = 200,
):
    logging.info("=== Week 7: SHAP + Counterfactual Explainable AI ===")
    os.makedirs(output_dir, exist_ok=True)

    # 1. Load model and features
    model = joblib.load(model_path)
    with open(features_path) as f:
        feature_names = json.load(f)["features"]
    logging.info(f"Loaded model from '{model_path}' | {len(feature_names)} features")

    # 2. Load and prepare data (same pipeline as model_training.py)
    import sys
    sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
    from model_training import load_and_engineer_model_features
    df = load_and_engineer_model_features(data_path)
    df_encoded = pd.get_dummies(df, columns=["City"], drop_first=False)

    # Align columns to trained feature set
    for col in feature_names:
        if col not in df_encoded.columns:
            df_encoded[col] = 0
    X = df_encoded[feature_names]

    # Sample rows — focus on high-AQI cases for meaningful counterfactuals
    high_aqi_mask = df["AQI_CPCB"] >= 200
    X_high  = X[high_aqi_mask].head(n_samples // 2)
    X_other = X[~high_aqi_mask].head(n_samples // 2)
    X_sample = pd.concat([X_high, X_other]).reset_index(drop=True)
    logging.info(f"Sample: {len(X_sample)} rows ({len(X_high)} high-AQI + {len(X_other)} other)")

    # 3. SHAP explanations
    shap_values, global_importance = compute_shap_explanations(
        model, X_sample, model_name="LightGBM"
    )

    # Save global feature importance
    gi_path = os.path.join(output_dir, "shap_global_importance.csv")
    global_importance.reset_index().rename(
        columns={"index": "Feature", 0: "Mean_SHAP"}
    ).to_csv(gi_path, index=False)
    logging.info(f"Global SHAP importance saved → '{gi_path}'")

    # Save per-sample SHAP values
    shap_df = pd.DataFrame(shap_values, columns=feature_names)
    shap_df.to_csv(os.path.join(output_dir, "shap_values_sample.csv"), index=False)

    # 4. Counterfactual explanations
    logging.info("Generating counterfactual explanations...")
    cf_records = []
```

```
    for i in range(len(X_sample)):
        cf = generate_counterfactual(
            model        = model,
            row          = X_sample.iloc[i],
            shap_row     = shap_values[i],
            feature_names= feature_names,
        )
        cf["sample_idx"] = i
        cf_records.append(cf)

    # Flatten for CSV (recommendations as semicolon-joined string)
    cf_flat = []
    for r in cf_records:
        cf_flat.append({
            "sample_idx":         r["sample_idx"],
            "original_aqi":       r["original_aqi"],
            "original_bucket":    r["original_bucket"],
            "counterfactual_aqi": r["counterfactual_aqi"],
            "cf_bucket":          r["cf_bucket"],
            "aqi_reduction":      r["aqi_reduction"],
            "top_features":       ", ".join(r["top_features"]),
            "recommendations":    " | ".join(r["recommendations"]),
        })

    cf_df = pd.DataFrame(cf_flat)
    cf_path = os.path.join(output_dir, "counterfactual_explanations.csv")
    cf_df.to_csv(cf_path, index=False)
    logging.info(f"Counterfactual explanations saved → '{cf_path}'")

    # 5. Summary stats
    improved = (cf_df["aqi_reduction"] > 0).sum()
    bucket_changes = (cf_df["original_bucket"] != cf_df["cf_bucket"]).sum()
    mean_drop = cf_df["aqi_reduction"].mean()

    logging.info(f"Counterfactual summary:")
    logging.info(f"  Samples improved:       {improved}/{len(cf_df)}")
    logging.info(f"  Bucket category change: {bucket_changes}/{len(cf_df)}")
    logging.info(f"  Mean AQI reduction:     {mean_drop:.2f}")

    # Save summary JSON
    summary = {
        "model":                 "LightGBM",
        "n_samples":             len(X_sample),
        "top5_global_features":  list(global_importance.head(5).index),
        "cf_samples_improved":   int(improved),
        "cf_bucket_changes":     int(bucket_changes),
        "cf_mean_aqi_reduction": round(float(mean_drop), 2),
        "reduction_pct_applied": TARGET_REDUCTION_PCT,
    }
    summary_path = os.path.join(output_dir, "xai_summary.json")
    with open(summary_path, "w") as f:
        json.dump(summary, f, indent=4)
    logging.info(f"XAI summary saved → '{summary_path}'")

    return summary


if __name__ == "__main__":
    run_xai_pipeline(
```

```
        model_path    = "models/lightgbm_aqi_model.pkl",
        features_path = "models/model_features.json",
        data_path     = "datasets/processed/cleaned_dataset.csv",
        output_dir    = "reports/xai",
        n_samples     = 200,
    )
```

5.3.5 Personalized health-risk engine (src/health_risk_assessment.py)

```
"""
PureBreath-AI: Week 5 - Personalized Health Risk Assessment Engine

Computes an individualized health risk score for each user based on:
  - Predicted AQI (from Week 3 models)
  - Age group vulnerability
  - Asthma severity
  - Pregnancy status
  - Occupation-based outdoor exposure
  - Daily outdoor hours

Outputs a risk score (0–100), risk level, and actionable health advisory
for every user in the user profile dataset.

Author: PureBreath-AI Team
License: MIT
"""

import os
import json
import logging
import joblib
import numpy as np
import pandas as pd
from typing import Dict, Tuple

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler()]
)

# ── Risk weight tables (evidence-based, WHO / CPCB guidelines) ──────────────

# AQI band → base risk score (0–100)
AQI_BASE_RISK: Dict[str, float] = {
    "Good":        10.0,
    "Satisfactory":25.0,
    "Moderate":    45.0,
    "Poor":        62.0,
    "Very Poor":   78.0,
    "Severe":      95.0,
}

# Age group multipliers
def age_multiplier(age: int) -> float:
    if age <= 5:   return 1.8   # infants / toddlers
    if age <= 12:  return 1.5   # children
    if age <= 17:  return 1.2   # adolescents
    if age <= 60:  return 1.0   # healthy adults (baseline)
```

```
    if age <= 70:  return 1.3   # older adults
    return 1.6                  # elderly >70

# Occupations with elevated outdoor / pollutant exposure
HIGH_EXPOSURE_OCCUPATIONS = {
    "Traffic Police":       1.5,
    "Construction Worker":  1.4,
    "Factory Worker":       1.4,
    "Delivery Executive":   1.3,
    "Farmer":               1.3,
    "Driver":               1.25,
    "Shopkeeper":           1.1,
}

# Risk level thresholds
def risk_level(score: float) -> str:
    if score < 20:  return "Low"
    if score < 40:  return "Moderate"
    if score < 60:  return "High"
    if score < 80:  return "Very High"
    return "Hazardous"

# ── Advisory messages ──────────────────────────────────────────────────────

ADVISORIES: Dict[str, str] = {
    "Low": (
        "Air quality is acceptable. Enjoy outdoor activities. "
        "Sensitive individuals should monitor symptoms."
    ),
    "Moderate": (
        "Unusually sensitive people should consider reducing prolonged outdoor exertion. "
        "Keep windows open for ventilation."
    ),
    "High": (
        "Reduce prolonged or heavy outdoor exertion. Take more breaks during outdoor activities. "
        "Wear an N95 mask if outdoors for extended periods."
    ),
    "Very High": (
        "Avoid prolonged outdoor exertion. People with respiratory or heart conditions, "
        "children and elderly should stay indoors. Use air purifiers indoors."
    ),
    "Hazardous": (
        "Stay indoors. Avoid all outdoor physical activity. Keep windows and doors closed. "
        "Use N95/P100 respirator if going outside is unavoidable. Seek medical advice if symptoms
appear."
    ),
}

# Additional condition-specific advisories
ASTHMA_ADDON    = " Carry rescue inhaler at all times. Avoid triggers like dust and smoke."
PREGNANCY_ADDON = " Pregnant women should minimise outdoor exposure and consult a
physician."
CHILD_ADDON     = " Keep children indoors during peak pollution hours (7–10 AM, 6–9 PM)."
ELDERLY_ADDON   = " Elderly individuals should avoid outdoor exercise entirely today."



# ── Core risk computation ──────────────────────────────────────────────────
```

```
def compute_risk(
    aqi_bucket:     str,
    age:            int,
    asthma:         bool,
    pregnancy:      bool,
    occupation:     str,
    outdoor_hours:  float,
) -> Tuple[float, str, str]:
    """
    Compute personalised risk score, level, and advisory for one user.

    Returns
    -------
    risk_score  : float  (0–100, clipped)
    level       : str
    advisory    : str
    """
    base        = AQI_BASE_RISK.get(aqi_bucket, 50.0)
    age_mult    = age_multiplier(age)
    occ_mult    = HIGH_EXPOSURE_OCCUPATIONS.get(occupation, 1.0)

    # Outdoor hours modifier: each hour above 2 adds 2 % extra risk
    outdoor_mod = 1.0 + max(0.0, outdoor_hours - 2.0) * 0.02

    # Condition multipliers
    asthma_mult    = 1.35 if asthma    else 1.0
    pregnancy_mult = 1.25 if pregnancy else 1.0

    score = base * age_mult * occ_mult * outdoor_mod * asthma_mult * pregnancy_mult
    score = float(np.clip(score, 0.0, 100.0))

    level    = risk_level(score)
    advisory = ADVISORIES[level]

    if asthma:
        advisory += ASTHMA_ADDON
    if pregnancy:
        advisory += PREGNANCY_ADDON
    if age <= 12:
        advisory += CHILD_ADDON
    if age > 70:
        advisory += ELDERLY_ADDON

    return round(score, 2), level, advisory


# ── Batch assessment ──────────────────────────────────────────────────────────

def assess_all_users(
    users_path: str,
    aqi_value:  float,
    aqi_bucket: str,
    city:       str = "Unknown",
    date:       str = "Today",
) -> pd.DataFrame:
    """
```

```
    Run risk assessment for every user in the profile CSV.

    Parameters
    ----------
    users_path : path to user_profile_100.csv
    aqi_value  : numeric AQI predicted by Week 3 model
    aqi_bucket : AQI category string
    city       : city name for context
    date       : forecast date string

    Returns
    -------
    DataFrame with one row per user including risk score, level, advisory.
    """
    df = pd.read_csv(users_path)
    results = []

    for _, row in df.iterrows():
        score, level, advisory = compute_risk(
            aqi_bucket    = aqi_bucket,
            age           = int(row["Age"]),
            asthma        = str(row["Asthma"]).strip().lower() == "yes",
            pregnancy     = str(row["Pregnancy"]).strip().lower() == "yes",
            occupation    = str(row["Occupation"]).strip(),
            outdoor_hours = float(row["Outdoor_Hours"]),
        )
        results.append({
            "User_ID":       row["User_ID"],
            "Age":           row["Age"],
            "Gender":        row["Gender"],
            "Asthma":        row["Asthma"],
            "Pregnancy":     row["Pregnancy"],
            "Occupation":    row["Occupation"],
            "Outdoor_Hours": row["Outdoor_Hours"],
            "City":          city,
            "Date":          date,
            "AQI_Value":     aqi_value,
            "AQI_Bucket":    aqi_bucket,
            "Risk_Score":    score,
            "Risk_Level":    level,
            "Advisory":      advisory,
        })

    return pd.DataFrame(results)


# ── AQI bucket from numeric value (CPCB standard) ──────────────────────────

def aqi_to_bucket(aqi: float) -> str:
    if aqi <= 50:   return "Good"
    if aqi <= 100:  return "Satisfactory"
    if aqi <= 200:  return "Moderate"
    if aqi <= 300:  return "Poor"
    if aqi <= 400:  return "Very Poor"
    return "Severe"
```

```
#
═══════════════════════════════════════════════════════════════════════════
# AI HEALTH REPORT ENGINE
# Generates a fully dynamic, personalized health report from user + AQI data.
# Designed to be LLM-replaceable: swap generate_health_report() with an LLM
# call and the rest of the pipeline stays unchanged.
#
═══════════════════════════════════════════════════════════════════════════

_BUCKET_ORDER = ["Good", "Satisfactory", "Moderate", "Poor", "Very Poor", "Severe"]

_OUTDOOR_SAFE_HOURS = {
    "Good": 8, "Satisfactory": 6, "Moderate": 4,
    "Poor": 2, "Very Poor": 1, "Severe": 0,
}

_BEST_TIME = {
    "Good":         "Any time of day is suitable.",
    "Satisfactory": "Morning (6–9 AM) or evening (after 7 PM) is preferable.",
    "Moderate":     "Early morning (before 7 AM) when pollution is lowest.",
    "Poor":         "Avoid peak hours (7–10 AM, 5–9 PM). If necessary, go out between 11 AM–2 PM.",
    "Very Poor":    "Minimize all outdoor time. If unavoidable, early morning is least harmful.",
    "Severe":       "Do not go outdoors. All outdoor activity is unsafe.",
}

_HIGH_EXPOSURE_OCCS = {
    "Traffic Police", "Construction Worker", "Factory Worker",
    "Delivery Executive", "Farmer", "Driver",
}


def _bucket_index(bucket: str) -> int:
    try:
        return _BUCKET_ORDER.index(bucket)
    except ValueError:
        return 2


def _age_group(age: int) -> str:
    if age <= 5:   return "infant"
    if age <= 12:  return "child"
    if age <= 17:  return "adolescent"
    if age <= 60:  return "adult"
    if age <= 70:  return "older adult"
    return "elderly"


def _is_sensitive(age: int, asthma: bool, pregnancy: bool) -> bool:
    return asthma or pregnancy or age <= 12 or age > 70


def _health_score(
    aqi: float, risk_score: float, outdoor_hours: float,
    asthma: bool, pregnancy: bool, age: int,
) -> int:
    """Invert risk into a 0-100 health score (100 = best)."""
```

```
    base = max(0.0, 100.0 - risk_score)
    # Penalise excess outdoor hours relative to safe limit
    bucket = aqi_to_bucket(aqi)
    safe_h = _OUTDOOR_SAFE_HOURS[bucket]
    excess = max(0.0, outdoor_hours - safe_h)
    penalty = min(excess * 3.0, 20.0)
    score = max(0.0, base - penalty)
    return int(round(score))


def _ai_summary(
    aqi: float, bucket: str, risk_level: str, risk_score: float,
    age: int, gender: str, asthma: bool, pregnancy: bool,
    occupation: str, outdoor_hours: float,
    top_shap: list,
) -> str:
    """Generate a natural-language risk summary from data."""
    age_grp = _age_group(age)
    parts = [
        f"Based on the current AQI of {aqi:.0f} ({bucket}), "
        f"your overall health risk is assessed as {risk_level} "
        f"(score: {risk_score:.0f}/100)."
    ]

    env_drivers = [f["feature"] for f in top_shap[:3]] if top_shap else []
    if env_drivers:
        parts.append(
            f"The primary environmental drivers are {', '.join(env_drivers)}, "
            "which are currently elevated above safe thresholds."
        )

    personal = []
    if asthma:
        personal.append("a pre-existing asthma condition")
    if pregnancy:
        personal.append("pregnancy")
    if age <= 12:
        personal.append(f"your age ({age}) placing you in the high-vulnerability child group")
    elif age > 70:
        personal.append(f"your age ({age}) placing you in the high-vulnerability elderly group")
    if occupation in _HIGH_EXPOSURE_OCCS:
        personal.append(f"your occupation ({occupation}) involving prolonged outdoor pollutant
exposure")
    if outdoor_hours > _OUTDOOR_SAFE_HOURS.get(bucket, 4):
        personal.append(
            f"your daily outdoor exposure ({outdoor_hours:.1f} hrs) exceeding the "
            f"recommended safe limit for {bucket} air quality"
        )

    if personal:
        parts.append("Personal risk factors include: " + "; ".join(personal) + ".")
    else:
        parts.append("No major personal risk amplifiers were detected beyond the ambient AQI.")

    return " ".join(parts)


def _health_effects(
    bucket: str, age: int, asthma: bool, pregnancy: bool, outdoor_hours: float,
```

```
) -> list:
    """Return only the health effects relevant to this user."""
    idx = _bucket_index(bucket)
    effects = []

    # Universal effects by severity
    if idx >= 2:  # Moderate+
        effects.append({"icon": "□□", "effect": "Mild respiratory irritation",
                        "term": "Short-term"})
    if idx >= 3:  # Poor+
        effects.append({"icon": "□", "effect": "Reduced lung function and breathing difficulty",
                        "term": "Short-term"})
        effects.append({"icon": "□", "effect": "Fatigue and reduced physical performance",
                        "term": "Short-term"})
    if idx >= 4:  # Very Poor+
        effects.append({"icon": "□", "effect": "Cardiovascular stress and elevated heart rate",
                        "term": "Short-term"})
        effects.append({"icon": "□", "effect": "Eye and throat irritation",
                        "term": "Short-term"})
    if idx >= 5:  # Severe
        effects.append({"icon": "□", "effect": "Acute respiratory distress",
                        "term": "Short-term"})
        effects.append({"icon": "□", "effect": "Risk of hospitalization for vulnerable groups",
                        "term": "Short-term"})

    # Condition-specific
    if asthma and idx >= 2:
        effects.append({"icon": "□", "effect": "Increased risk of asthma attack or exacerbation",
                        "term": "Short-term"})
    if pregnancy and idx >= 2:
        effects.append({"icon": "□", "effect": "Potential impact on fetal development and birth weight",
                        "term": "Long-term"})
    if age <= 12 and idx >= 2:
        effects.append({"icon": "□", "effect": "Impaired lung development in children",
                        "term": "Long-term"})
    if age > 60 and idx >= 3:
        effects.append({"icon": "□", "effect": "Aggravation of existing cardiovascular or pulmonary
conditions",
                        "term": "Short-term"})

    # Long-term chronic exposure
    if outdoor_hours >= 4 and idx >= 3:
        effects.append({"icon": "□", "effect": "Chronic lung disease risk from prolonged daily exposure",
                        "term": "Long-term"})

    return effects


def _recommendations(
    aqi: float, bucket: str, age: int, asthma: bool,
    pregnancy: bool, occupation: str, outdoor_hours: float, gender: str,
) -> list:
    """Build a prioritized, non-redundant recommendation list from data."""
    idx = _bucket_index(bucket)
    recs = []

    # Mask
    if idx >= 3:
        recs.append({"priority": "High", "icon": "□",
```

```
                "text": "Wear an N95 or N99 mask when outdoors."})
    elif idx == 2:
        recs.append({"priority": "Medium", "icon": "□",
                "text": "Consider wearing a surgical mask during outdoor activities."})

    # Outdoor duration
    safe_h = _OUTDOOR_SAFE_HOURS[bucket]
    if outdoor_hours > safe_h:
        recs.append({"priority": "High", "icon": "□",
                "text": f"Reduce outdoor exposure to under {safe_h} hour(s) per day."})
    elif idx >= 2:
        recs.append({"priority": "Medium", "icon": "□",
                "text": f"Limit outdoor time to {safe_h} hours or less today."})

    # Exercise
    if idx >= 3:
        recs.append({"priority": "High", "icon": "□",
                "text": "Avoid strenuous outdoor exercise. Move workouts indoors."})
    elif idx == 2:
        recs.append({"priority": "Medium", "icon": "□",
                "text": "Reduce intensity of outdoor exercise."})

    # Windows
    if idx >= 3:
        recs.append({"priority": "High", "icon": "□",
                "text": "Keep windows and doors closed during peak pollution hours (7–10 AM, 5–9
PM)."})

    # Air purifier
    if idx >= 3:
        recs.append({"priority": "Medium", "icon": "□",
                "text": "Use an indoor HEPA air purifier, especially in bedrooms."})

    # Hydration
    if idx >= 2:
        recs.append({"priority": "Low", "icon": "□",
                "text": "Increase water intake to help flush airway irritants."})

    # Asthma-specific
    if asthma:
        recs.append({"priority": "High", "icon": "□",
                "text": "Keep your rescue inhaler accessible at all times."})
        if idx >= 2:
            recs.append({"priority": "High", "icon": "□",
                    "text": "Follow your asthma action plan and monitor peak flow readings."})

    # Pregnancy-specific
    if pregnancy:
        recs.append({"priority": "High", "icon": "□",
                "text": "Minimize outdoor exposure and consult your physician about air quality risks."})

    # Children
    if age <= 12:
        recs.append({"priority": "High", "icon": "□",
                "text": "Keep children indoors during peak pollution hours (7–10 AM, 6–9 PM)."})

    # Elderly
    if age > 70:
```

```
        recs.append({"priority": "High", "icon": "□",
                     "text": "Avoid all outdoor physical activity. Stay in a well-ventilated indoor space."})

    # High-exposure occupation
    if occupation in _HIGH_EXPOSURE_OCCS:
        recs.append({"priority": "High", "icon": "□",
                     "text": f"As a {occupation}, request PPE (N95 mask) from your employer and take regular
indoor breaks."})

    # Symptom monitoring
    if idx >= 2:
        recs.append({"priority": "Low", "icon": "□",
                     "text": "Monitor for symptoms: coughing, wheezing, chest tightness, or eye irritation."})

    # Sort: High → Medium → Low
    order = {"High": 0, "Medium": 1, "Low": 2}
    recs.sort(key=lambda r: order.get(r["priority"], 3))
    return recs



def _daily_advice(
    bucket: str, outdoor_hours: float, age: int,
    asthma: bool, occupation: str,
) -> dict:
    """Generate daily outdoor exposure guidance."""
    safe_h = _OUTDOOR_SAFE_HOURS[bucket]
    idx = _bucket_index(bucket)

    avoid = []
    if idx >= 3:
        avoid.append("Running, cycling, or any vigorous outdoor exercise")
        avoid.append("Outdoor activities during rush hours (7–10 AM, 5–9 PM)")
    if idx >= 4:
        avoid.append("Opening windows during daytime")
        avoid.append("Prolonged time in traffic or near construction sites")
    if asthma and idx >= 2:
        avoid.append("Exposure to dust, smoke, or strong odors")
    if age <= 12 and idx >= 2:
        avoid.append("Outdoor sports or recess during school hours")

    exercise_safe = idx <= 1 or (idx == 2 and not asthma and age > 12 and age <= 60)

    return {
        "safe_outdoor_hours": safe_h,
        "best_time":          _BEST_TIME[bucket],
        "activities_to_avoid": avoid if avoid else ["No specific restrictions today."],
        "outdoor_exercise_safe": exercise_safe,
        "exercise_note": (
            "Outdoor exercise is safe today." if exercise_safe
            else "Move exercise indoors or postpone until air quality improves."
        ),
    }



def _high_risk_alerts(
    aqi: float, bucket: str, asthma: bool, pregnancy: bool,
    age: int, occupation: str, outdoor_hours: float,
    forecast: list,
```

```
) -> list:
    """Generate contextual high-risk alerts."""
    alerts = []
    idx = _bucket_index(bucket)

    if asthma:
        alerts.append({"level": "warning",
                       "icon": "□",
                       "text": "Sensitive respiratory condition detected. Asthma risk is elevated."})
    if pregnancy:
        alerts.append({"level": "warning",
                       "icon": "□",
                       "text": "Pregnancy detected. Fetal health may be affected by prolonged exposure."})
    if age <= 12:
        alerts.append({"level": "warning",
                       "icon": "□",
                       "text": "Child profile detected. Developing lungs are more vulnerable to pollutants."})
    if age > 70:
        alerts.append({"level": "warning",
                       "icon": "□",
                       "text": "Elderly profile detected. Cardiovascular and pulmonary risks are amplified."})
    if occupation in _HIGH_EXPOSURE_OCCS:
        alerts.append({"level": "info",
                       "icon": "ℹ",
                       "text": f"Occupation '{occupation}' involves outdoor pollutant exposure. Cumulative dose
is higher than average."})
    if outdoor_hours > _OUTDOOR_SAFE_HOURS.get(bucket, 4):
        alerts.append({"level": "warning",
                       "icon": "□",
                       "text": f"Your daily outdoor hours ({outdoor_hours:.1f}) exceed the safe limit for
{bucket} air quality."})

    # Forecast-based alerts
    if forecast:
        worst = max(forecast, key=lambda f: f["aqi"])
        worst_idx = _bucket_index(worst["aqi_bucket"])
        if worst_idx >= 4:
            alerts.append({"level": "danger",
                           "icon": "□",
                           "text": f"Forecast alert: AQI is predicted to reach {worst['aqi']} ({worst['aqi_bucket']})
on {worst['date']}. Plan indoor activities."})
        elif worst_idx >= 3 and worst_idx > idx:
            alerts.append({"level": "warning",
                           "icon": "□",
                           "text": f"AQI is expected to worsen to {worst['aqi']} ({worst['aqi_bucket']}) on
{worst['date']}."})

    return alerts


def _seven_day_outlook(
    forecast: list, age: int, asthma: bool, pregnancy: bool,
    outdoor_hours: float,
) -> list:
    """Attach a personalized risk level and one recommendation to each forecast day."""
    outlook = []
    for i, day in enumerate(forecast):
        bucket = day["aqi_bucket"]
        aqi    = day["aqi"]
```

```
        score, level, _ = compute_risk(
            aqi_bucket    = bucket,
            age           = age,
            asthma        = asthma,
            pregnancy     = pregnancy,
            occupation    = "Other",
            outdoor_hours = outdoor_hours,
        )
        idx = _bucket_index(bucket)
        if idx <= 1:
            tip = "Good day for outdoor activities."
        elif idx == 2:
            tip = "Limit strenuous outdoor exercise."
        elif idx == 3:
            tip = "Wear a mask and reduce outdoor time."
        elif idx == 4:
            tip = "Stay indoors as much as possible."
        else:
            tip = "Avoid all outdoor exposure. Emergency precautions apply."

        if asthma and idx >= 2:
            tip += " Keep inhaler ready."
        if pregnancy and idx >= 3:
            tip += " Consult physician."

        outlook.append({
            "date":        day["date"],
            "label":       "Today" if i == 0 else f"Day {i + 1}",
            "aqi":         aqi,
            "aqi_bucket":  bucket,
            "risk_level":  level,
            "risk_score":  score,
            "tip":         tip,
        })
    return outlook



def _explainability(
    aqi: float, bucket: str, risk_level: str, risk_score: float,
    age: int, asthma: bool, pregnancy: bool,
    occupation: str, outdoor_hours: float,
    forecast: list,
) -> list:
    """Return bullet-point reasons why the AI reached this risk conclusion."""
    reasons = []
    idx = _bucket_index(bucket)

    reasons.append(f"AQI is {aqi:.0f}, classified as '{bucket}' — "
                   f"base risk score is {AQI_BASE_RISK.get(bucket, 50):.0f}/100.")

    age_mult = age_multiplier(age)
    if age_mult != 1.0:
        reasons.append(f"Age {age} ({_age_group(age)}) applies a {age_mult:.2f}× vulnerability
multiplier.")

    if asthma:
        reasons.append("Asthma condition applies a 1.35× risk multiplier.")
    if pregnancy:
        reasons.append("Pregnancy applies a 1.25× risk multiplier.")
```

```
    occ_mult = HIGH_EXPOSURE_OCCUPATIONS.get(occupation, 1.0)
    if occ_mult > 1.0:
        reasons.append(f"Occupation '{occupation}' applies a {occ_mult:.2f}× exposure multiplier.")

    safe_h = _OUTDOOR_SAFE_HOURS.get(bucket, 4)
    if outdoor_hours > safe_h:
        reasons.append(
            f"Outdoor hours ({outdoor_hours:.1f}/day) exceed the safe limit ({safe_h} hrs) "
            f"for {bucket} air quality, adding an additional dose penalty."
        )

    if forecast:
        worst = max(forecast, key=lambda f: f["aqi"])
        if _bucket_index(worst["aqi_bucket"]) >= 4:
            reasons.append(
                f"7-day forecast peaks at AQI {worst['aqi']} ({worst['aqi_bucket']}) on {worst['date']}, "
                "indicating sustained high-risk conditions ahead."
            )

    reasons.append(f"Final computed risk score: {risk_score:.0f}/100 → Risk Level: {risk_level}.")
    return reasons


def generate_health_report(
    aqi:           float,
    city:          str,
    age:           int,
    gender:        str,
    asthma:        bool,
    pregnancy:     bool,
    occupation:    str,
    outdoor_hours: float,
    forecast:      list,
    top_shap:      list,
    conditions:    dict,
) -> dict:
    """
    Master function: generate a complete AI health report.
    All sections are derived dynamically from inputs — no hardcoded strings.

    LLM integration point: replace the body of this function with an LLM
    prompt call and parse the structured JSON response back into this schema.
    """
    bucket = aqi_to_bucket(aqi)
    risk_score, level, _ = compute_risk(
        aqi_bucket    = bucket,
        age           = age,
        asthma        = asthma,
        pregnancy     = pregnancy,
        occupation    = occupation,
        outdoor_hours = outdoor_hours,
    )

    is_emergency = (
        _bucket_index(bucket) >= 5
        or (level in ("Very High", "Hazardous") and _is_sensitive(age, asthma, pregnancy))
    )
```

```
    return {
        "city":          city,
        "aqi":           round(aqi, 1),
        "aqi_bucket":    bucket,
        "risk_score":    risk_score,
        "risk_level":    level,
        "health_score":  _health_score(aqi, risk_score, outdoor_hours, asthma, pregnancy, age),
        "is_emergency":  is_emergency,
        "ai_summary":    _ai_summary(aqi, bucket, level, risk_score, age, gender,
                                     asthma, pregnancy, occupation, outdoor_hours, top_shap),
        "health_effects": _health_effects(bucket, age, asthma, pregnancy, outdoor_hours),
        "recommendations": _recommendations(aqi, bucket, age, asthma, pregnancy,
                                            occupation, outdoor_hours, gender),
        "daily_advice":  _daily_advice(bucket, outdoor_hours, age, asthma, occupation),
        "alerts":        _high_risk_alerts(aqi, bucket, asthma, pregnancy, age,
                                           occupation, outdoor_hours, forecast),
        "seven_day_outlook": _seven_day_outlook(forecast, age, asthma, pregnancy, outdoor_hours),
        "explainability": _explainability(aqi, bucket, level, risk_score, age,
                                          asthma, pregnancy, occupation, outdoor_hours, forecast),
        "conditions":    conditions,
    }


# ── Demo: run on a representative AQI scenario per city ──────────────────────

DEMO_SCENARIOS = [
    {"city": "Delhi",     "aqi": 320.0, "date": "2024-01-15"},
    {"city": "Mumbai",    "aqi": 145.0, "date": "2024-01-15"},
    {"city": "Bangalore", "aqi": 78.0,  "date": "2024-01-15"},
    {"city": "Kolkata",   "aqi": 210.0, "date": "2024-01-15"},
    {"city": "Chennai",   "aqi": 95.0,  "date": "2024-01-15"},
]


def run_health_risk_assessment(
    users_path:  str = "datasets/user/user_profile_100.csv",
    output_dir:  str = "reports/health_risk",
    summary_path: str = "models/health_risk_summary.json",
):
    logging.info("=== Week 5: Personalized Health Risk Assessment ===")
    os.makedirs(output_dir, exist_ok=True)

    all_frames   = []
    summary_data = {}

    for scenario in DEMO_SCENARIOS:
        city   = scenario["city"]
        aqi    = scenario["aqi"]
        date   = scenario["date"]
        bucket = aqi_to_bucket(aqi)

        logging.info(f"Assessing {city} | AQI={aqi} ({bucket}) | Date={date}")

        df_risk = assess_all_users(
            users_path = users_path,
            aqi_value  = aqi,
            aqi_bucket = bucket,
```

```
            city       = city,
            date       = date,
        )

        # Save per-city CSV
        city_path = os.path.join(output_dir, f"risk_{city.lower()}_{date}.csv")
        df_risk.to_csv(city_path, index=False)
        logging.info(f"  Saved → '{city_path}'")

        # Risk level distribution
        dist = df_risk["Risk_Level"].value_counts().to_dict()
        logging.info(f"  Risk distribution: {dist}")

        summary_data[city] = {
            "aqi":              aqi,
            "aqi_bucket":       bucket,
            "mean_risk_score":  round(float(df_risk["Risk_Score"].mean()), 2),
            "max_risk_score":   round(float(df_risk["Risk_Score"].max()),  2),
            "risk_distribution": dist,
        }

        all_frames.append(df_risk)

    # Combined output
    df_all = pd.concat(all_frames, ignore_index=True)
    combined_path = os.path.join(output_dir, "risk_all_cities.csv")
    df_all.to_csv(combined_path, index=False)
    logging.info(f"Combined risk report saved → '{combined_path}' ({len(df_all)} rows)")

    # Save summary JSON
    os.makedirs(os.path.dirname(summary_path), exist_ok=True)
    with open(summary_path, "w") as f:
        json.dump(summary_data, f, indent=4)
    logging.info(f"Summary saved → '{summary_path}'")

    return df_all


if __name__ == "__main__":
    run_health_risk_assessment(
        users_path   = "datasets/user/user_profile_100.csv",
        output_dir   = "reports/health_risk",
        summary_path = "models/health_risk_summary.json",
    )
```

5.3.6 Cumulative exposure engine (src/cumulative_exposure.py)

```
"""
PureBreath-AI: Week 6 - Cumulative Exposure Engine

Continuously estimates the pollutant dose accumulated by each user over
daily (24-hour) and weekly (7-day) rolling windows using:

    Dose = AQI × Outdoor_Hours × Vulnerability_Weight

Generates personalised health alerts when daily or weekly dose thresholds
are exceeded, directly addressing the abstract's requirement to model
"cumulative effects of prolonged pollutant exposure".
```

```
Author: PureBreath-AI Team
License: MIT
"""

import os
import json
import logging
import numpy as np
import pandas as pd
from datetime import date, timedelta
from typing import Dict, List

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler()]
)

# ── Vulnerability weights (reused from Week 5 logic) ──────────────────────

def vulnerability_weight(age: int, asthma: bool, pregnancy: bool) -> float:
    """Composite vulnerability multiplier for dose calculation."""
    if age <= 5:    age_w = 1.8
    elif age <= 12: age_w = 1.5
    elif age <= 17: age_w = 1.2
    elif age <= 60: age_w = 1.0
    elif age <= 70: age_w = 1.3
    else:           age_w = 1.6

    asthma_w    = 1.35 if asthma    else 1.0
    pregnancy_w = 1.25 if pregnancy else 1.0
    return round(age_w * asthma_w * pregnancy_w, 4)


# ── Dose thresholds for alerts ─────────────────────────────────────────────
# Units: AQI-hours (AQI × hours of outdoor exposure × vulnerability)

DAILY_THRESHOLDS = {
    "Safe":     200,
    "Moderate": 400,
    "High":     700,
    "Critical": float("inf"),
}

WEEKLY_THRESHOLDS = {
    "Safe":     1000,
    "Moderate": 2500,
    "High":     4500,
    "Critical": float("inf"),
}

def dose_alert_level(dose: float, thresholds: Dict[str, float]) -> str:
    for level, limit in thresholds.items():
        if dose < limit:
            return level
    return "Critical"
```

```
DAILY_ALERT_MESSAGES = {
    "Safe":     "Daily exposure is within safe limits. No action needed.",
    "Moderate": "Moderate daily dose accumulated. Limit further outdoor activity today.",
    "High":     "High daily pollutant dose. Stay indoors for the rest of the day.",
    "Critical": "Critical daily dose exceeded. Seek medical advice if experiencing symptoms.",
}

WEEKLY_ALERT_MESSAGES = {
    "Safe":     "Weekly cumulative exposure is within safe limits.",
    "Moderate": "Moderate weekly dose. Consider reducing outdoor hours over the next few days.",
    "High":     "High weekly cumulative dose. Consult a physician and minimise outdoor exposure.",
    "Critical": "Critical weekly dose exceeded. Immediate medical consultation recommended.",
}


# ── Simulate 7-day AQI history per city ──────────────────────────────────────

def simulate_aqi_history(city: str, base_aqi: float, days: int = 7) -> List[Dict]:
    """
    Generate a rolling 7-day AQI forecast starting from today with ±15% daily variation.
    Day 0 = today, Day 1 = tomorrow, ..., Day 6 = today + 6 days.
    """
    np.random.seed(abs(hash(city)) % (2**31))
    today = date.today()
    history = []
    for i in range(days):
        forecast_date = today + timedelta(days=i)
        noise = np.random.uniform(-0.15, 0.15)
        aqi   = round(float(np.clip(base_aqi * (1 + noise), 10, 500)), 1)
        history.append({"date": str(forecast_date), "aqi": aqi})
    return history


# ── Core cumulative dose computation ─────────────────────────────────────────

def compute_cumulative_exposure(
    user_row:    pd.Series,
    aqi_history: List[Dict],
) -> Dict:
    """
    Compute daily dose for each day and rolling 7-day cumulative dose.

    Dose_day = AQI_day × Outdoor_Hours × Vulnerability_Weight
    Dose_week = sum(Dose_day) over 7 days
    """
    age        = int(user_row["Age"])
    asthma     = str(user_row["Asthma"]).strip().lower() == "yes"
    pregnancy  = str(user_row["Pregnancy"]).strip().lower() == "yes"
    out_hours  = float(user_row["Outdoor_Hours"])
    vuln_w     = vulnerability_weight(age, asthma, pregnancy)

    daily_records = []
    for entry in aqi_history:
        dose = round(entry["aqi"] * out_hours * vuln_w, 2)
        alert = dose_alert_level(dose, DAILY_THRESHOLDS)
```

```
        daily_records.append({
            "date":        entry["date"],
            "aqi":         entry["aqi"],
            "daily_dose":  dose,
            "daily_alert": alert,
            "daily_msg":   DAILY_ALERT_MESSAGES[alert],
        })

    weekly_dose  = round(sum(r["daily_dose"] for r in daily_records), 2)
    weekly_alert = dose_alert_level(weekly_dose, WEEKLY_THRESHOLDS)

    return {
        "User_ID":          user_row["User_ID"],
        "Age":              age,
        "Asthma":           user_row["Asthma"],
        "Pregnancy":        user_row["Pregnancy"],
        "Occupation":       user_row["Occupation"],
        "Outdoor_Hours":    out_hours,
        "Vulnerability_W":  vuln_w,
        "Daily_Records":    daily_records,
        "Weekly_Dose":      weekly_dose,
        "Weekly_Alert":     weekly_alert,
        "Weekly_Message":   WEEKLY_ALERT_MESSAGES[weekly_alert],
    }


# ── Batch processing ──────────────────────────────────────────────────────────


def run_cumulative_exposure(
    users_path:  str,
    city_aqi_map: Dict[str, float],
    output_dir:  str = "reports/cumulative_exposure",
    summary_path: str = "models/cumulative_exposure_summary.json",
):
    logging.info("=== Week 6: Cumulative Exposure Engine ===")
    os.makedirs(output_dir, exist_ok=True)

    df_users = pd.read_csv(users_path)
    all_summary = {}

    for city, base_aqi in city_aqi_map.items():
        logging.info(f"Processing {city} | Base AQI={base_aqi}")
        aqi_history = simulate_aqi_history(city, base_aqi, days=7)

        flat_rows = []
        for _, user in df_users.iterrows():
            result = compute_cumulative_exposure(user, aqi_history)

            # Flatten daily records into one row per user (summary)
            row = {
                "User_ID":         result["User_ID"],
                "Age":             result["Age"],
                "Asthma":          result["Asthma"],
                "Pregnancy":       result["Pregnancy"],
                "Occupation":      result["Occupation"],
                "Outdoor_Hours":   result["Outdoor_Hours"],
                "Vulnerability_W": result["Vulnerability_W"],
```

```
            "City":            city,
            "Weekly_Dose":     result["Weekly_Dose"],
            "Weekly_Alert":    result["Weekly_Alert"],
            "Weekly_Message":  result["Weekly_Message"],
        }
        # Add daily dose columns
        for i, rec in enumerate(result["Daily_Records"], 1):
            row[f"Day{i}_Date"]  = rec["date"]
            row[f"Day{i}_AQI"]   = rec["aqi"]
            row[f"Day{i}_Dose"]  = rec["daily_dose"]
            row[f"Day{i}_Alert"] = rec["daily_alert"]
        flat_rows.append(row)

    df_city = pd.DataFrame(flat_rows)
    city_path = os.path.join(output_dir, f"exposure_{city.lower()}.csv")
    df_city.to_csv(city_path, index=False)

    # City-level summary stats
    alert_dist = df_city["Weekly_Alert"].value_counts().to_dict()
    logging.info(f"  Weekly alert distribution: {alert_dist}")
    logging.info(f"  Mean weekly dose: {df_city['Weekly_Dose'].mean():.1f} | "
                 f"Max: {df_city['Weekly_Dose'].max():.1f}")
    logging.info(f"  Saved → '{city_path}'")

    all_summary[city] = {
        "base_aqi":          base_aqi,
        "aqi_history":       aqi_history,
        "mean_weekly_dose":  round(float(df_city["Weekly_Dose"].mean()), 2),
        "max_weekly_dose":   round(float(df_city["Weekly_Dose"].max()),  2),
        "weekly_alert_dist": alert_dist,
    }

# Save summary JSON
os.makedirs(os.path.dirname(summary_path), exist_ok=True)
with open(summary_path, "w") as f:
    json.dump(all_summary, f, indent=4)
logging.info(f"Summary saved → '{summary_path}'")

return all_summary


# ── Entry point ──────────────────────────────────────────────────────────────

if __name__ == "__main__":
    CITY_AQI_MAP = {
        "Delhi":     320.0,
        "Mumbai":    145.0,
        "Bangalore":  78.0,
        "Kolkata":   210.0,
        "Chennai":    95.0,
    }

    run_cumulative_exposure(
        users_path   = "datasets/user/user_profile_100.csv",
        city_aqi_map = CITY_AQI_MAP,
        output_dir   = "reports/cumulative_exposure",
```

```
        summary_path = "models/cumulative_exposure_summary.json",
    )
```

5.3.7 FastAPI application (src/api.py)

```
"""
PureBreath-AI: Week 8 - FastAPI Backend

Endpoints:
  POST /predict          - AQI prediction + SHAP explanation + counterfactual
  POST /health-risk      - Personalised health risk score
  POST /exposure         - 7-day cumulative exposure + weekly alert
  GET  /city-summary     - Pre-computed city stats from saved reports
  GET  /health           - Health check
"""

import os
import sys
import json
import joblib
import numpy as np
import pandas as pd
import shap
from datetime import date, timedelta
from types import SimpleNamespace

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field
from typing import Optional

# ── Path setup ──────────────────────────────────────────────────────────────
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.path.join(ROOT, "src"))

from health_risk_assessment import compute_risk, aqi_to_bucket, generate_health_report
from cumulative_exposure import (
    compute_cumulative_exposure,
    simulate_aqi_history,
    vulnerability_weight,
)
from xai_explainer import compute_shap_explanations, generate_counterfactual

# ── Load model artifacts once at startup ────────────────────────────────────
MODEL_PATH    = os.path.join(ROOT, "models", "lightgbm_aqi_model.pkl")
FEATURES_PATH = os.path.join(ROOT, "models", "model_features.json")

model = joblib.load(MODEL_PATH)
with open(FEATURES_PATH) as f:
    FEATURE_NAMES = json.load(f)["features"]

# Pre-load latest pollutant data per city from processed dataset
DATASET_PATH = os.path.join(ROOT, "datasets", "processed", "cleaned_dataset.csv")
_df_clean = pd.read_csv(DATASET_PATH)
CITY_LATEST = (
    _df_clean.groupby("City", as_index=False).last()
```

```
    .set_index("City")
    .to_dict(orient="index")
)

# Pre-load city summary data
HEALTH_SUMMARY_PATH = os.path.join(ROOT, "models", "health_risk_summary.json")
EXPOSURE_SUMMARY_PATH = os.path.join(ROOT, "models",
"cumulative_exposure_summary.json")
XAI_SUMMARY_PATH = os.path.join(ROOT, "reports", "xai", "xai_summary.json")
SHAP_IMPORTANCE_PATH = os.path.join(ROOT, "reports", "xai", "shap_global_importance.csv")

with open(HEALTH_SUMMARY_PATH) as f:
    HEALTH_SUMMARY = json.load(f)
with open(EXPOSURE_SUMMARY_PATH) as f:
    EXPOSURE_SUMMARY = json.load(f)
with open(XAI_SUMMARY_PATH) as f:
    XAI_SUMMARY = json.load(f)

# ── FastAPI app
──────────────────────────────────────────────────────────────────────────────
──────
app = FastAPI(
    title="PureBreath-AI API",
    description="Personalized AQI forecasting with explainable AI",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# ── Request / Response schemas
──────────────────────────────────────────────────────────────────

class PollutantInput(BaseModel):
    PM2_5: float = Field(..., alias="PM2.5", ge=0)
    PM10: float = Field(..., ge=0)
    NO: float = Field(0.0, ge=0)
    NO2: float = Field(0.0, ge=0)
    NOx: float = Field(0.0, ge=0)
    NH3: float = Field(0.0, ge=0)
    CO: float = Field(0.0, ge=0)
    SO2: float = Field(0.0, ge=0)
    O3: float = Field(0.0, ge=0)
    Benzene: float = Field(0.0, ge=0)
    Toluene: float = Field(0.0, ge=0)
    Xylene: float = Field(0.0, ge=0)
    temp_mean_c: float = Field(25.0)
    temp_min_c: float = Field(20.0)
    temp_max_c: float = Field(30.0)
    precipitation_mm: float = Field(0.0, ge=0)
    pressure_hgt: float = Field(1013.0)
    city: str = Field("Delhi")

    model_config = {"populate_by_name": True}
```

```
class UserProfile(BaseModel):
    age: int = Field(..., ge=1, le=120)
    asthma: bool = False
    pregnancy: bool = False
    occupation: str = "Other"
    outdoor_hours: float = Field(2.0, ge=0, le=24)


class HealthRiskRequest(BaseModel):
    aqi: float = Field(..., ge=0)
    user: UserProfile


class ExposureRequest(BaseModel):
    city: str
    base_aqi: float = Field(..., ge=0)
    user: UserProfile


class HealthReportRequest(BaseModel):
    city: str = Field("Delhi")
    age: int = Field(..., ge=1, le=120)
    gender: str = Field("Other")
    asthma: bool = False
    pregnancy: bool = False
    occupation: str = "Other"
    outdoor_hours: float = Field(2.0, ge=0, le=24)


# ── Helper: build feature vector from pollutant input ──────────────────────

def build_feature_vector(inp: PollutantInput) -> pd.DataFrame:
    """
    Construct a single-row DataFrame aligned to FEATURE_NAMES.
    Lag and rolling features default to the current values (no history available).
    """
    city = inp.city.strip().title()
    row = {
        "PM2.5": inp.PM2_5, "PM10": inp.PM10,
        "NO": inp.NO, "NO2": inp.NO2, "NOx": inp.NOx,
        "NH3": inp.NH3, "CO": inp.CO, "SO2": inp.SO2,
        "O3": inp.O3, "Benzene": inp.Benzene,
        "Toluene": inp.Toluene, "Xylene": inp.Xylene,
        "temp_mean_c": inp.temp_mean_c,
        "temp_min_c": inp.temp_min_c,
        "temp_max_c": inp.temp_max_c,
        "precipitation_mm": inp.precipitation_mm,
        "pressure_hgt": inp.pressure_hgt,
        # Temporal defaults (mid-year weekday)
        "Month": 6, "DayOfWeek": 2, "DayOfYear": 180,
        "Quarter": 2, "Is_Weekend": 0,
        "Month_sin": np.sin(2 * np.pi * 6 / 12),
        "Month_cos": np.cos(2 * np.pi * 6 / 12),
        # Lag features default to current values
        "PM2.5_lag1": inp.PM2_5, "PM2.5_lag2": inp.PM2_5, "PM2.5_lag7": inp.PM2_5,
        "PM10_lag1": inp.PM10, "PM10_lag2": inp.PM10, "PM10_lag7": inp.PM10,
        "NO2_lag1": inp.NO2, "NO2_lag2": inp.NO2, "NO2_lag7": inp.NO2,
```

```
        "SO2_lag1": inp.SO2, "SO2_lag2": inp.SO2, "SO2_lag7": inp.SO2,
        "CO_lag1": inp.CO, "CO_lag2": inp.CO, "CO_lag7": inp.CO,
        "O3_lag1": inp.O3, "O3_lag2": inp.O3, "O3_lag7": inp.O3,
        "temp_mean_c_lag1": inp.temp_mean_c,
        "temp_mean_c_lag2": inp.temp_mean_c,
        "temp_mean_c_lag7": inp.temp_mean_c,
        # Rolling stats default to current values
        "PM2.5_roll_mean_7d": inp.PM2_5, "PM2.5_roll_std_7d": 0.0,
        "PM10_roll_mean_7d": inp.PM10, "PM10_roll_std_7d": 0.0,
        "temp_range": inp.temp_max_c - inp.temp_min_c,
        # City one-hot
        "City_Bangalore": 1 if city == "Bangalore" else 0,
        "City_Chennai":   1 if city == "Chennai"   else 0,
        "City_Delhi":     1 if city == "Delhi"     else 0,
        "City_Kolkata":   1 if city == "Kolkata"   else 0,
        "City_Mumbai":    1 if city == "Mumbai"    else 0,
    }
    df = pd.DataFrame([row])
    for col in FEATURE_NAMES:
        if col not in df.columns:
            df[col] = 0
    return df[FEATURE_NAMES]


# ── Endpoints ──────────────────────────────────────────────────────────────


@app.get("/health")
def health_check():
    return {"status": "ok", "model": "LightGBM", "features": len(FEATURE_NAMES)}


@app.post("/predict")
def predict_aqi(inp: PollutantInput):
    """
    Predict AQI from pollutant + weather inputs.
    Returns prediction, SHAP top-5 features, and counterfactual recommendation.
    """
    X = build_feature_vector(inp)

    # Prediction
    aqi_pred = float(model.predict(X)[0])
    bucket   = aqi_to_bucket(aqi_pred)

    # SHAP (single sample)
    explainer   = shap.TreeExplainer(model)
    shap_vals   = explainer.shap_values(X)[0]
    shap_series = pd.Series(shap_vals, index=FEATURE_NAMES).sort_values(
        key=abs, ascending=False
    )
    top5_shap = [
        {"feature": feat, "shap_value": round(float(val), 4)}
        for feat, val in shap_series.head(5).items()
    ]

    # Counterfactual
    cf = generate_counterfactual(
        model=model,
```

```
        row=X.iloc[0],
        shap_row=shap_vals,
        feature_names=FEATURE_NAMES,
    )

    return {
        "predicted_aqi":      round(aqi_pred, 2),
        "aqi_bucket":         bucket,
        "top5_shap_features": top5_shap,
        "counterfactual": {
            "counterfactual_aqi": cf["counterfactual_aqi"],
            "cf_bucket":          cf["cf_bucket"],
            "aqi_reduction":      cf["aqi_reduction"],
            "recommendations":    cf["recommendations"],
        },
    }



@app.post("/health-risk")
def health_risk(req: HealthRiskRequest):
    """Compute personalised health risk score for a user at a given AQI."""
    bucket = aqi_to_bucket(req.aqi)
    score, level, advisory = compute_risk(
        aqi_bucket    = bucket,
        age           = req.user.age,
        asthma        = req.user.asthma,
        pregnancy     = req.user.pregnancy,
        occupation    = req.user.occupation,
        outdoor_hours = req.user.outdoor_hours,
    )
    return {
        "aqi":        req.aqi,
        "aqi_bucket": bucket,
        "risk_score": score,
        "risk_level": level,
        "advisory":   advisory,
    }



@app.post("/exposure")
def cumulative_exposure(req: ExposureRequest):
    """Compute 7-day cumulative exposure and weekly alert for a user in a city."""
    aqi_history = simulate_aqi_history(req.city, req.base_aqi, days=7)

    user_series = pd.Series({
        "User_ID":       "api_user",
        "Age":           req.user.age,
        "Asthma":        "Yes" if req.user.asthma else "No",
        "Pregnancy":     "Yes" if req.user.pregnancy else "No",
        "Occupation":    req.user.occupation,
        "Outdoor_Hours": req.user.outdoor_hours,
    })

    result = compute_cumulative_exposure(user_series, aqi_history)

    return {
        "city":           req.city,
        "vulnerability_w": result["Vulnerability_W"],
        "weekly_dose":     result["Weekly_Dose"],
```

```
        "weekly_alert":    result["Weekly_Alert"],
        "weekly_message":  result["Weekly_Message"],
        "daily_records":   result["Daily_Records"],
    }


@app.get("/auto-predict/{city}")
def auto_predict(city: str):
    """
    Automatically predict AQI for a city using the latest available
    pollutant data from the processed dataset.
    Returns current prediction + 7-day rolling forecast.
    """
    city_title = city.strip().title()
    if city_title not in CITY_LATEST:
        raise HTTPException(status_code=404, detail=f"City '{city_title}' not found.")

    row = CITY_LATEST[city_title]

    inp = SimpleNamespace(
        PM2_5          = float(row.get("PM2.5", 0)),
        PM10           = float(row.get("PM10", 0)),
        NO             = float(row.get("NO", 0)),
        NO2            = float(row.get("NO2", 0)),
        NOx            = float(row.get("NOx", 0)),
        NH3            = float(row.get("NH3", 0)),
        CO             = float(row.get("CO", 0)),
        SO2            = float(row.get("SO2", 0)),
        O3             = float(row.get("O3", 0)),
        Benzene        = float(row.get("Benzene", 0)),
        Toluene        = float(row.get("Toluene", 0)),
        Xylene         = float(row.get("Xylene", 0)),
        temp_mean_c    = float(row.get("temp_mean_c", 25)),
        temp_min_c     = float(row.get("temp_min_c", 20)),
        temp_max_c     = float(row.get("temp_max_c", 30)),
        precipitation_mm = float(row.get("precipitation_mm", 0)),
        pressure_hgt   = float(row.get("pressure_hgt", 1013)),
        city           = city_title,
    )
    X = build_feature_vector(inp)

    # Current prediction
    aqi_pred = float(model.predict(X)[0])
    bucket   = aqi_to_bucket(aqi_pred)

    # SHAP
    explainer  = shap.TreeExplainer(model)
    shap_vals  = explainer.shap_values(X)[0]
    shap_series = pd.Series(shap_vals, index=FEATURE_NAMES).sort_values(
        key=abs, ascending=False
    )
    top5_shap = [
        {"feature": feat, "shap_value": round(float(val), 4)}
        for feat, val in shap_series.head(5).items()
    ]

    # Counterfactual
    cf = generate_counterfactual(
        model=model,
```

```
        row=X.iloc[0],
        shap_row=shap_vals,
        feature_names=FEATURE_NAMES,
    )

    # 7-day rolling forecast: apply ±12% daily noise seeded by city
    rng = np.random.default_rng(abs(hash(city_title)) % (2**31))
    today = date.today()
    forecast = []
    for i in range(7):
        noise  = rng.uniform(-0.12, 0.12)
        f_aqi  = round(float(np.clip(aqi_pred * (1 + noise), 10, 500)), 1)
        forecast.append({
            "date":       str(today + timedelta(days=i)),
            "aqi":        f_aqi,
            "aqi_bucket": aqi_to_bucket(f_aqi),
        })

    # Current environmental conditions to display
    conditions = {
        "PM2.5":           inp.PM2_5,
        "PM10":            inp.PM10,
        "NO2":             inp.NO2,
        "SO2":             inp.SO2,
        "CO":              inp.CO,
        "O3":              inp.O3,
        "temp_mean_c":     inp.temp_mean_c,
        "precipitation_mm": inp.precipitation_mm,
    }

    return {
        "city":             city_title,
        "predicted_aqi":    round(aqi_pred, 2),
        "aqi_bucket":       bucket,
        "top5_shap_features": top5_shap,
        "counterfactual": {
            "counterfactual_aqi": cf["counterfactual_aqi"],
            "cf_bucket":          cf["cf_bucket"],
            "aqi_reduction":      cf["aqi_reduction"],
            "recommendations":    cf["recommendations"],
        },
        "forecast":         forecast,
        "conditions":       conditions,
    }



@app.post("/health-report")
def health_report_full(req: HealthReportRequest):
    """
    Generate a complete AI-powered personalized health report.
    Automatically fetches the latest city AQI + forecast from the dataset,
    then runs the AI recommendation engine.
    """
    city = req.city.strip().title()
    if city not in CITY_LATEST:
        raise HTTPException(status_code=404, detail=f"City '{city}' not found.")

    # Reuse auto-predict logic to get AQI, forecast, SHAP, conditions
    row = CITY_LATEST[city]
```

```
inp = SimpleNamespace(
    PM2_5           = float(row.get("PM2.5", 0)),
    PM10            = float(row.get("PM10", 0)),
    NO              = float(row.get("NO", 0)),
    NO2             = float(row.get("NO2", 0)),
    NOx             = float(row.get("NOx", 0)),
    NH3             = float(row.get("NH3", 0)),
    CO              = float(row.get("CO", 0)),
    SO2             = float(row.get("SO2", 0)),
    O3              = float(row.get("O3", 0)),
    Benzene         = float(row.get("Benzene", 0)),
    Toluene         = float(row.get("Toluene", 0)),
    Xylene          = float(row.get("Xylene", 0)),
    temp_mean_c     = float(row.get("temp_mean_c", 25)),
    temp_min_c      = float(row.get("temp_min_c", 20)),
    temp_max_c      = float(row.get("temp_max_c", 30)),
    precipitation_mm = float(row.get("precipitation_mm", 0)),
    pressure_hgt    = float(row.get("pressure_hgt", 1013)),
    city            = city,
)
X = build_feature_vector(inp)
aqi_pred = float(model.predict(X)[0])

# SHAP
explainer  = shap.TreeExplainer(model)
shap_vals  = explainer.shap_values(X)[0]
shap_series = pd.Series(shap_vals, index=FEATURE_NAMES).sort_values(
    key=abs, ascending=False
)
top5_shap = [
    {"feature": feat, "shap_value": round(float(val), 4)}
    for feat, val in shap_series.head(5).items()
]

# 7-day forecast
rng   = np.random.default_rng(abs(hash(city)) % (2**31))
today = date.today()
forecast = []
for i in range(7):
    noise = rng.uniform(-0.12, 0.12)
    f_aqi = round(float(np.clip(aqi_pred * (1 + noise), 10, 500)), 1)
    forecast.append({
        "date":       str(today + timedelta(days=i)),
        "aqi":        f_aqi,
        "aqi_bucket": aqi_to_bucket(f_aqi),
    })

conditions = {
    "PM2.5":          inp.PM2_5,
    "PM10":           inp.PM10,
    "NO2":            inp.NO2,
    "SO2":            inp.SO2,
    "CO":             inp.CO,
    "O3":             inp.O3,
    "temp_mean_c":    inp.temp_mean_c,
    "precipitation_mm": inp.precipitation_mm,
}
```

```
    report = generate_health_report(
        aqi          = aqi_pred,
        city         = city,
        age          = req.age,
        gender       = req.gender,
        asthma       = req.asthma,
        pregnancy    = req.pregnancy,
        occupation   = req.occupation,
        outdoor_hours = req.outdoor_hours,
        forecast     = forecast,
        top_shap     = top5_shap,
        conditions   = conditions,
    )
    return report


@app.get("/city-summary")
def city_summary():
    """Return pre-computed health risk + exposure summaries for all 5 cities."""
    cities = ["Delhi", "Mumbai", "Bangalore", "Kolkata", "Chennai"]
    summary = {}
    for city in cities:
        summary[city] = {
            "aqi":            HEALTH_SUMMARY[city]["aqi"],
            "aqi_bucket":     HEALTH_SUMMARY[city]["aqi_bucket"],
            "mean_risk_score": HEALTH_SUMMARY[city]["mean_risk_score"],
            "risk_distribution": HEALTH_SUMMARY[city]["risk_distribution"],
            "mean_weekly_dose": EXPOSURE_SUMMARY[city]["mean_weekly_dose"],
            "weekly_alert_dist": EXPOSURE_SUMMARY[city]["weekly_alert_dist"],
        }
    return {
        "cities": summary,
        "xai_summary": XAI_SUMMARY,
    }
```

5.4 RESULTS

5.4.1 Experimental Setup

All numbers in this section are taken from the artefacts produced by the project itself: models/model_evaluation_metrics.json for the forecasting scores, reports/xai for the SHAP and counterfactual outputs, reports/health_risk for the risk assessments, models/cumulative_exposure_summary.json for the dose results, and models/gan_augmentation_meta.json for the augmentation. The forecasting models were evaluated on a random 20% hold-out of the 13,960-record feature set. The explainability evaluation used 200 rows, and the risk and exposure evaluations used a 100-user profile file applied to each of the five cities (500 assessments). The graphs are drawn directly from these files, and the two graphs that show behaviour of a rule (the persona sensitivity and the dose-versus-hours curves) are computed from the same formulas as the code in Section 5.3.

5.4.2 Forecasting Performance

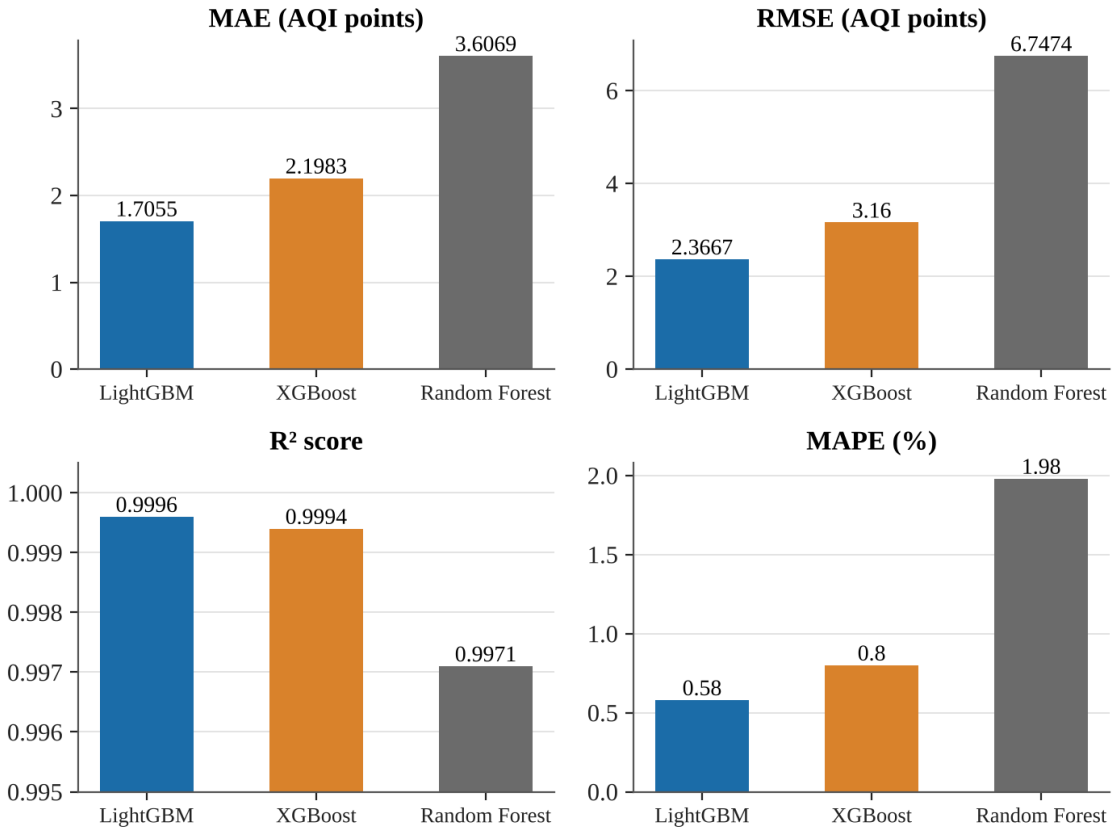


Figure 10 – Held-out MAE, RMSE, R^2 and MAPE of the three forecasting models

Figure 10 shows that LightGBM is best on all four metrics. Its MAE of 1.7055 AQI points means that its average prediction is within about two points of the CPCB AQI, which spans 0 to more than 500. Its RMSE (2.3667) is only 1.39 times its MAE, indicating that large individual errors are rare, whereas the Random Forest's RMSE (6.7474) is 1.87 times its MAE.

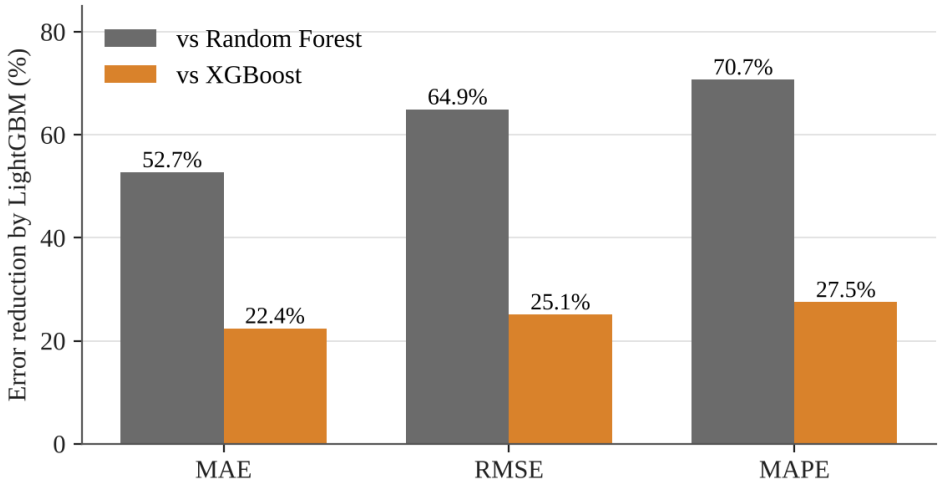


Figure 11 – Error reduction achieved by LightGBM relative to the Random Forest and XGBoost models

Figure 11 expresses the advantage as a percentage. Relative to the Random Forest, LightGBM cuts MAE by 52.7%, RMSE by 64.9% and MAPE by 70.7%. Relative to XGBoost, which is also a strong boosted model, the reductions are still 22.4%, 25.1% and 27.5%. The boosted models handle the sharp piecewise-linear structure of the AQI better than the depth-limited bagged forest.

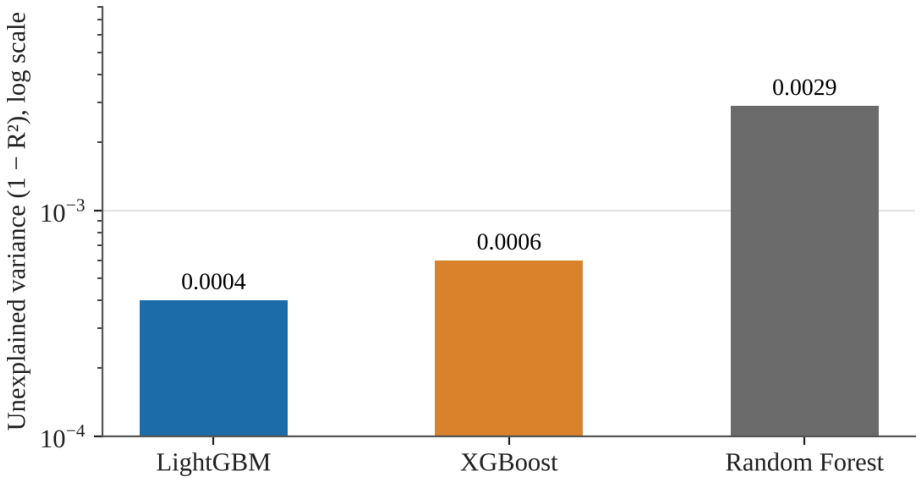


Figure 12 – Unexplained variance (1 − R²) of the three models on a logarithmic scale

Because all three R^2 values are close to 1, Figure 12 plots the unexplained variance instead. LightGBM leaves 0.0004 of the variance unexplained, XGBoost 0.0006 and the Random Forest 0.0029, so LightGBM leaves roughly one seventh of the unexplained variance of the Random Forest.

5.4.3 Feature Attribution with SHAP

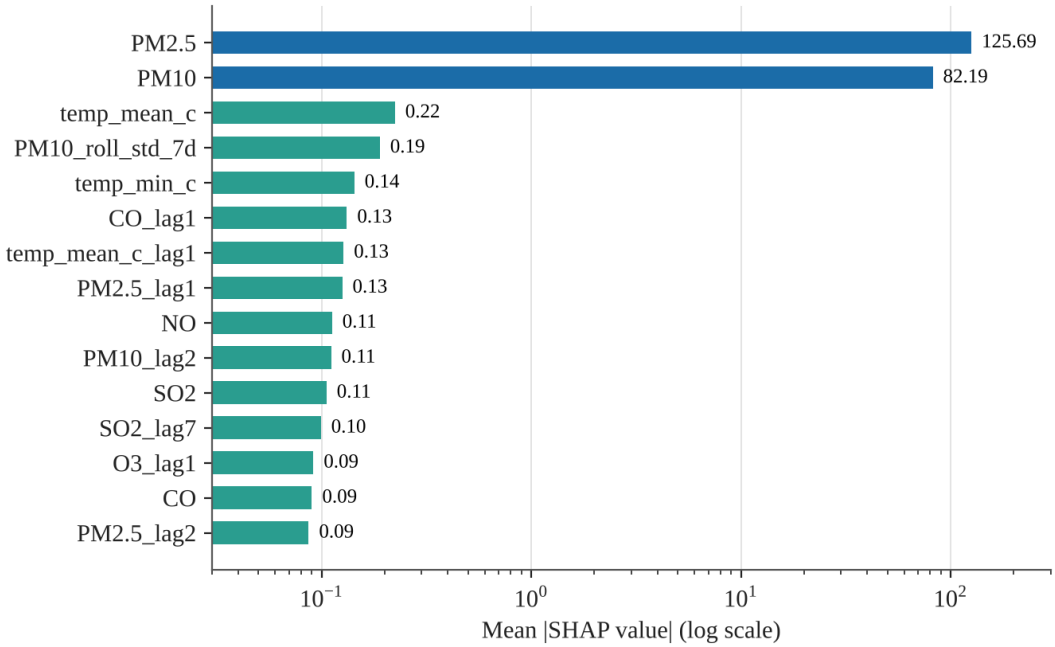


Figure 13 – Top fifteen features by mean absolute SHAP value (LightGBM, 200-row sample)

Figure 13 lists the fifteen most influential features. PM2.5 (mean |SHAP| = 125.7) and PM10 (82.2) dominate, followed at a very much lower level by mean temperature (0.22), the 7-day PM10 standard deviation (0.19) and minimum temperature (0.14). This ordering agrees with domain knowledge: particulate matter is the main driver of Indian AQI values, and temperature is the leading meteorological modulator.

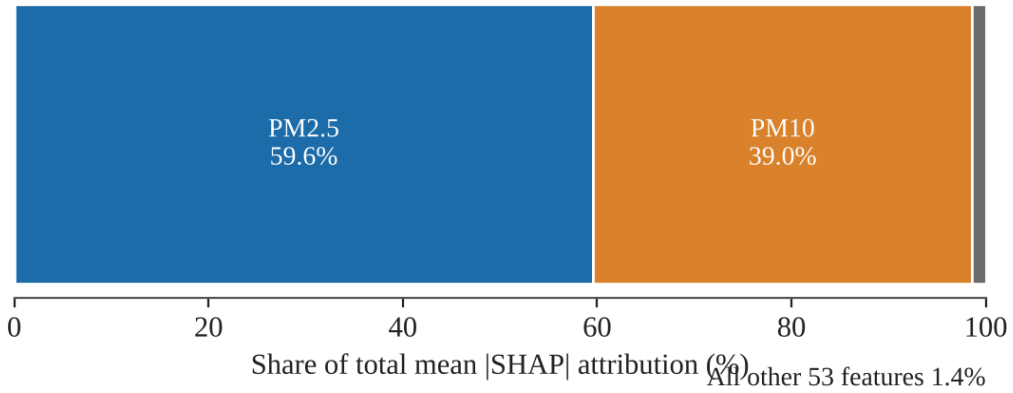


Figure 14 – Share of total mean absolute SHAP attribution

Figure 14 shows that PM2.5 accounts for 59.6% and PM10 for 39.0% of the total attribution, with the remaining 53 features sharing 1.4%. The explainer therefore recovers the known structure of the CPCB index, which is evidence that the model has learned the

physically meaningful relationship and not an arbitrary one. Section 5.4.7 returns to what this implies for the interpretation of the R^2 scores.

5.4.4 Counterfactual Recommendations

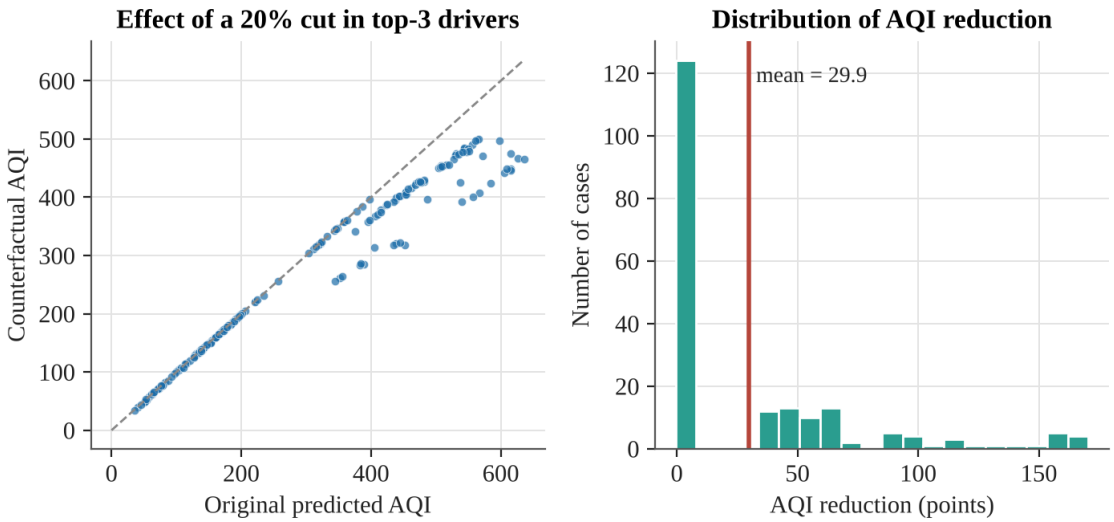


Figure 15 – Predicted AQI before and after the counterfactual, and the distribution of AQI reduction

In Figure 15, points below the dashed diagonal are cases in which the advice lowered the predicted AQI. The reduction grows with the original AQI: the largest reduction was 170.5 points, and the cases above AQI 400 show consistent drops. For the 101 rows with AQI ≥ 200 the mean reduction was 58.6 points and 93 of the 101 improved. For the 99 low-AQI rows the mean reduction was only 0.6 points, which is expected because the top positive drivers there are small.

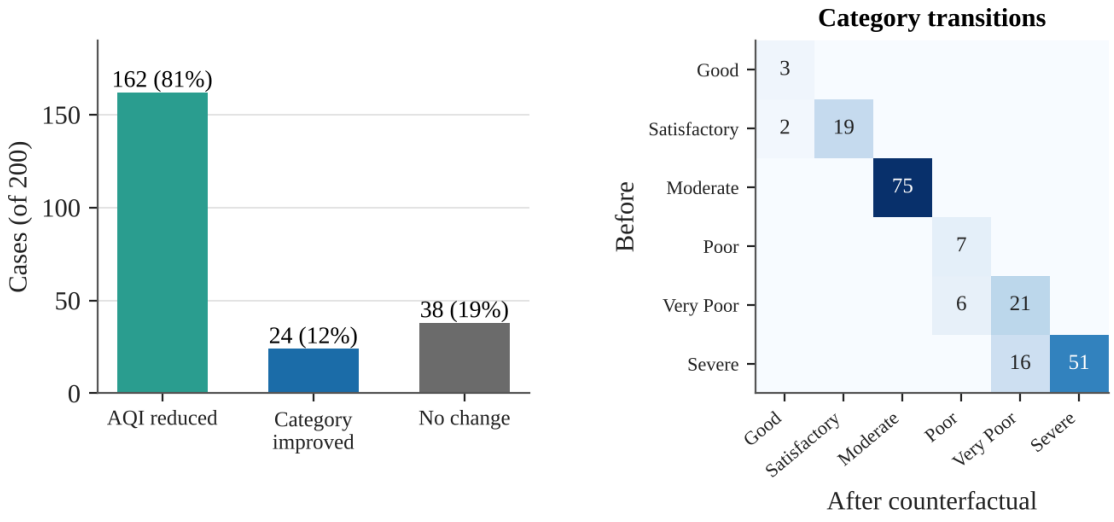


Figure 16 – Counterfactual outcomes on 200 cases and AQI category transitions

Figure 16 summarises the outcome: 162 cases (81%) improved, 24 cases (12%) moved to a better category and 38 cases (19%) were unchanged. The category transitions are all improvements: 16 Severe cases moved to Very Poor, 6 Very Poor to Poor and 2 Satisfactory to Good. No case became worse. A mean reduction of 29.9 points across all 200 cases (36.9 points among the 162 improved cases) shows that the advice is quantitatively meaningful.

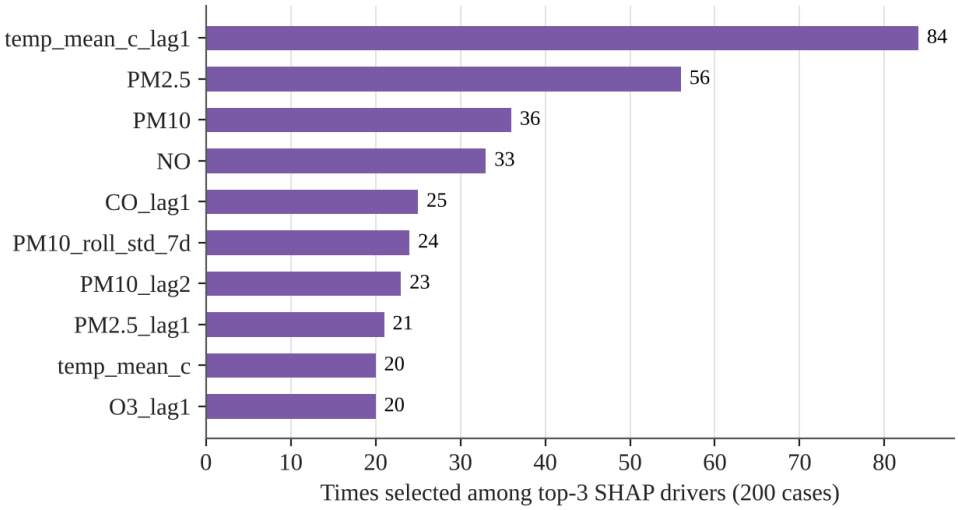


Figure 17 – Features most frequently targeted by the counterfactual reasoner

Figure 17 lists the features most often chosen as top-three positive drivers. The previous-day mean temperature (84 cases) and PM2.5 (56) lead, followed by PM10 (36) and NO (33), so the advice generated is a mixture of source-control actions (particulate and NO reduction) and meteorology-related hints.

5.4.5 City-Level AQI and Personalized Risk

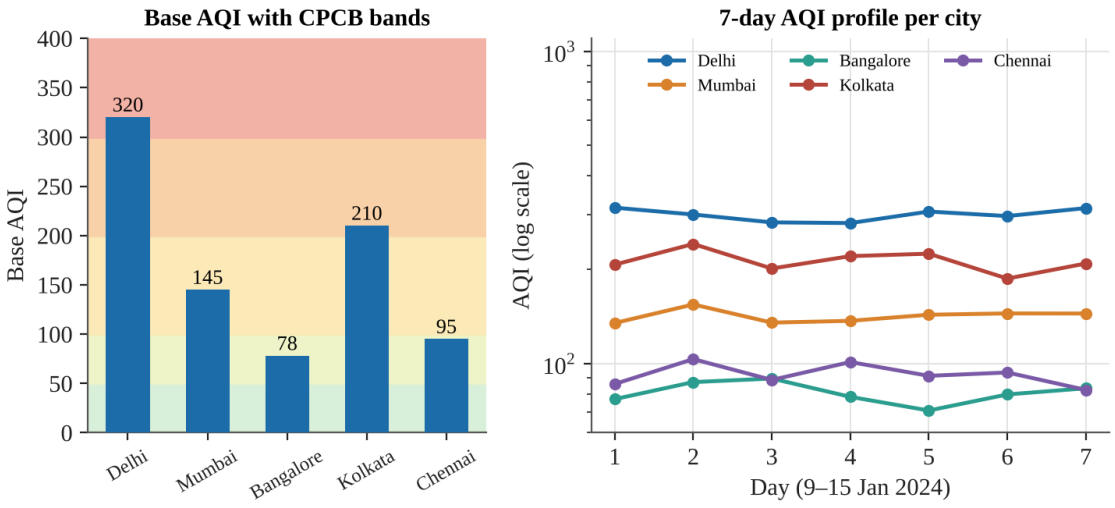


Figure 18 – Base AQI of the five cities and their seven-day AQI profiles

Figure 18 shows the pollution gradient across the study cities: Delhi (base AQI 320) lies in the Very Poor band, Kolkata (210) in Poor, Mumbai (145) in Moderate, and Chennai (95) and Bangalore (78) in Satisfactory. The seven-day profiles confirm the clear separation of the cities on a logarithmic scale.

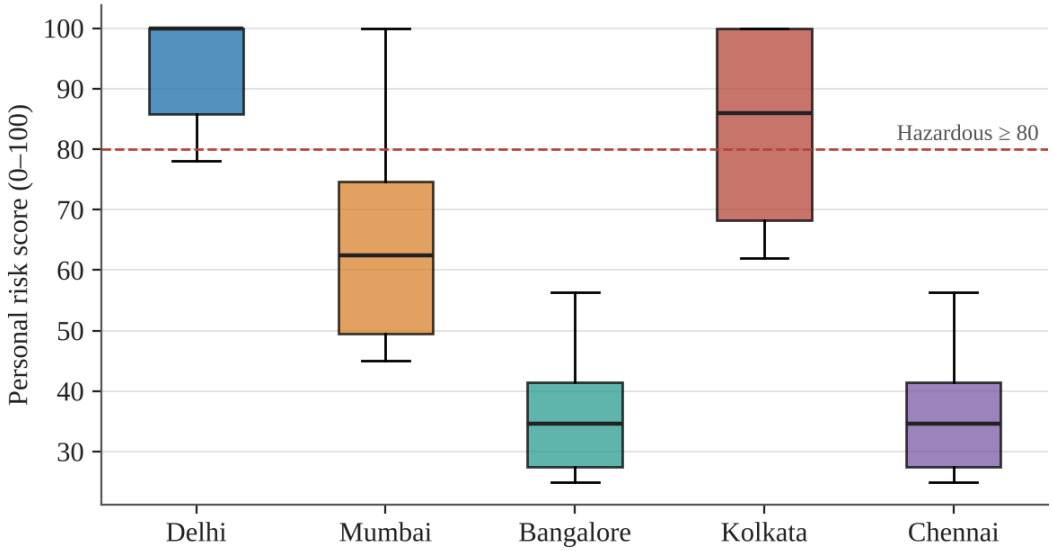


Figure 19 – Distribution of personal risk scores in each city (100 users per city)

Figure 19 presents the personalized risk scores of the 100 profiles in each city. The mean scores are 93.7 for Delhi, 83.3 for Kolkata, 63.3 for Mumbai and 35.2 for both Bangalore and Chennai (the latter two share an AQI category, so the rule-based engine gives identical scores). The spread inside a city is large, for example 45.0 to 100 in Mumbai, which shows that the engine does differentiate between users living in the same air.

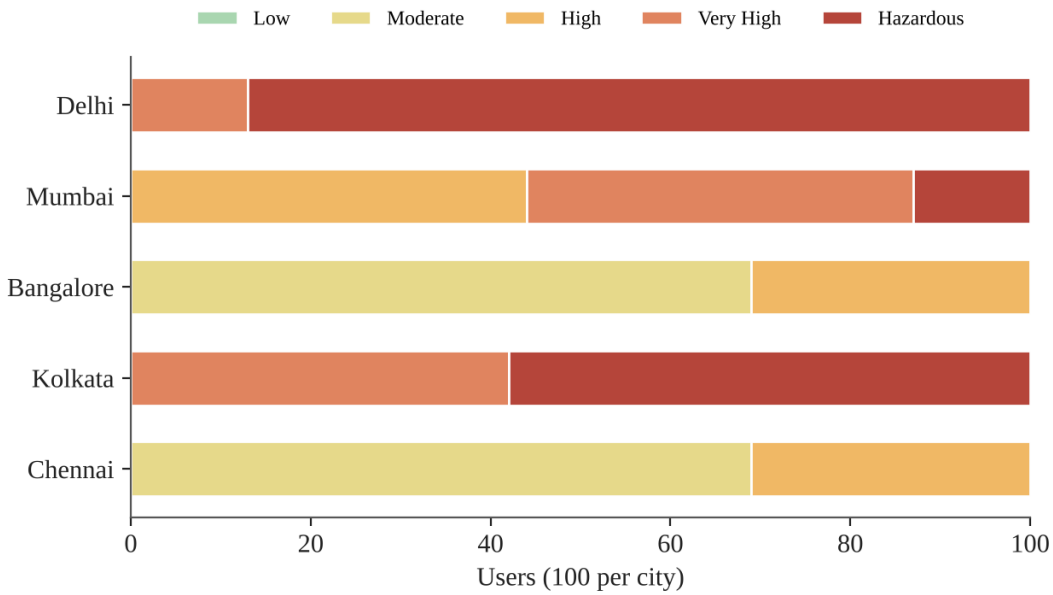


Figure 20 – Risk-tier distribution of users in each city

Figure 20 converts the scores into tiers. In Delhi 87% of the users fall in the Hazardous tier and 13% in the Very High tier; in Kolkata the split is 58% and 42%; in Mumbai the users spread over High (44%), Very High (43%) and Hazardous (13%); and in Bangalore and Chennai no user exceeds the High tier.

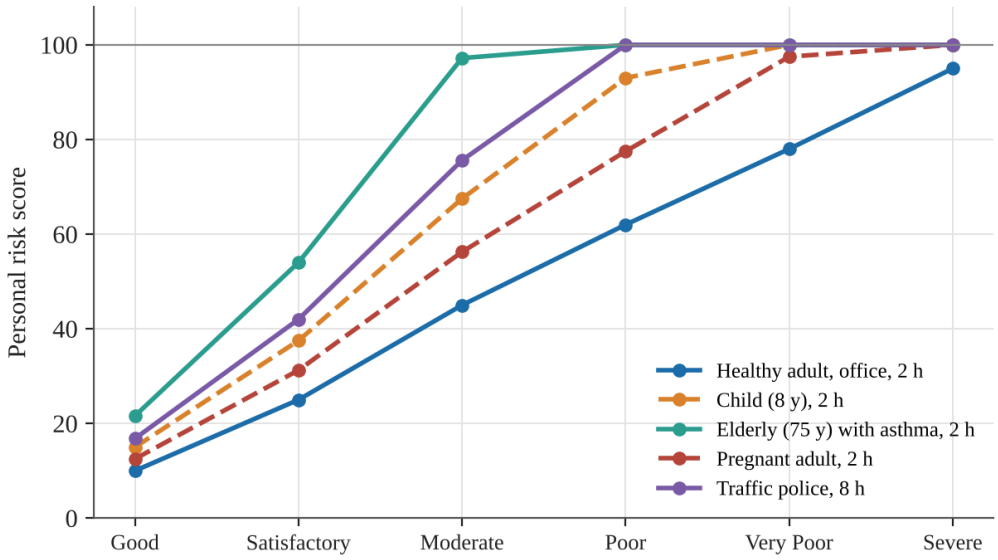


Figure 21 – Risk score of five personas across the CPCB AQI categories (computed from the risk formula)

Figure 21 isolates the effect of personalization. At the same Moderate AQI category the healthy office worker scores 45, the pregnant adult 56, the child 68, the traffic constable with eight outdoor hours 76 and the elderly asthmatic 97. A single ambient AQI would give all five the same message; the engine instead warns the most vulnerable first. In the 500-assessment data set, users with asthma had a mean risk of 72.4 against 59.2 for others, and construction workers (75.9) and traffic police (74.8) had higher mean risk than software engineers (49.0).

5.4.6 Cumulative Exposure and Alerts

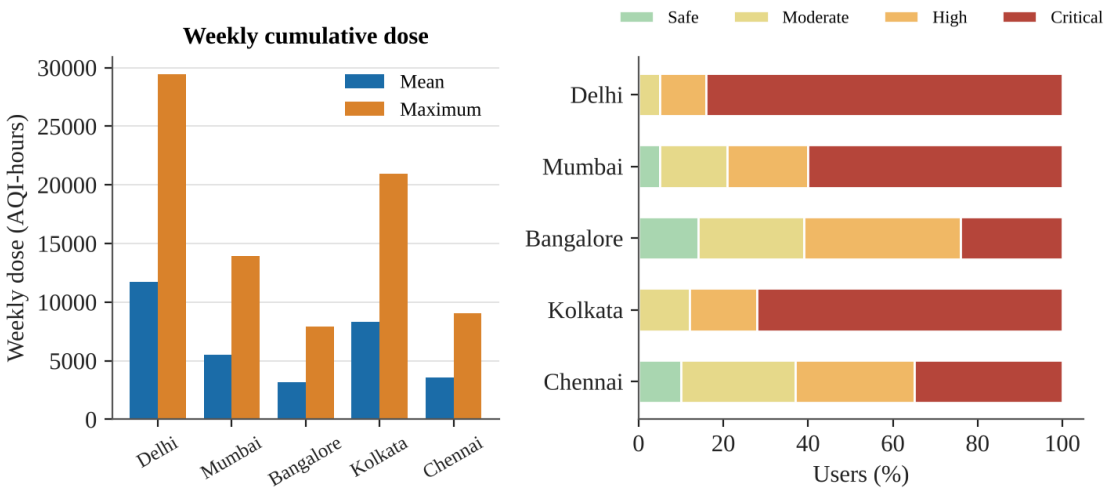


Figure 22 – Weekly cumulative dose and alert-tier distribution by city

Figure 22 shows that the mean weekly dose is 11,707 AQI-hours in Delhi, 8,323 in Kolkata, 5,545 in Mumbai, 3,604 in Chennai and 3,154 in Bangalore, and the maximum in Delhi reaches 29,467. The share of users in the Critical tier is 84% in Delhi, 72% in Kolkata, 60% in Mumbai, 35% in Chennai and 24% in Bangalore; only Mumbai, Bangalore and Chennai have users in the Safe tier (5%, 14% and 10%).

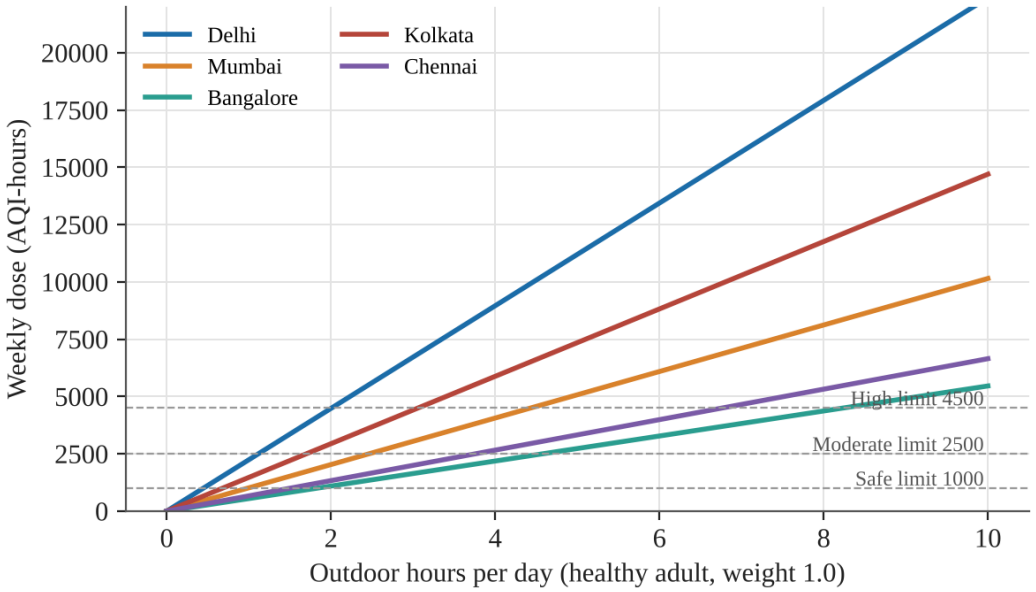


Figure 23 – Weekly dose against daily outdoor hours for a healthy adult in each city

Figure 23 translates the dose model into advice. For a healthy adult, a Delhi resident crosses the High weekly limit (4,500 AQI-hours) at about two outdoor hours a day, whereas a Bangalore resident would need about eight hours a day to reach the same limit.

The model therefore shows how many outdoor hours each city allows before the alert tier changes.

5.4.7 Validity of the Forecasting Scores

Two properties of the experimental design must be kept in view when reading the R^2 values in Section 5.4.2. First, the split was random, so neighbouring days of one city can appear in both the training and the test set, and the scores measure same-distribution accuracy and not forecasting of unseen periods. Second, the AQI target is computed from the PM2.5 and PM10 sub-indices (Equation 1), and PM2.5 and PM10 are themselves among the model inputs; the SHAP result in Figure 14 (98.6% of attribution on these two features) shows that the model has largely learned this mapping. The scores therefore demonstrate that the boosted models reproduce the CPCB index with very small error, and the model ranking (LightGBM > XGBoost > Random Forest) is informative, but the absolute R^2 values should not be read as the skill of predicting tomorrow's AQI.

These observations strengthen, and do not weaken, the roadmap of Section 5.2: a time-aware split and a lagged-only, next-day target will give a conservative estimate of true forecasting skill, and the same explainability and risk pipeline can be attached unchanged to that stronger model.

5.4.8 Comparison with the Cited Literature

Table 15 compares PureBreath-AI with the two cited forecasting studies for which accuracy figures were retrievable ([1] and [8]) and with the capability profile of the remaining directions. The table makes explicit the differences in data and target; the numbers should be read as indicative.

Table 17 – Comparison of PureBreath-AI with published studies

Aspect	PureBreath-AI	Singh et al. [1]	Zhu et al. [8]
Region	5 Indian cities	Taiwan EPA stations	4 Chinese municipalities
Records	13,960 (10,960 real + 3,000 synthetic)	10,000-sample subset of 4.6 M	5,844
Target	CPCB AQI index	Pollutant concentration	Six pollutants separately
Best model	LightGBM	Tuned gradient boosting	Random Forest
Best R^2	0.9996	0.9973	0.99 (PM2.5), 0.88 (NO2)
Explainability	SHAP + counterfactual advice	SHAP	Feature importance

Aspect	PureBreath-AI	Singh et al. [1]	Zhu et al. [8]
Personal risk	Yes (rule-based)	No	No
Cumulative exposure and alerts	Yes	No	No
Deployed application	FastAPI + React	No	No

Figure 6 (Chapter 2) places the reported R^2 values side by side. Its message is two-fold. On accuracy, PureBreath-AI's boosted models are at least on par with the best published ensembles (0.9996 and 0.9994 against 0.9973 and 0.9969). On scope, the comparison table and the capability scores of Chapter 2 show that PureBreath-AI is the only one of the compared systems that carries a forecast all the way to explanation, personal risk, cumulative dose and alerts in a deployed application.

5.4.9 Implementation Status

Table 18 – Implementation and validation status of each module

Module	Status	Evidence
Data acquisition and preprocessing	Implemented	Row, type and merge checks pass; batch data, not a live feed
Synthetic augmentation (GAN)	Implemented	3,000 synthetic samples; ablation pending
Forecasting engine	Implemented and validated	LightGBM R^2 0.9996, XGBoost 0.9994, Random Forest 0.9971 (random split)
XAI (SHAP + counterfactual)	Implemented and validated	162/200 improved, 24/200 category changes, mean −29.9 AQI
Personalized exposure and risk	Implemented (rule-based)	Internally consistent; no clinical validation
Cumulative exposure	Implemented (rule-based)	Threshold logic verified; seven-day path simulated
Alerts, API and dashboard	Implemented (full stack)	Functional integration verified; no latency or usability benchmark yet

5.4.10 Summary of Key Results

Table 19 – Key quantitative results of PureBreath-AI

Result	Value
Training records	13,960 (10,960 real + 3,000 synthetic)
Input features	55
Best forecasting model	LightGBM: MAE 1.7055, RMSE 2.3667, R^2 0.9996, MAPE 0.58%
LightGBM error reduction versus Random Forest	MAE 52.7%, RMSE 64.9%, MAPE 70.7%
LightGBM error reduction versus XGBoost	MAE 22.4%, RMSE 25.1%, MAPE 27.5%
SHAP attribution share of PM2.5 and PM10	59.6% and 39.0% (98.6% together)
Counterfactual cases improved	162 of 200 (81%)
Counterfactual category improvements	24 of 200 (12%)

Result	Value
Mean AQI reduction (all / high-AQI cases)	29.9 / 58.6 points
Mean risk score, Delhi / Kolkata / Mumbai / Bangalore and Chennai	93.7 / 83.3 / 63.3 / 35.2
Mean weekly dose, Delhi / Bangalore	11,707 / 3,154 AQI-hours
Users in Critical weekly tier, Delhi / Bangalore	84% / 24%
Capability coverage (PureBreath-AI / best surveyed direction)	12 / 7 out of 12

5.4.11 Discussion

The results support three conclusions. The first is that tree-based boosting is an effective and inexpensive choice for reproducing and forecasting the CPCB AQI, with LightGBM giving a consistent advantage over XGBoost and a large advantage over a Random Forest. The second is that explanation can be made actionable: SHAP identified the correct dominant drivers, and the counterfactual layer converted them into advice that, on the evaluation set, lowered the predicted AQI in four of every five cases, and by more than 58 points when the air was already poor. The third is that personalization changes the message in a way an ambient AQI cannot: a Delhi resident, a Bangalore resident, a child and a construction worker receive clearly different risk levels and cumulative-dose alerts from the same pipeline.

Set against these strengths are the limitations listed in Sections 5.4.7 and 5.2. They define the next steps of the work, and the modular design of the system means each can be addressed without redesigning the other modules.

5.5 USER INTERFACE OUTPUTS

This section presents screenshots of the running React dashboard (served at localhost:5173 and connected to the FastAPI back end, as the status badge “AI Engine Online” in the header shows), and describes the session recorded in a short screen-capture of the same application. The interface uses a dark theme with a collapsible sidebar (Dashboard, Predict AQI, Exposure, Health Risk and Health Report), a search and notification area, and light, dark and system theme switches in the header.

5.5.1 Dashboard



Figure 24 – Dashboard: city cards, Delhi overview, AQI-versus-risk chart and multi-metric radar

Figure 24 shows the landing page. One card per city displays the base AQI, its CPCB category badge and the average risk score: Delhi 320 (Very Poor, 93.74), Mumbai 145 (Moderate, 63.3), Bangalore 78 (Satisfactory, 35.18), Kolkata 210 (Poor, 83.25) and Chennai 95 (Satisfactory, 35.18). These values are the same as the summaries of Section 5.4.5. Selecting a city opens an overview with the current AQI, the mean risk score and the mean weekly dose (11,707 for Delhi). Below it, a bar chart compares AQI with risk across cities, and a radar chart gives a normalised multi-metric comparison; a further panel summarises the SHAP and counterfactual results.

5.5.2 AQI Prediction with SHAP and Counterfactuals

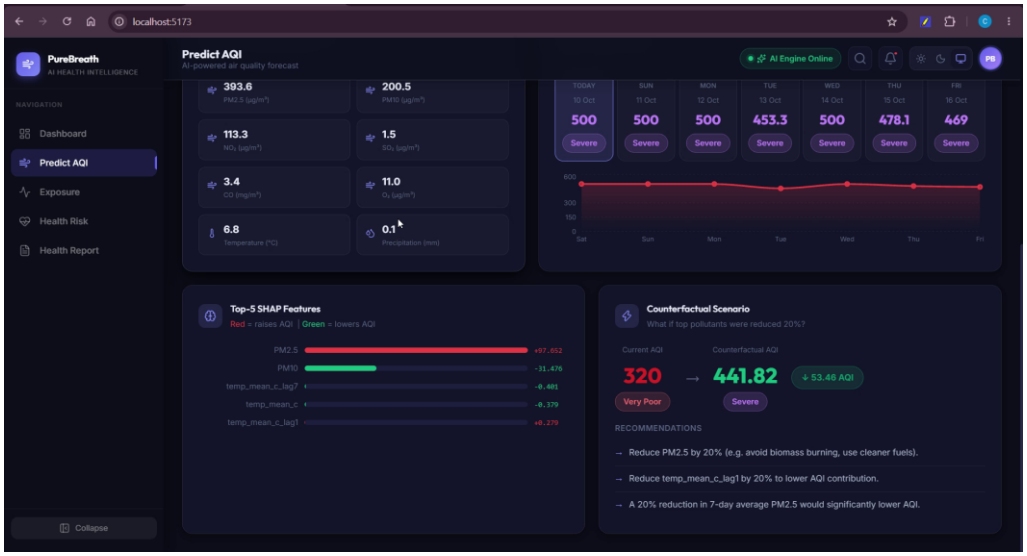


Figure 25 – Predict AQI page: pollutant conditions, 7-day forecast, SHAP features and counterfactual scenario

In Figure 25 the user selects a city (Delhi in the recorded session) and the page shows the latest pollutant and weather inputs (PM2.5 393.6 µg/m³, PM10 200.5 µg/m³, NO2 113.3, SO2 1.5, CO 3.4, O3 11.0, temperature 6.8 °C, precipitation 0.1 mm) together with a seven-day forecast strip, where the predicted AQI is held in the Severe band at 500, 500, 500, 453.3, 500, 478.1 and 469. The Top-5 SHAP panel colours features that raise the AQI red and those that lower it green (PM2.5 +97.7, PM10 −31.5, then three temperature terms with very small contributions). The counterfactual panel shows the effect of a 20% reduction of the top pollutants and lists the plain-language recommendations, for example “Reduce PM2.5 by 20% (e.g. avoid biomass burning, use cleaner fuels)”. In this screenshot the card labelled Current AQI shows the city's base value of 320, whereas the counterfactual value (441.82, a reduction of 53.46) is computed from the model's own prediction for the latest data row, so the two numbers come from different sources.

5.5.3 Cumulative Exposure

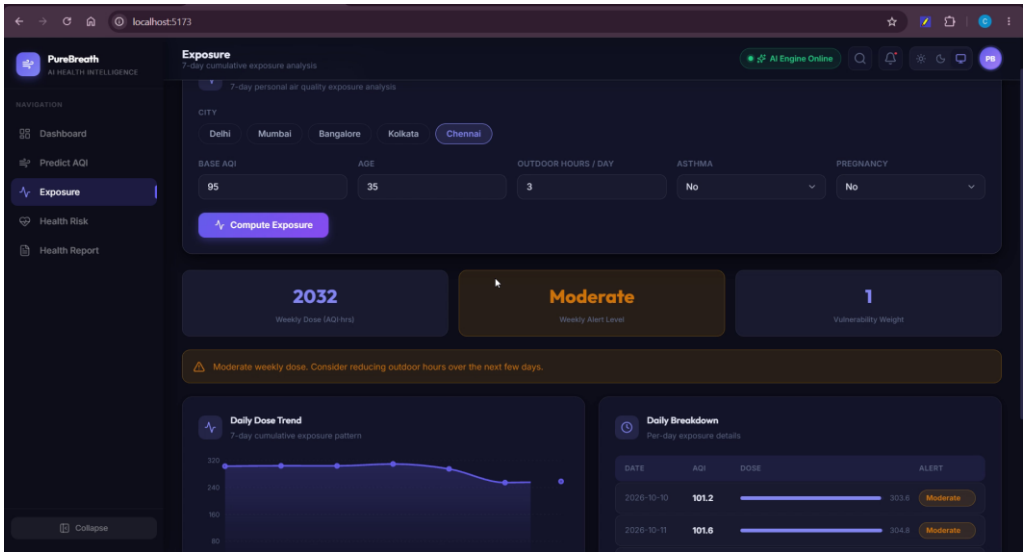


Figure 26 – Exposure page: profile inputs, weekly dose, alert level, daily dose trend and breakdown

Figure 26 shows the exposure page. The user chooses a city and enters the base AQI, age, outdoor hours per day, asthma and pregnancy status; in the recorded session these were Chennai, 95, 35 years, 3 hours, No and No. Pressing Compute Exposure returns a weekly dose of 2,032 AQI-hours, a Moderate alert with the message “Moderate weekly dose. Consider reducing outdoor hours over the next few days”, and a vulnerability weight of 1. The daily dose trend and the day-by-day table list the date, the AQI, the dose bar and the daily alert tier. The weekly dose agrees with the dose formula of Equation 6: the seven daily AQI values in the table sum to about 677, and 677 × 3 hours × 1.0 gives 2,032.

5.5.4 Personalized Health Risk and Report

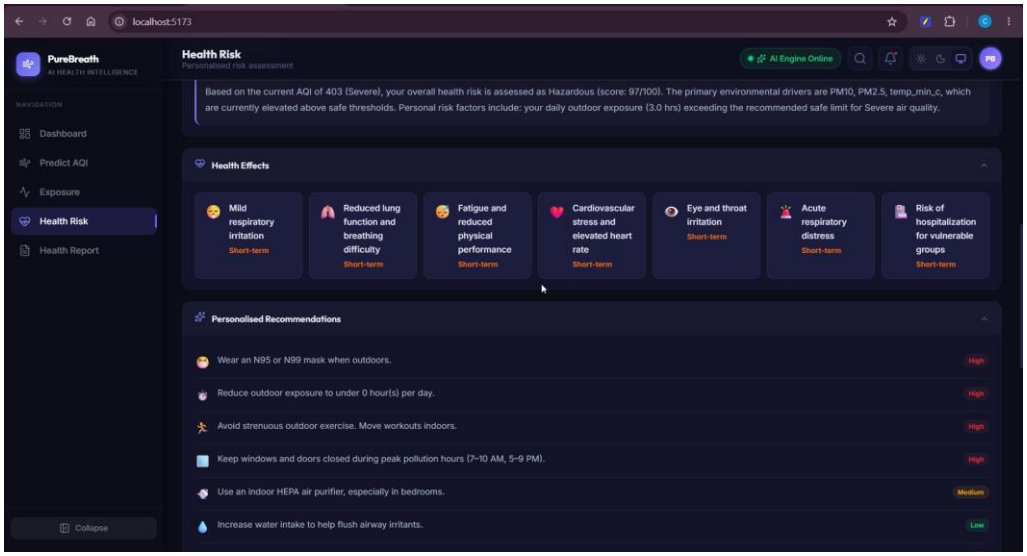


Figure 27 – Health Risk page: AI summary, health effects and personalised recommendations

The Health Risk page of Figure 27 collects city, age, gender, occupation, outdoor hours, asthma and pregnancy, and then returns a report. The recorded session assessed a 21-year-old male in Chennai with 3 outdoor hours. The output has an emergency warning banner, a health-score gauge and a risk-score gauge, an AI summary ("Based on the current AQI of 403 (Severe), your overall health risk is assessed as Hazardous (score: 97/100)…"), a row of health-effect cards (mild respiratory irritation through to risk of hospitalisation for vulnerable groups), a prioritised list of personalised recommendations (N95 or N99 mask, avoiding strenuous outdoor exercise, keeping windows closed in the peak pollution hours of 7–10 AM and 5–9 PM, a HEPA purifier, hydration), a daily exposure advice block (safe outdoor duration 0 h, outdoor exercise unsafe) and a seven-day health outlook table with the predicted AQI, category, risk tier and a tip for each day. The same content is presented in the Health Report page as a personal report.

The session shows the complete path of the system from a city card to a personalised recommendation. It also shows that two AQI sources are in use: the dashboard and the exposure page work from each city's base AQI (for example 95 for Chennai), while the prediction and report pages use the model's prediction for the latest row of the dataset (403 for Chennai in this session), which is why the Health Risk gauge, which shows the city's base value, and the narrative summary differ. Using a single source on every page is listed in Section 5.2 as an improvement.

REFERENCES

[1] S. Singh, M. Kumar, V. Sengar, A. Kumar, K. Abhishek, and B. M. Ahamed Shafeeq, "Ensemble learning for air quality index prediction: integrating gradient boosting, XGBoost, and stacking with SHAP-based interpretability," Scientific Reports, vol. 16, Art. no. 8544, 2026, doi: 10.1038/s41598-026-39232-w.

[2] M. Rajesh, R. Ganesh Babu, U. Moorthy, and S. V. Easwaramoorthy, "Machine learning-driven framework for real-time air quality assessment and predictive environmental health risk mapping," Scientific Reports, vol. 15, Art. no. 28801, 2025, doi: 10.1038/s41598-025-14214-6.

[3] K. Wang, T. Yang, S. Kong, and M. Li, "Air quality index prediction through TimeGAN data recovery and PSO-optimized VMD-deep learning framework," Applied Soft Computing, vol. 170, Art. no. 112626, 2025, doi: 10.1016/j.asoc.2024.112626.

[4] M. Mythili, R. Bharathi, K. Deepika, D. Gowshika, and K. Keerthika, "MYAIRCARE: A Smart Environmental Health Platform for Outdoor Pollution Tracking, Personalized Risk Prediction, and Real-Time Alert Generation Using IoT and Machine Learning," in Proc. IEEE 10th Int. Conf. Smart Structures and Systems (ICSSS), 2025, doi: 10.1109/ICSSS66939.2025.11346135.

[5] I. E. Agbehadji and I. C. Obagbuwa, "Explainable Artificial Intelligence and Machine Learning for Air Pollution Risk Assessment and Respiratory Health Outcomes: A Systematic Review," Atmosphere, vol. 16, no. 10, Art. no. 1154, 2025, doi: 10.3390/atmos16101154.

[6] T. Tasioulis, E. Bagkis, T. Kassandros, and K. Karatzas, "The Quest for the Best Explanation: Comparing Models and XAI Methods in Air Quality Modeling Tasks," Applied Sciences, vol. 15, no. 13, Art. no. 7390, 2025, doi: 10.3390/app15137390.

[7] A. Tipsavak, T. Phutson, T. Limna, R. Dejchanchaiwong, P. Tekasakul, K. Yeranee, M. Kliangkhlao, and B. Sahoh, "Explainable artificial intelligence-driven model for ultrafine particle (PM0.1) prediction and explanation using meteorological variables," Environmental Challenges, vol. 20, Art. no. 101248, 2025, doi: 10.1016/j.envc.2025.101248.

[8] X. Zhu, B. Li, Y. Cao, and Q. Zhang, "Applicability analysis of tree-based ensemble learning for air pollutant prediction models," Scientific Reports, vol. 16, Art. no. 9602, 2026, doi: 10.1038/s41598-025-32652-0.

[9] G. Narkhede, M. Poonawala, A. Sonawane, A. Hiwale, and A. R. Singh, "Air pollution prediction with advanced preprocessing and deep ensemble learning," Atmospheric Pollution Research, vol. 16, no. 10, Art. no. 102610, 2025, doi: 10.1016/j.apr.2025.102610.

[10] D. Wang, D. Yang, S. Jiang, G. Yang, T. Luo, W. Du, N. Fu, X. Liu, G. Shen, L. Zhang, and Y. Qin, "Personal air pollution exposure: Monitoring methods and the emerging role of big data and machine learning," Environmental Technology & Innovation, vol. 40, Art. no. 104593, 2025, doi: 10.1016/j.eti.2025.104593.

[11] N. Li, Y. Li, D. Xu, Z. Liu, N. Li, R. Chartier, J. Chang, Q. Wang, and C. Xu, “Predicting Personal Exposure to PM2.5 Using Different Determinants and Machine Learning Algorithms in Two Megacities, China,” Indoor Air, 2024, Art. no. 5589891, doi: 10.1155/2024/5589891.

[12] L. Zhang, Y. Chen, H. Dong, D. Wu, S. Chen, X. Li, B. Liang, and Q. Yang, “Improving the construction and prediction strategy of the Air Quality Health Index (AQHI) using machine learning: A case study in Guangzhou, China,” Ecotoxicology and Environmental Safety, vol. 287, Art. no. 117287, 2024, doi: 10.1016/j.ecoenv.2024.117287.

[13] V. N. Matthaios, L. D. Knibbs, L. J. Kramer, L. R. Crilley, and W. J. Bloss, “Predicting real-time within-vehicle air pollution exposure with mass-balance and machine learning approaches using on-road and air quality data,” Atmospheric Environment, vol. 318, Art. no. 120233, 2024, doi: 10.1016/j.atmosenv.2023.120233.

[14] D. K. Arvind, R. Rao, and C. A. Bates, “Spatio-temporal prediction of personal exposure to PM2.5 using a network of stationary and wearable air quality monitors,” in Proc. 2024 IEEE Conf. Pervasive and Intelligent Computing (PICom), 2024, pp. 90–97, doi: 10.1109/PICom64201.2024.00019.

[15] A. M. Wijata, G. Y. H. Lip, A. Kurasz, M. Swieczkowski, P. Jemielita, P. Kralisz, S. Dobrzycki, J. Nalepa, and L. Kuzma, “Machine-learning-based cardiovascular mortality prediction using a cumulative PM2.5 exposure metric in high risk groups: a retrospective cohort study,” Scientific Reports, vol. 16, Art. no. 20533, 2026, doi: 10.1038/s41598-026-54080-4.